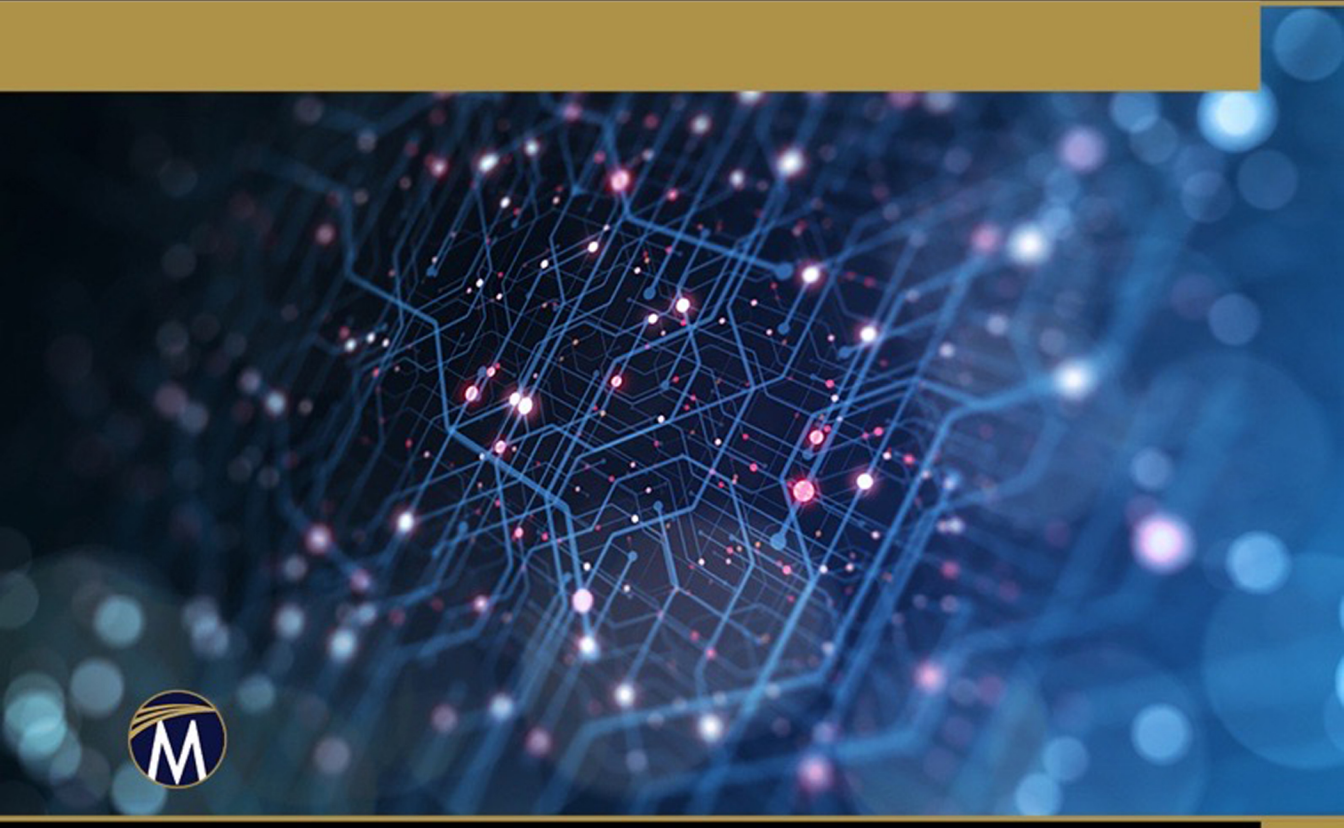
# Python 3 and Feature Engineering



Oswald Campesato

# Python 3
# and
# Feature Engineering

**LICENSE, DISCLAIMER OF LIABILITY, AND LIMITED WARRANTY**

By purchasing or using this book and companion files (the "Work"), you agree that this license grants permission to use the contents contained herein, including the disc, but does not give you the right of ownership to any of the textual content in the book / disc or ownership to any of the information or products contained in it. *This license does not permit uploading of the Work onto the Internet or on a network (of any kind) without the written consent of the Publisher*. Duplication or dissemination of any text, code, simulations, images, etc. contained herein is limited to and subject to licensing terms for the respective products, and permission must be obtained from the Publisher or the owner of the content, etc., in order to reproduce or network any portion of the textual material (in any media) that is contained in the Work.

Mercury Learning and Information ("MLI" or "the Publisher") and anyone involved in the creation, writing, or production of the companion disc, accompanying algorithms, code, or computer programs ("the software"), and any accompanying Web site or software of the Work, cannot and do not warrant the performance or results that might be obtained by using the contents of the Work. The author, developers, and the Publisher have used their best efforts to ensure the accuracy and functionality of the textual material and/or programs contained in this package; we, however, make no warranty of any kind, express or implied, regarding the performance of these contents or programs. The Work is sold "as is" without warranty (except for defective materials used in manufacturing the book or due to faulty workmanship).

The author, developers, and the publisher of any accompanying content, and anyone involved in the composition, production, and manufacturing of this work will not be liable for damages of any kind arising out of the use of (or the inability to use) the algorithms, source code, computer programs, or textual material contained in this publication. This includes, but is not limited to, loss of revenue or profit, or other incidental, physical, or consequential damages arising out of the use of this Work.

The sole remedy in the event of a claim of any kind is expressly limited to replacement of the book and/or disc, and only at the discretion of the Publisher. The use of "implied warranty" and certain "exclusions" vary from state to state, and might not apply to the purchaser of this product.

*Companion files for this title are available by writing to the publisher at info@merclearning.com.*

# PYTHON 3
# AND
# FEATURE ENGINEERING

Oswald Campesato



MERCURY LEARNING AND INFORMATION
Boston, Massachusetts

Copyright ©2024 by Mercury Learning and Information LLC.
An Imprint of DeGruyter Inc. All rights reserved.

*This publication, portions of it, or any accompanying software may not be reproduced in any way, stored in a retrieval system of any type, or transmitted by any means, media, electronic display or mechanical display, including, but not limited to, photocopy, recording, Internet postings, or scanning, without prior permission in writing from the publisher.*

Publisher: David Pallai
Mercury Learning and Information
121 High Street, 3rd Floor
Boston, MA 02110
info@merclearning.com
www.merclearning.com
800-232-0223

O. Campesato. *Python 3 and Feature Engineering.*
ISBN: 978-1-68392-949-9

The publisher recognizes and respects all marks used by companies, manufacturers, and developers as a means to distinguish their products. All brand names and product names mentioned in this book are trademarks or service marks of their respective companies. Any omission or misuse (of any kind) of service marks or trademarks, etc. is not an attempt to infringe on the property of others.

Library of Congress Control Number: 2023947146
232425321 This book is printed on acid-free paper in the United States of America.

Our titles are available for adoption, license, or bulk purchase by institutions, corporations, etc. For additional information, please contact the Customer Service Dept. at 800-232-0223(toll free).

All of our titles are available in digital format at academiccourseware.com and other digital vendors. *Companion files (figures and code listings) for this title are available by contacting info@merclearning.com*. The sole obligation of Mercury Learning and Information to the purchaser is to replace the disc, based on defective materials or faulty workmanship, but not based on the operation or functionality of the product.

*I'd like to dedicate this book to my parents*
*– may this bring joy and happiness into their lives.*

# CONTENTS

*Preface* *xiii*

**Chapter 1: Working With Datasets** **1**

Exploratory Data Analysis (EDA) 2
EDA Code Sample: Titanic 2
EDA and Histograms 6
Dealing With Data: What Can Go Wrong? 7
Datasets 8
Explanation of Data Types 9
Binary Data 9
Nominal Data 9
Ordinal Data 10
Categorical Data 10
Interval Data 10
Ratio Data 10
Continuous Data Versus Discrete Data 10
Random Variables 11
Qualitative and Quantitative Data 11
Types of Statistical Data 11
Data Preprocessing 12
Working With Data Types 13
Data Drift 14
What Is Data Leakage? 14
Data Leakage and Differential Privacy 15
Model Selection and Preparing Datasets 15
Model Selection 15
Discrete Data Versus Continuous Data 16
"Binning" Data Values 16
Programmatic Binning Techniques 17
Potential Issues When Binning Data Values 18
Handling Categorical Data 18

Processing Inconsistent Categorical Data 18
Mapping Categorical Data to Numeric Values 19
Types of Dependencies Among Features 20
Homoskedasticity and Heteroskedasticity 20
Collinearity 21
Variance Inflation Factor 22
Multicollinearity 22
Correlation 22
Working With Currency 22
Working With Dates 26
Splitting and Scaling Data 30
Why Normalize Data? 30
Split Before Normalizing Data 31
Scaling Numeric Data via Normalization 31
Scaling Numeric Data to the Range [a,b] 32
Scaling Numeric Data via Standardization 32
The StandardScaler Class 33
Scaling Numeric Data via Robust Standardization 34
Deciding How to Scale Data 34
Summary 35

**Chapter 2: Outlier and Anomaly Detection 37**

Working With Outliers 37
Outliers Versus Data Drift 37
Outlier Detection/Removal 38
Incorrectly Scaled Values Versus Outliers 39
Other Outlier Techniques 39
Finding Outliers With Numpy 40
Finding Outliers With Pandas 42
Calculating Z-Scores to Find Outliers 45
Finding Outliers With SkLearn (Optional) 47
Fraud Detection 48
Techniques for Anomaly Detection 49
Summary 50

**Chapter 3: Data Cleaning Tasks 51**

What Is Data Cleaning? 51
Data Cleaning for Personal Titles 52
Data Cleaning in SQL 54
Replace NULL With 0 54
Replace NULL Values With Average Value 54
Replace Multiple Values With a Single Value 56
Handle Mismatched Attribute Values 57
Convert Strings to Date Values 58

Data Cleaning From the Command Line (Optional) 60
Working With the sed Utility 60
Working With Variable Column Counts 62
Truncating Rows in CSV Files 63
Generating Rows With Fixed Columns With the awk Utility 64
Converting Phone Numbers 66
Converting Numeric Date Formats 68
Converting Alphabetic Date Formats 72
Working With Date and Time Date Formats 74
Working With Codes, Countries, and Cities 80
Data Cleaning on a Kaggle Dataset 86
Summary 89

**Chapter 4: Data Wrangling 91**
What Is Data Wrangling? 91
Data Transformation: What Does This Mean? 92
CSV Files With Multi-Row Records 93
Pandas Solution (1) 94
Pandas Solution (2) 95
CSV Solution 95
CSV Files, Multi-Row Records, and the awk Command 96
Quoted Fields Split on Two Lines (Optional) 97
Overview of the Events Project 100
Why This Project? 101
Project Tasks 101
Generate Country Codes 102
Prepare List of Cities in Countries 102
Generating City Codes From Country Codes: awk 103
Generating City Codes From Country Codes: Python 106
Generating SQL Statements for the city_codes Table 108
Generating a CSV File for Band Members (Java) 109
Generating a CSV File for Band Members (Python) 113
Generating a Calendar of Events (COE) 115
Project Automation Script 118
Project Follow-Up Comments 120
Summary 121

**Chapter 5: Feature Selection 123**
What Is Feature Selection? 123
Three Types of Feature Selection Methods 123
Filter Methods 124
Variance Threshold 124
Chi-Squared Test 124
ANOVA F-test 124

Mutual Information 125
Correlation Coefficient 125
Wrapper Methods 125
Recursive Feature Elimination (RFE) 126
Recursive Feature Elimination With Cross-Validation (RFECV) 126
Sequential Feature Selection (SFS) 126
Backward Feature Elimination 126
Boruta 127
Embedded Methods 127
L1 Regularization (Lasso) 127
Decision Trees (and Tree-Based Models) 128
Elastic Net 128
LightGBM 128
Linear Models with Recursive Feature Elimination 129
The Need for Feature Scaling and Transformations 129
Labeled, Unlabeled, and Multiclass Classification 130
Labeled Versus Unlabeled Data 130
Working With Imbalanced Datasets 130
Detecting Imbalanced Data 131
Rebalancing Datasets 132
Specify Stratify in Data Splits 132
Feature Importance 132
What Is SMOTE? 134
SMOTE Extensions 135
An Alternative to SMOTE 135
What Are Transforms? 136
Cube Root Transformation 137
Other Transformations 138
Summary 139

**Chapter 6: Feature Engineering 141**

What Is Feature Engineering? 141
Types of Feature Engineering 142
What Steps Are Required to Train a Model? 143
Machine Learning and Algorithm Selection 144
Training Large Datasets 145
Feature Importance 145
Feature Engineering and Extraction 147
Feature Engineering 147
Feature Extraction 147
Feature Extraction Algorithms 148
Feature Hashing 148
Feature Scaling and ML Algorithms 150
Selecting the Type of Scaling 150

Algorithms That Require Feature Scaling 150
Algorithms That Do Not Require Feature Scaling 150
Data Sampling Techniques 151
Undersampling 151
Oversampling 151
Resampling 152
Data Augmentation 152
Summary 154

**Chapter 7: Dimensionality Reduction 155**

Covariance and Correlation Matrices 155
Covariance Matrix 155
Covariance Matrix: An Example 156
The Correlation Matrix 157
Eigenvalues and Eigenvectors 157
Calculating Eigenvectors: A Simple Example 157
Gauss Jordan Elimination (Optional) 158
PCA (Principal Component Analysis) 159
The New Matrix of Eigenvectors 161
Dimensionality Reduction 162
Dimensionality Reduction Techniques 163
The Curse of Dimensionality 163
What Are Manifolds (Optional)? 163
SVD (Singular Value Decomposition) 164
LLE (Locally Linear Embedding) 165
UMAP 165
t-SNE ("tee-snee") 165
PHATE 166
Linear Versus Nonlinear Reduction Techniques 166
Types of Distance Metrics 167
Well-Known Distance Metrics 169
Pearson Correlation Coefficient 169
Jaccard Index (or Similarity) 169
Local Sensitivity Hashing (Optional) 170
What Is Sklearn? 170
Sklearn, Pandas, and the IRIS Dataset 172
Sklearn and Outlier Detection 174
What Is Bayesian Inference? 175
Bayes' Theorem 176
Some Bayesian Terminology 176
What Is MAP? 177
Why Use Bayes' Theorem? 177
What Are Vector Spaces? 177
Summary 178

**Appendix: Working With awk** **179**
The awk Command 180
Built-In Variables That Control awk 180
How Does the awk Command Work? 180
Aligning Text With the printf Statement 181
Conditional Logic and Control Statements 182
The while Statement 182
A for loop in awk 183
A for loop with a break Statement 184
The next and continue Statements 184
Deleting Alternate Lines in Datasets 184
Merging Lines in Datasets 185
Printing File Contents as a Single Line 186
Joining Groups of Lines in a Text File 186
Joining Alternate Lines in a Text File 187
Matching With Meta Characters and Character Sets 188
Printing Lines Using Conditional Logic 189
Splitting Filenames With awk 190
Working With Postfix Arithmetic Operators 190
Numeric Functions in awk 191
One-Line awk Commands 194
Useful Short awk Scripts 195
Printing the Words in a Text String in awk 196
Counting Occurrences of a String in Specific Rows 196
Printing a String in a Fixed Number of Columns 197
Printing a Dataset in a Fixed Number of Columns 198
Aligning Columns in Datasets 199
Aligning Columns and Multiple Rows in Datasets 200
Removing a Column From a Text File 201
Subsets of Column-Aligned Rows in Datasets 201
Counting Word Frequency in Datasets 203
Displaying Only "Pure" Words in a Dataset 204
Working With Multiline Records in awk 206
A Simple Use Case 207
Another Use Case 208
Summary 210

*Index* *211*

# PREFACE

## WHAT IS THE PRIMARY VALUE PROPOSITION FOR THIS BOOK?

This book is a resource-packed guide for rapid assimilation of feature engineering concepts, structured for those seeking both theoretical knowledge and practical skills in Python 3. It is designed for data scientists, machine learning practitioners, and anyone with a foundational understanding of Python 3.x.

With its hands-on approach and emphasis on Scikit-Learn, readers will gain proficiency in transforming data into a powerful ally for their machine learning models. Chapter 1 starts with an introduction to the types of data that you encounter in datasets, as well as issues that can arise in data, such as missing data and invalid data. Chapter 2 also contains topics pertaining to datasets, such as ways to detect outliers and anomalies in a dataset, and how to use Scikit-Learn for outlier detection.

Chapter 3 focuses on data cleaning, such as converting strings to date values, truncating rows in CSV files, converting phone numbers, and also working with country codes and cities.

Chapter 4 covers data wrangling, which different people interpret in different ways. You'll learn how to handle CSV files that contain multi-row records using the `awk` command-line utility that is described in the appendix. You'll also learn the details of an "events" project that tracks the dates, cities, and countries in which bands will perform.

Chapter 5 introduces feature selection and the three main categories of feature selection. You'll also learn how to work with imbalanced datasets and details regarding the SMOTE algorithm.

Chapter 6 discusses feature engineering, which consists of feature selection (in Chapter 5) as well as feature extraction. You'll also see which machine learning algorithms require feature scaling and the algorithms that do not require feature scaling in a dataset.

Chapter 7 discusses dimensionality reduction algorithms, which are typically performed when a dataset contains a large number of features. You'll learn about the covariance matrix and how to find its eigenvalues and eigenvectors in order to perform PCA

(Principal Component Analysis). You will also learn about linear versus non-linear reduction techniques, and some well-known distance metrics.

## WHAT WILL I LEARN FROM THIS BOOK?

The introductory section of the preface contains a brief outline of the topics in each of the chapters. As the title suggests, you will acquire a basic level of knowledge about a variety of feature selection and feature extraction algorithms.

As you will see, some of the chapters contain various partial code blocks that are designed to show you the type of code that you need for accessing algorithms that are usually in Scikit-Learn. In fact, you can often find complete code samples in the online documentation for Scikit-Learn.

## WHAT DO I NEED TO KNOW FOR THIS BOOK?

A current knowledge of Python 3.x is useful because all the code samples are in `Python`. Knowledge of data structures will enable you to progress through the related chapters more quickly. The less technical knowledge that you have, the more diligence will be required in order to understand the various topics that are covered.

If you want to be sure that you can grasp the material in this book, glance through some of the code samples to get an idea of how much is familiar to you and how much is new for you.

## DOES THIS BOOK CONTAIN PRODUCTION-LEVEL CODE SAMPLES?

The primary purpose of the code samples in this book is to show you solutions to tasks that involve data structures. Hence, clarity has higher priority than writing more compact or highly optimized code, For example, inspect the loops in the Python code sample to see if they can be made more efficient. Suggestion: treat such code samples as opportunities for you to optimize the code samples in this book.

If you decide to use any of the code in this book in a production website, you ought to subject that code to the same rigorous analysis as the other parts of your code base.

## COMPANION FILES

All the code samples and figures in this book may be obtained by writing to the publisher at *info@merclearning.com*.

**O. Campesato**
**December 2023**

CHAPTER 1

# WORKING WITH DATASETS

This chapter introduces you to various data types that you will encounter in datasets, how to work with currency and dates, types of dependencies that can occur among features in datasets, and how to scale data values.

The first part of this chapter contains a description of datasets and also explains data wrangling, along with details regarding various types of data that you might encounter in datasets.

The second part of this chapter shows you how to process inconsistent categorical data and how to map categorical data to numeric values. You will also learn about several types of dependencies among features, such as homoskedasticity and heroskedasticity, collinearity, variance inflation factor (VIF), multi-collinearity, and correlation.

The third part of this chapter shows you how to process data that involves currency as well as date-related data. Moreover, you will learn about techniques for scaling numeric data, such as normalization and standardization, as well as the StandardScaler class in SkLearn.

As you will soon see, this chapter provides a high-level view of concepts that will help you work with datasets that require preprocessing before using them to train machine learning models. While the code samples reference APIs from `Python` libraries (such as `Numpy` and `Pandas`), the APIs are intuitive: `mean()` for calculating the mean of a set of numbers and `std()` for calculating the standard deviation of a set of numbers.

Several points to keep in mind before you read this chapter. First, the code sample that involves `Scikit-learn` is marked "optional" because it uses the `EllipticEnvelope` class in `scikit-learn.covariance`, whose functionality is not intuitive (yet good to be aware of for future study).

Second, you need basic knowledge of `Pandas` to follow the details of the `Pandas` code samples, and if need be, you can quickly gain an understanding of the code by reading online tutorials.

Third, there several `awk`-based code samples in this chapter, which obviously requires knowledge of `awk`. You can either skip these code samples (they might not be useful for you), or you can learn about the fundamentals of `awk` through online code samples.

## EXPLORATORY DATA ANALYSIS (EDA)

According to Wikipedia, EDA involves analyzing datasets to summarize their main characteristics, often with visual methods. EDA also involves searching through data to detect patterns (if there are any) and anomalies, and in some cases, test hypotheses regarding the distribution of the data.

The purpose of EDA is to perform an analysis of the data in datasets, typically via data visualization, in order to produce a summary of their most important characteristics. The four types of EDA are univariate non-graphical, multivariate non-graphical, univariate graphical, and multivariate graphical. The two primary methods for data analysis are qualitative data analysis techniques and quantitative data analysis techniques.

As an example of EDA, consider the plethora of cell phones that customers can purchase for various needs. Visualize the data in an associated dataset to determine the top ten (or top three) most popular cell phones, which can potentially be performed by state (or province) and country.

Common visualization techniques used in EDA include histograms, box plots, and multivariate charts.

The next step often involves data cleaning in order to find and correct errors in the dataset, such as missing data, duplicate data, or invalid data. This task also involves data consistency, which pertains to updating different representations of the same value in a consistent manner.

As a trivial example, suppose that a Web page contains a form with an input field whose valid input is either Y or N, but users are able to enter Yes, Ys, or ys as text input. Obviously these values correspond to the value Y, and they must all be converted to the value Y (which is easy to do in Pandas).

Finally, data wrangling can be performed after the data cleaning task is completed. Although interpretations of data wrangling do vary, in this book the term refers to transforming datasets into different formats as well as combining datasets. Hence, data wrangling does not examine the individual data values to determine whether or not they are valid: this step is performed during data cleaning.

Keep in mind that sometimes it's worthwhile to perform another data cleaning step after data wrangling. For example, suppose that two CSV files contain employee-related data, and you merge these CSV files into a third CSV file. The newly created CSV file might contain duplicate values: it's certainly possible to have two people with the same name (such as John Smith). See also:

*https://github.com/Oluwabukunmige/Pipeline-for-Exploratory-Data-Analysis-and-Data-Cleaning.*

### EDA Code Sample: Titanic

EDA involves a plethora of tasks, such as detecting patterns in the data, detecting outliers/anomalies (if any), and also testing various hypotheses. As a starting point, Listing 1.1 displays the contents of `desc_titanic.py` that shows you how to describe the contents of a dataset.

***LISTING 1.1: desc_titanic.py***

```
import pandas as pd

filename="titanic.csv"
df = pd.read_csv(filename)
print("Description of",filename)
print(df.describe())
```

Listing 1.1 starts with an `import` statement and then initializes the variable `filename` with the name of a `CSV` file. The next code snippet initializes the `Pandas` data frame `df` with the contents of the `CSV` file, whose description is then displayed. Launch the code in Listing 1.1 and you will see the following output:

```
Description of titanic.csv
         survived      pclass         age       sibsp       parch        fare
count  891.000000  891.000000  714.000000  891.000000  891.000000  891.000000
mean     0.383838    2.308642   29.699118    0.523008    0.381594   32.204208
std      0.486592    0.836071   14.526497    1.102743    0.806057   49.693429
min      0.000000    1.000000    0.420000    0.000000    0.000000    0.000000
25%      0.000000    2.000000   20.125000    0.000000    0.000000    7.910400
50%      0.000000    3.000000   28.000000    0.000000    0.000000   14.454200
75%      1.000000    3.000000   38.000000    1.000000    0.000000   31.000000
max      1.000000    3.000000   80.000000    8.000000    6.000000  512.329200
```

Listing 1.2 displays the contents of `info_titanic.py` that shows you how display information about a dataset.

***LISTING 1.2: info_titanic.py***

```
import pandas as pd

filename="titanic.csv"
df = pd.read_csv(filename)
print("Information about",filename)
print(df.info())
```

Listing 1.2 starts with an `import` statement and then initializes the variable `filename` with the name of a `CSV` file. The next code snippet initializes the `Pandas` data frame `df` with the contents of the `CSV` file, whose information is then displayed. Launch the code in Listing 1.1 and you will see the following output:

```
Information about titanic.csv
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    object
```

```
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    object
 8   class        891 non-null    object
 9   who          891 non-null    object
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    object
 12  embark_town  889 non-null    object
 13  alive        891 non-null    object
 14  alone        891 non-null    bool
dtypes: bool(2), float64(2), int64(4), object(7)
memory usage: 92.4+ KB
```

As you can see, the preceding output displays the number of non-null values for the age feature (714) and the deck feature (204).

You can also use the `Pandas isnull()` function that indicates whether values are missing. Listing 1.3 displays the contents of `null_titanic.py` that shows you how display information about null values in a dataset.

***LISTING 1.3: null_titanic.py***

```
import pandas as pd

filename="titanic.csv"
train_df = pd.read_csv(filename)

shape = train_df.shape
print("=> Number of rows:")
print(shape[0])
print()

print("=> Number of columns (features):")
print(shape[1])
print()

null_sum = train_df.isnull().sum()
print("=> Number of Null values for each feature:")
print(null_sum)
print()

deck = train_df['deck']
print("=> Number of Null deck values:")
print(deck.isnull().sum())
print()

print("=> Number of non-Null deck values:")
print(deck.notna().sum())
print()

age = train_df['age']
print("=> Number of Null age values:")
print(age.isnull().sum())
print()
```

```
print("=> Number of non-Null age values:")
print(age.notna().sum())
print()
```

Listing 1.3 starts with an `import` statement and then initializes the variable `filename` with the name of a `CSV` file. The next code snippet initializes the `Pandas` data frame `df` with the contents of the `CSV` file.

The next two code snippets display the number of rows and the number of columns (features) in the `CSV` file `titanic.csv`. The third code snippet displays the number of null values for each feature.

The next portion of Listing 1.3 displays the number of null and non-null `deck` values via the invocation of `isnull().sum()`, followed by another code block that displays the number of null and non-null `age` values. Now launch the code in Listing 1.3 and you will see the following output:

```
=> Number of rows:
891

=> Number of columns (features):
15

=> Number of Null values for each feature:
survived         0
pclass           0
sex              0
age            177
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64

=> Number of Null deck values:
688

=> Number of non-Null deck values:
203

=> Number of Null age values:
177

=> Number of non-Null age values:
714
```

Listing 1.4 displays the contents of `mean_titanic.py` that shows you how display calculated mean values in a dataset.

***LISTING 1.4: mean_titanic.py***

```
import pandas as pd

filename="titanic.csv"
df = pd.read_csv(filename)

mean_values = df.groupby(['sex']).mean()
print("Mean values by Gender:")
print(mean_values)
print()

mean_values2 = train_df.groupby(['sex','fare']).mean()
print("=> Mean values by Gender and Fare:")
print(mean_values2)
```

Listing 1.4 starts with an `import` statement and then initializes the variable `filename` with the name of a `CSV` file. The next code snippet initializes the `Pandas` data frame `df` with the contents of the `CSV` file.

The next code snippet groups the rows in the sex feature by gender, and then calculates the mean value of each gender, after which those values are displayed. The final portion of Listing 1.4 calculates the mean values of the sex feature and the fare feature and displays those values. Launch the code in Listing 1.4 and you will see the following output:

```
=> Mean values by Gender:
   survived    pclass      age     sibsp    parch      are  adult_male   alone
sex
female 0.742038 2.159236 27.915709 0.694268 0.649682 44.479818 0.000000 0.401274
male   0.188908 2.389948 30.726645 0.429809 0.235702 25.523893 0.930676 0.712305

=> Mean values by Gender and Fare:
                survived  pclass   age  sibsp  parch  adult_male  alone
sex    fare
female 6.7500        0.0     3.0  18.0    0.0    0.0         0.0    1.0
       7.2250        1.0     3.0  15.0    0.0    0.0         0.0    1.0
       7.2292        1.0     3.0  13.0    0.0    0.0         0.0    1.0
       7.2500        1.0     3.0  22.0    0.0    0.0         0.0    1.0
       7.4958        1.0     3.0  18.0    0.0    0.0         0.0    1.0
...                  ...     ...   ...    ...    ...         ...    ...
male   221.7792      0.0     1.0   NaN    0.0    0.0         1.0    1.0
       227.5250      0.0     1.0   NaN    0.0    0.0         1.0    1.0
       247.5208      0.0     1.0  24.0    0.0    1.0         1.0    0.0
       263.0000      0.0     1.0  41.5    2.0    3.0         1.0    0.0
       512.3292      1.0     1.0  35.5    0.0    0.5         1.0    0.5

[349 rows x 7 columns]
```

## EDA and Histograms

One technique for analyzing the distribution of the data in a dataset is to render the data in a histogram. Listing 1.5 displays the contents of `hist_titanic.py` that shows you how to render the data in the `Titanic` dataset in a histogram.

***LISTING 1.5: hist_titanic.py***

```
import pandas as pd
import matplotlib.pyplot as plt

filename="titanic2.csv"
df = pd.read_csv(filename)
df2 = train_df[['survived','age']]
df2['age'] = train_df2['age'].astype(int)

df2.hist("age",by="survived",edgecolor='blue',linewidth=1,grid=True,colo
r="red",figsize=(10, 8),bins=30)
plt.suptitle("")
plt.xlabel("")
plt.show()
```

Listing 1.5 starts with `import` statements and then initializes the variable `filename` with the name of a `CSV` file. The next code snippet initializes the `Pandas` data frame `df` with the contents of the `CSV` file. In addition, the data frame `df2` is initialized with the columns that contain the `survived` feature and the `age` feature in the data frame `df`. Notice that the values in the `age` feature in the data frame `df2` are treated as integer values (i.e., not string values).

The final portion of Listing 1.4 generates a histogram based on the `age` feature values and the `survived` feature values, after which the histogram is displayed. Launch the code in Listing 1.5 and you will see the histogram that is shown in Figure 1.1.

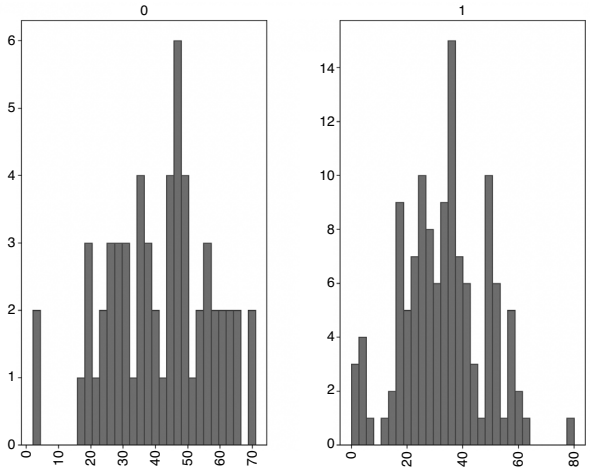



***FIGURE 1.1*** A histogram with data for the Titanic.

This concludes the brief introduction to `EDA`, with simple code samples that showed you the type of information that you can extract from a dataset in order to gain a better understanding of the distribution of the data.

## DEALING WITH DATA: WHAT CAN GO WRONG?

In a perfect world, all datasets are in pristine condition, with no extreme values, no missing values, and no erroneous values. Every feature value is captured correctly, with no chance for

any confusion. Moreover, no conversion is required between date formats, currency values, or languages because of the One Universal Standard that defines the correct formats and acceptable values for every possible set of data values.

Unfortunately, many datasets are not in pristine condition, which is the reason for the techniques that are discussed in this chapter. Even after you manage to create a wonderfully clean and robust dataset, other issues can arise, such as data drift that is described in a subsequent section.

In fact, the task of cleaning data is not necessarily complete even after a machine learning model is deployed to a production environment. For instance, an online system that gathers significant amounts of data on a daily basis can contain skewed values that in turn adversely affect the performance of the model. Such adverse effects can be revealed through the changes in the metrics that are associated with the production model.

## Datasets

In simple terms, a dataset is a source of data (such as a text file) that contains rows and columns of data. Each row is typically called a "data point," and each column is called a "feature." A dataset can be a `CSV` (comma separated values), `TSV` (tab separated values), `Excel` spreadsheet, a table in an `RDMBS`, a document in a `NoSQL` database, the output from a Web service, and so forth.

A dataset can vary from very small (a couple of features and 100 rows) to very large (more than 1,000 features and more than one million rows). If you are unfamiliar with the problem domain, then you might struggle to determine the most important features in a large dataset. In this situation, you might need a "domain expert" who understands the importance of the features, their interdependencies (if any), and whether or not the data values for the features are valid. In addition, there are algorithms (called dimensionality reduction algorithms) that can help you determine the most important features, such as `PCA` (principal component analysis).

Before delving into topics such as data preprocessing, data types, and so forth, let's take a brief detour to introduce the concept of feature importance, which is the topic of the next section.

As you will see, someone needs to analyze the dataset to determine which features are the most important and which features can be safely ignored in order to train a model with the given dataset.

A dataset can contain various data formats, such as:

- audio data
- image data
- numeric data
- text-based data
- video data
- combinations of the previous formats

In this book, we'll only consider datasets that contain columns with numeric or text-based data types, which can be further classified as follows:

- nominal (string-based or numeric)
- ordinal (ordered values)
- categorical (enumeration)
- interval (positive/negative values)
- ratio (nonnegative values)

The next section contains brief descriptions of the data types that are in the preceding bullet list. See also:

*https://careerfoundry.com/en/blog/data-analytics/what-is-ordinal-data/*

## EXPLANATION OF DATA TYPES

This section contains subsections that provide brief descriptions about the following data types:

- binary data
- nominal data
- ordinal data
- categorical data
- interval data
- ratio data

Later you will learn about the difference between continuous data and discrete data, as well as the difference between qualitative data and quantitative data. In addition, the `Pandas` documentation describes data types and how to use them in `Python`.

### Binary Data

*Binary data* involves numeric data that can only take two distinct values. As such, binary data is the simplest type of data. A common example involves flipping a fair coin: the only outcomes are in the set `{H,T}`. Other terms for binary data include dichotomous, logical data, Boolean data, and indicator data. Binary data is also a type of categorical data that is discussed later.

### Nominal Data

The word "nominal" has multiple meanings, and in our case it refers to something that constitutes a name (think "name" from the prefix "nom"). Thus, *nominal data* is often (see next paragraph) name-based data that involves different name labels. Examples of nominal data include hair color, music preferences, movie types, and so forth. As you can see, there is no hierarchy or ordering involved, so all values are considered to have the same level or significance. However, the number of items in nominal values might be different, such as the number of people that belong to different political parties.

Nominal data can involve numeric values to represent different values of a feature. For example, the numbers in the set `{0,1}` can represent {Male, Female}, and the numbers in the set `{0,1,2,3,4,5,6}` can represent the days in a week. However, there is no hierarchical

interpretation associated with these numeric values: the day of the week represented by "0" is not considered more important or more valuable than any of the other numeric labels for the other days of the week. Instead, think of each element in terms of its predecessor and successor: note that the first element has no predecessor and the last element has no successor. If you are familiar with programming languages, the counterpart to integer-based nominal values would be an enumeration, an example of which is here:

```
enum DAY {SUN,MON,TUE,WED,THU,FRI,SAT};
```

### Ordinal Data

*Ordinal data* implies an ordering of the elements in a finite set (think "ordering" from the prefix "ord"). For example, there are different values for titles regarding software developers. As a simplified example, the set consisting of `{D1, D2, SD1, SD2}` can be used to specify junior developers (`D1`) up through senior developers (`SD2`), which have criteria associated with each level. Hence, integer-based and string-based elements of ordinal data are ordered.

Keep in mind that integer-based ordinal data does not have an implied relative value. For example, consider the following set of ordinal data `S = {1,2,3,4,5,6}` that represent grade levels in an organization. A level 2 employee is not "twice" as experienced as a level 1 employee, nor would a level 6 employee be three times as experienced as a level 2 (unless you define these values in such a manner).

Please read the `Scikit-learn` documentation regarding the class `OrdinalEncoder` (located in `scikit-learn.preprocessing.OrdinalEncorder`) for handling ordinal data.

### Categorical Data

*Categorical data* refers to nominal data as well as ordinal data: refer to the preceding sections to learn about the nuances involved in nominal data and ordinal data. Categorical data can only assume a finite set of distinct values, such as enumerations that are explained in a previous section. In addition, `Pandas` can explicitly specify a column as type categorical when you read the contents of a `CSV` file via the `read_csv()` method.

### Interval Data

*Interval data* pertains to data that is ordered and lies in an interval or range, such as the numbers in the interval `[-1,1]`. Examples of interval data include temperature, income-versus-debt, and so forth. Notice that nominal values can be negative as well as positive.

### Ratio Data

*Ratio data* involves measured intervals, such as barometric pressure, height, altitude, and so forth. Notice the difference between interval data and ratio data: unlike interval data, ratio data cannot be negative. Obviously, it makes no sense to refer to negative barometric pressure or height (and in some cases, altitude as well).

### Continuous Data Versus Discrete Data

*Continuous data* can take on any value in an interval, such as `[-1,1], [0,1], [5,10]`. Hence, continuous data involves floating point numbers, which includes include interval data. Keep in mind that an interval contains an uncountably infinite number of values.

One other point to note pertains to possible values and their floating point representation. For instance, a random number in the interval [0,1] can be one of an uncountably infinite number of values, whereas its representation as a floating point number is limited to a large yet finite number of values. Let's suppose that the integer 10*1000 equals the number of numbers in the interval `[0,1]` that can be represented as a floating point number. Then the smallest positive number in the interval `[0,1]` that can be represented as a floating point number is 1/N. However, there is an uncountably infinite number of values in the interval `[0,1/N)`, which we would approximate as the value 0 (or possibly 1/N).

*Discrete data* can take on a finite set of values, and the earlier comments regarding successors and predecessors apply to discrete data. As a simple example, the outcome of tossing a coin or throwing a die (or multiple dice) involve discrete data, which are also examples of nominal data. In addition, the associated probabilities for the outcomes form a discrete probability distribution (search online if you need a definition).

## Random Variables

A *random variable* X can have a set of outcomes, and each outcome has an associated probability. A discrete random variable has a finite set of values, otherwise it is a continuous random variable. For example, let X be a random variable for the outcomes of tossing a fair two-sided coin. Then X can be either heads or tails, and the probability of outcome equals 0.5. In the case of a fair six-sided die, the outcomes are in the set {1,2,3,4,5,6}, each of which can occur with probability 1/6.

## Qualitative and Quantitative Data

*Quantitative data* can be either discrete or continuous. For example, age that is measured in years is discrete, whereas the height of a person is continuous. One point to keep in mind: the word "continuous" in statistics does not always have the same meaning when it's used in a mathematical context. For instance, the price of a house is treated as a continuous feature, but it's not continuous in the mathematical sense because the smallest unit of measure is a penny, and there are many (in fact, an uncountably infinite number of) values between two consecutive penny values.

*Qualitative data* involves text-based values, such as different types of color or movie genres. As you saw in an earlier section, it's possible to use integer-based values for nominal values, such as days of the week, months of the year, and so forth. In fact, if a dataset contains a string-based feature that is selected as input for a machine learning algorithm, those values are typically converted into integer-based values, which can be performed via the `map()` function in `Pandas`.

## Types of Statistical Data

The preceding sections described several data types, whereas this section classifies data types from a statistical standpoint. There are four primary types of statistical data, as follows:

- nominal
- ordinal
- interval
- ratio

One way to remember these four types of statistical data is via the acronym NOIR (coincidentally the French word for "black"). Please refer to the earlier sections for details regarding any of these data types.

## DATA PREPROCESSING

Data preprocessing is the initial stage for validating the contents of a dataset, which involves a multistep process (not all of which is always necessary), as follows:

- removing dependent columns
- handling missing data values
- cleaning text-based data
- removing HTML tags
- removing emoticons
- filtering data
- grouping data
- handling currency and date formats

A *dependent column* is a column that can be derived from one or more columns in a dataset. As a very simple example, if a dataset contains the attributes `length` and `width` that represent the dimensions of rectangular rooms in a house, then the `area` of a room equals the product of the `width` and the `length` of the room. Hence, the area is a feature that depends on the `length` and the `width`, so it's redundant. Alternatively, you might decide to retain the `area` and the `width` and use their values whenever you need to compute the `length` attribute.

Other important topics pertaining to features in a dataset include collinearity, multicollinearity, VIF (variable inflation factor), homoskedasticity (constant variance of error terms), and heteroskedasticity (nonconstant variance of error terms), all of which are discussed later in this book.

*Cleaning data* involves removing unwanted data, handling missing data, and rectifying erroneous data. In the case of text-based data, you might need to remove `HTML` tags, punctuation, and so forth. In the case of data that is supposed to be numeric, it's possible that alphabetic characters are mixed together with numeric data. However, a dataset with numeric features might have incorrect values or missing values (discussed later). In addition, calculating the minimum, maximum, mean, median, and standard deviation of the values of a feature obviously pertain only to numeric values.

After the preprocessing step is completed, data wrangling is performed, which refers to transforming data into a new format, as well as combining features (as new columns and/or appending rows) from multiple datasets. For example, you might need to convert between different units of measurement (such as date formats, currency values, and so forth) so that the data values can be represented in a consistent manner in a dataset.

In case you didn't already know, currency and date values are part of something that is called `i18n` (internationalization), whereas `L10n` (localization) targets a specific nationality, language, or region. Hard-coded values (such as text strings) can be stored as resource strings in a file that's often called a resource bundle, where each string is referenced via a code. Each language has its own resource bundle.

Strictly speaking, a dataset that contains dependent columns involves removing those columns instead of modifying their contents. One example involves the length, width, and area of rectangular rooms in a house, which was discussed earlier in this section.

## WORKING WITH DATA TYPES

If you have experience with programming languages (e.g., `C`, `C++`, `Java`, etc.), then you know that explicit data types exist in programming languages. Some programming languages, such as `JavaScript` and `awk`, do not require initializing variables with an explicit type: the type of a variable is inferred dynamically via an implicit type system (i.e., one that is not directly exposed to a developer).

In machine learning, datasets can contain features that have different types of data, such as a combination of one or more of the following types of features:

- numeric data (integer/floating point and discrete/continuous)
- character/categorical data (different languages)
- date-related data (different formats)
- currency data (different formats)
- binary data (yes/no, 0/1, etc.)
- nominal data (multiple unrelated values)
- ordinal data (multiple and related values)

Consider a dataset that contains real estate data, which can have thirty or more columns, often with the following features:

- the number of bedrooms in a house: numeric value and a discrete value
- the number of square feet: a numeric value and (probably) a continuous value
- the name of the city: character data
- the construction date: a date value
- the selling price: a currency value and probably a continuous value
- the "for sale" status: binary data (either "yes" or "no")

An example of nominal data is the seasons in a year: although many countries have four distinct seasons, some countries have two distinct seasons. However, keep in mind that seasons can be associated with different temperature ranges (summer versus winter). An example of ordinal data is an employee pay grade: 1=entry level, 2=one year of experience, and so forth. Another example of nominal data is a set of colors, such as `{Red, Green, Blue}`.

A familiar example of binary data is the pair `{Male, Female}`, and some datasets contain a feature with these two values. If such a feature is required for training a model, first convert `{Male, Female}` to a numeric counterpart (such as `{0,1}`), and a `Pandas`-based example is here:

```
df['gender'] = df['gender'].map({'Male': 0, 'Female': 1})
```

Similarly, if you need to include a feature whose values are the previous set of colors, you can replace `{Red, Green, Blue}` with the values `{0,1,2}`. Categorical data is discussed in more detail later in this chapter.

## DATA DRIFT

In machine learning terms, *drift* refers to any type of change in distribution over a period of time. *Model drift* refers to a change (drift) in the accuracy of a model's prediction, whereas *data drift* refers to a change in the type of data that is collected. Note that data drift is also called input drift, feature drift, or covariate drift.

There are several factors that influence the value of data, such as accuracy, relevance, and age. For example, physical stores that sell mobile phones are much more likely to sell recent phone models than older models. In some cases, data drift occurs over a period of time, and in other cases it's because some data is no longer relevant due to feature-related changes in an application. Always keep in mind that there might be multiple factors that can influence data drift in a specific dataset.

Two techniques for handling data drift are *domain classifier* and the *black-box shift detector*, both of which are discussed here:

*https://blog.dataiku.com/towards-reliable-mlops-with-drift-detectors*

There are several other types of changes that can occur in a dataset, some of which are listed below:

- concept shift
- covariate shift
- domain shift
- prior probability shift
- spurious correlation shift
- subpopulation shift
- time shift

Perform an online search to find more information about the topics in the preceding list of bullet items. Finally, the following list contains links to open-source `Python`-based tools/libraries that provide drift detection:

- alibi-detect (*https://github.com/SeldonIO/alibi-detect*)
- evidently (*https://github.com/evidentlyai/evidently*)
- Torchdrift (*http://torchdrift.org/*)

## WHAT IS DATA LEAKAGE?

Data leakage occurs when data that is external to the training dataset is included during the training of a model (i.e., inward leaks instead of outward leaks of data). This external data can influence the capability of the model in unexpected ways, which in turn can adversely affect previous metrics that are associated with the model.

Data leakage tends to be an issue with complex datasets, and some of those types are listed here:

- time series datasets
- graph problems
- data stored in multiple files

Two good techniques to minimize data leakage when developing predictive models are as follows:

- Perform data preparation within your cross-validation folds.
- Hold back a validation dataset for a final sanity check of your developed models.

Generally, it is good practice to use both of these techniques. See also:

*https://insidebigdata.com/2014/11/26/ask-data-scientist-data-leakage/*

### Data Leakage and Differential Privacy

Machine learning models can involve large amounts of data, some of which can be sensitive and confidential. For example, salaries, social security numbers, and medical conditions of people are three obvious examples of confidential information that must not be revealed by a trained model. See also:

*https://pair.withgoogle.com/explorables/data-leak/?linkId=8028464*

*https://www.w3schools.in/cyber-security/data-leak-prevention/*

*https://machinelearningmastery.com/data-leakage-machine-learning/*

*https://towardsdatascience.com/uncertainty-in-deep-learning-brief-introduction-1f9a5de3ae04*

Differential privacy prevents training data from leaking confidential information by restricting the information that is available in each data point. Moreover, a model's privacy can be improved through techniques such as aggregation.

## MODEL SELECTION AND PREPARING DATASETS

If you have the good fortune to inherit a dataset that is in pristine condition, then data cleaning tasks (discussed later) are vastly simplified: in fact, it might not be necessary to perform *any* data cleaning for the dataset. On the other hand, if you need to create a dataset that combines data from multiple datasets that contain different formats for dates and currency, then you need to perform a conversion to a common format. Now let's list some of the criteria for model selection, as discussed in the next section.

### Model Selection

As you can probably infer, *model selection* refers to the steps involved in selecting a machine learning algorithm. Although there is no "silver bullet" with respect to model selection, here are some simple guidelines:

- Choose a model based on its expected performance.
- Prefer simpler models rather than complex models.
- Use a pre-trained model.

Expected performance refers to highest accuracy or lowest prediction error, and simpler models generally make fewer assumptions. If you are fortunate, you might find a pre-trained model that enables you to perform some additional training with your custom dataset. Despite these recommendations, always be prepared to try different algorithms and to change the values of some of the hyperparameters of those algorithms.

If you need to train a model that includes features that have categorical data, then you need to convert that categorical data to numeric data. For instance, the Titanic dataset contains a feature called "gender," which is either male or female. As you will see later in this chapter, `Pandas` makes it extremely simple to "map" male to 0 and female to 1.

## Discrete Data Versus Continuous Data

As a simple rule of thumb: discrete data is a set of values that can be counted whereas continuous data must be measured. Discrete data can reasonably fit in a drop-down list of values, but there is no exact value for making such a determination. One person might think that a list of 500 values is discrete, whereas another person might think it's continuous.

For example, the list of provinces of Canada and the list of states of the United States are discrete data values, but is the same true for the number of countries in the world (roughly 200) or for the number of languages in the world (more than 7,000)?

On the other hand, values for temperature, humidity, and barometric pressure are considered continuous. Currency is also treated as continuous, even though there is a measurable difference between two consecutive values. The smallest unit of currency for U.S. currency is one penny, which is 1/100th of a dollar (accounting-based measurements use the "mil," which is 1/1,000th of a dollar).

Continuous data types can have subtle differences. For example, someone who is 200 centimeters tall is twice as tall as someone who is 100 centimeters tall; similarly for 100 kilograms versus 50 kilograms. However, temperature is different: 80 degrees Fahrenheit is not twice as hot as 40 degrees Fahrenheit.

Furthermore, keep in mind that the meaning of the word "continuous" in mathematics is not necessarily the same as continuous in machine learning. In the former, a continuous variable (let's say in the 2D Euclidean plane) can have an uncountably infinite number of values. On the other hand, a feature in a dataset that can have more values than can be "reasonably" displayed in a drop-down list is treated *as though* it's a continuous variable.

For instance, values for stock prices are discrete: they must differ by at least a penny (or some other minimal unit of currency), which is to say, it's meaningless to say that the stock price changes by one-millionth of a penny. However, since there are "so many" possible stock values, it's treated as a continuous variable. The same comments apply to car mileage, ambient temperature, barometric pressure, and so forth.

## "Binning" Data Values

The term *binning* refers to subdividing a set of values into multiple intervals, and then treating all the numbers in the same interval as though they had the same value. In addition, there are at least three techniques for binning data, as shown here:

- bins of equal widths
- bins of equal frequency
- bins based on k-means

See also:

*https://towardsdatascience.com/from-numerical-to-categorical-3252cf805ea2*

As a simple example of bins of equal widths, suppose that a feature in a dataset contains the age of people in a dataset. The range of values is approximately between 0 and 120, and we could "bin" them into 12 equal intervals, where each consists of 10 values: 0 through 9, 10 through 19, 20 through 29, and so forth.

As another example, using quartiles is even more coarse-grained than the earlier age-related binning example. The issue with binning pertains to the unintended consequences of classifying people in different bins, even though they are in close proximity to each other. For instance, some people struggle financially because they earn a meager wage, and they are also disqualified from financial assistance because their salary is higher than the cut-off point for receiving any assistance.

`Scikit-learn` provides the `KBinsKDiscretizer` class that uses a clustering algorithm for binning data:

*https://scikit-learn.org/stable/modules/generated/scikit-learn.preprocessing.KBinsDiscretizer.html*

In case you're interested, a highly technical paper (PDF) with information about clustering and binning is here:

*https://www.stat.cmu.edu/tr/tr870/tr870.pdf*

## Programmatic Binning Techniques

Earlier in this chapter you saw a `Pandas`-based example of generating a histogram using data from a `Titanic` dataset. The number of bins was chosen on an *ad hoc* basis, with no relation to the data itself. However, there are several techniques that enable you to programmatically determine the optimal number of bins, some of which are shown as follows:

- Doane's formula
- Freedman–Diaconis's choice
- Rice's rule
- Scott's normal reference rule
- Square-root choice
- Sturges's rule

Doane's formula for calculating the number of bins depends on the number of observations n and the kurtosis of the data, and it's reproduced here:

```
k = 1 + log(n) + log(1 + kurtosis(data) * sqrt(n / 6.0))
```

Freedman–Diaconis's choice specifies the number of bins for a sample x, and it's based on the IQR (interquartile range), the number of observations n, and a sample, and the formula is shown as follows:

```
k = 2 * IRQ(x)/[cube root of n]
```

Sturges's rule to determine the number of bins k for Gaussian-based data is based on the number of observations n, and it's expressed as follows:

```
k = 1 + 3.322 * log n
```

In addition, after specifying the number of bins k, set the minimum bin width mbw as follows:

```
mbw = (Max Observed Value – Min Observed Value) / k
```

Experiment with the preceding formulas to determine which one provides the best visual display for your data. For more information about calculating the optimal number of bins, perform an online search for blog posts and articles.

### Potential Issues When Binning Data Values

Partitioning the values of people's age as described in the preceding section can be problematic. In particular, suppose that person A, person B, and person C are 29, 30, and 39, respectively. Then person A and person B are probably more similar to each other than person B and person C, but because of the way in which the ages are partitioned, B is classified as closer to C than to A. In fact, binning can increase Type I errors (false positive) and Type II errors (false negative), as discussed in this blog post (along with some alternatives to binning):

*https://medium.com/@peterflom/why-binning-continuous-data-is-almost-always-a-mistake-ad0b3a1d141f*

## HANDLING CATEGORICAL DATA

A feature with categorical data can suffer from various issues, such as missing data, invalid data, or inconsistently formatted data. The following section discusses examples of inconsistent categorical data followed by a section that discusses how to map categorical data to numeric values.

### Processing Inconsistent Categorical Data

This section contains examples of processing inconsistent data values. For features that have very low cardinality, consider dropping those features, and similarly for numeric columns with zero or very low variance.

Next, check the contents of categorical columns for inconsistent spellings or errors. For example, suppose that a feature contains the values M and F (for male and female), along with a mixture of gender-related strings, some of which are in the following list:

```
male
Male
female
Female
m
f
M
F
```

The preceding categorical values for gender can be replaced with two categorical values (unless you have a valid reason to retain some of the other values). Moreover, if you are training a model whose analysis involves a single gender, then you need to determine which rows (if any) of a dataset must be excluded. Also check categorical data columns for redundant or missing whitespaces.

Check for data values that have multiple data types, such as a numerical column with numbers as numerals and some numbers as strings or objects. Also ensure consistent data formats: numbers as integer or floating numbers and ensure that dates have the same format (for example, do not mix `mm/dd/yyyy` date formats with another date format, such as `dd/mm/yyyy`).

## Mapping Categorical Data to Numeric Values

Character data is often called categorical data, examples of which include people's names, home or work addresses, email addresses, and so forth. Many types of categorical data involve short lists of values. For example, the days of the week and the months in a year involve seven and twelve distinct values, respectively. Notice that the days in a week have a relationship: each day has a previous day and a next day, and similarly for the months of a year.

On the other hand, the colors of an automobile are independent of each other: the color red is not "better" or "worse" than the color blue. However, cars of a certain color can have a statistically higher number of accidents, but we won't address this case here.

There are several well-known techniques for mapping categorical values to a set of numeric values. A simple example where you need to perform this conversion involves the gender feature in the Titanic dataset. This feature is one of the relevant features for training a machine learning model. The gender feature has `{M,F}` as its set of values. As you will see later in this chapter, `Pandas` makes it very easy to convert the set of values `{M,F}` to the set of values `{0,1}`.

Another mapping technique involves mapping a set of categorical values to a set of consecutive integer values. For example, the set `{Red, Green, Blue}` can be mapped to the set of integers `{0,1,2}`. The set `{Male, Female}` can be mapped to the set of integers `{0,1}`. The days of the week can be mapped to `{0,1,2,3,4,5,6}`. Note that the first day of the week depends on the country: in some cases it's Sunday and in other cases it's Monday.

Another technique is called *one-hot encoding*, which converts each value to a vector (check Wikipedia if you need a refresher regarding vectors). Thus, `{Male, Female}` can be represented by the vectors `[1,0]` and `[0,1]`, and the colors `{Red, Green, Blue}` can be represented by the vectors `[1,0,0]`, `[0,1,0]`, and `[0,0,1]`.

If you vertically "line up" the two vectors for gender, they form a 3x3 identity matrix, and doing the same for the colors `{R,G,B}` will form a 3x3 identity matrix, as shown here:

```
[1,0,0]
[0,1,0]
[0,0,1]
```

If you are familiar with matrices, you probably noticed that the preceding set of vectors looks like the 3x3 identity matrix. In fact, this technique generalizes in a straightforward manner. Specifically, if you have n distinct categorical values, you can map each of those values to one of the vectors in an nxn identity matrix.

As another example, the set of titles `{"Intern", "Junior", "Mid-Range", "Senior", "Project Leader", "Dev Manager"}` have a hierarchical relationship in terms of their salaries (which can also overlap, but we'll gloss over that detail for now).

Another set of categorical data involves the season of the year: `{"Spring", "Summer", "Autumn", "Winter"}` and while these values are generally independent of each other, there are cases in which the season is significant. For example, the values for the monthly rainfall, average temperature, crime rate, or foreclosure rate can depend on the season, month, week, or even the day of the year.

If a feature has a large number of categorical values, then a one-hot encoding will produce many additional columns for each data point. Since the majority of the values in the new columns equal 0, this can increase the sparsity of the dataset, which in turn can result in more overfitting and hence adversely affect the accuracy of machine learning algorithms that you adopt during the training process.

Another solution is to use a sequence-based solution in which N categories are mapped to the integers 1, 2, . . . , N. Another solution involves examining the row frequency of each categorical value. For example, suppose that N equals 20, and there are 3 categorical values that occur in 95% of the values for a given feature. You can try the following:

1. Assign the values 1, 2, and 3 to those three categorical values.
2. Assign numeric values that reflect the relative frequency of those categorical values.
3. Assign the category "OTHER" to the remaining categorical values.
4. Delete the rows whose categorical values belong to the remaining 5%.

## TYPES OF DEPENDENCIES AMONG FEATURES

The ideal case for a dataset involves clean data and independent features, which is rarely the case in real world datasets. As such, it's important to determine whether or not there are dependencies among features. This section contains information about the following terms that pertain to feature dependence:

- homoskedasticity
- collinearity
- multicollinearity
- VIF (variance inflation factor)
- Correlation

As a quick summary: collinearity adversely affects the significance of independent variables, which in turn reduces the effectiveness of a regression model.

Furthermore, statistical tests such as the Student's `t-test` and `ANOVA` (discussed later) assume homoskedasticity for the data. In other words, such algorithms assume that samples from a population (or even different populations) have the same variance.

### Homoskedasticity and Heteroskedasticity

*Homoskedasticity* refers to the variance of the residuals in regression models. Recall that one of the assumptions of linear regression is that the residuals have constant homoskedasticity.

*Heteroskedasticity* is the extent to which features are independent of each other. However, heteroskedasticity is more than just "the opposite" of homoskedasticity, and there are various statistical tests that determine heteroskedasticity, many of which are listed here:

*https://en.wikipedia.org/wiki/Heteroskedasticity*

In statistics, a vector of random variables is heteroscedastic (or heteroskedastic; [a] from Ancient Greek hetero "different" and skedasis "dispersion") if the variability of the random disturbance is different across elements of the vector. Here, variability could be quantified by the variance or any other measure of statistical dispersion. Thus heteroscedasticity is the absence of homoscedasticity. A typical example is the set of observations of income in different cities.

The existence of heteroscedasticity is a major concern in regression analysis and the analysis of variance, as it invalidates statistical tests of significance that assume that the modelling errors all have the same variance. While the ordinary least squares estimator is still unbiased in the presence of heteroscedasticity, it is inefficient and generalized least squares should be used instead.

The econometrician Robert Engle was awarded the 2003 Nobel Memorial Prize for Economics for his studies on regression analysis in the presence of heteroscedasticity, which led to his formulation of the autoregressive conditional heteroscedasticity (ARCH) modeling technique.

Heteroskedasticity is discussed in further detail here:

*https://www.investopedia.com/terms/h/heteroskedasticity.asp*

In statistics, heteroskedasticity (or heteroscedasticity) happens when the standard errors of a variable, monitored over a specific amount of time, are nonconstant.

With heteroskedasticity, the tell-tale sign upon visual inspection of the residual errors is that they will tend to fan out over time, as depicted in the image above.

Heteroskedasticity is a violation of the assumptions for linear regression modeling, and so it can impact the validity of econometric analysis or financial models like CAPM.

While heteroskedasticity does not cause bias in the coefficient estimates, it does make them less precise; lower precision increases the likelihood that the coefficient estimates are further from the correct population value.

In finance, conditional heteroskedasticity is often seen in the prices of stocks and bonds. The level of volatility of these equities cannot be predicted over any period. Unconditional heteroskedasticity can be used when discussing variables that have identifiable seasonal variability, such as electricity usage.

As it relates to statistics, heteroskedasticity (also spelled heteroscedasticity) refers to the error variance, or dependence of scattering, within a minimum of one independent variable within a particular sample. These variations can be used to calculate the margin of error between datasets, such as expected results and actual results, as it provides a measure of the deviation of data points from the mean value.

## Collinearity

Collinearity becomes a concern in regression analysis when there is a high correlation or an association between two potential predictor variables, when there is a dramatic increase in the p

value (i.e., reduction in the significance level) of one predictor variable when another predictor is included in the regression model, or when a high variance inflation factor is determined.

### Variance Inflation Factor

The variance inflation factor (VIF) provides a mechanism for determining the extent to which there is collinearity between two variables. Minimal collinearity (or none at all) is present for VIF values of 1 or 2; values between 5 and 15 indicate moderate collinearity; values greater than 20 indicate the presence of extreme collinearity. See also the following:

*https://medium.com/swlh/multicollinearity-and-variance-inflation-factor-bc74af36b1c9*

*https://medium.com/@tzjy/10-regression-metrics-data-scientist-must-know-tensorflow-keras-code-included-33f3ad53001c*

### Multicollinearity

Multicollinearity refers to the existence of two predictor variables whose inclusion in a model results in a decrease in statistical significance. The presence of multicollinearity can be detected via so-called variance inflation factors: values greater than 10 are indicative of a high degree of multicollinearity.

One complication regarding multicollinearity: it's possible for a pair of variables to have low correlation and yet contribute to multicollinearity that can only be detected through the model itself.

Also keep in mind that numerous statistical tests, such as the Student's `t-test` and `ANOVA` (discussed later), assume homoskedasticity for the data. In other words, such algorithms assume that samples from a population (or even different populations) have the same variance.

### Correlation

Correlation refers to the extent to which a pair of variables are related, which is a number between -1 and 1 inclusive. The most significant correlation values are -1, 0, and 1.

A correlation of 1 means that both variables increase and decrease in the same direction. A correlation of -1 means that both variables increase and decrease in the opposite direction. A correlation of 0 means that the variables are independent of each other.

`Pandas` provides the `corr()` method that generates a matrix containing the correlation between any pair of features in a data frame. Note that the diagonal values of this matrix are related to the variance of the features in the data frame.

A correlation matrix can be derived from a covariance matrix: each entry in the former matrix is a covariance value divided by the standard deviation of the two features in the row and column of a particular entry.

## WORKING WITH CURRENCY

The format for currency depends on the country, which includes different interpretations for a "," and "." in currency (and decimal values in general). For example, `1,124.78` equals "one thousand one hundred twenty-four point seven eight" in the United States, whereas `1.124,78` has the same meaning in Europe (i.e., the "." symbol and the "," symbol are interchanged).

If you need to combine data from datasets that contain different currency formats, then you probably need to convert all the disparate currency formats to a single common currency format. There is another detail to consider: currency exchange rates can fluctuate on a daily basis, which in turn can affect the calculation of taxes, late fees, and so forth. Although you might be fortunate enough where you won't have to deal with these issues, it's still worth being aware of them.

The next code sample shows you how to calculate the value between a pair of currencies, and also how to calculate the value of `Bitcoin` in `USD`. As an initial step, launch the following command from the command line:

```
pip3 install forex-python
```

Listing 1.6 displays the contents of `convert_currency.py` that illustrates how to determine some currency values.

***LISTING 1.6: convert_currency.py***

```
import pandas as pd

from forex_python.converter import CurrencyRates

curr = CurrencyRates()
usd = curr.get_rates('USD')
print("USD currency:",usd)
print()

#convert USD to EURO:
usd2eur = curr.get_rate('USD', 'EUR')
print("usd2eur:", usd2eur)

#rounded = print(round(usd2eur, 3))
#print("rounded:", rounded)

from forex_python.bitcoin import BtcConverter
# Bitcoin in USD:
btc = BtcConverter()
bcoin = btc.get_latest_price('USD')
print("bcoin:  ", bcoin)
```

Listing 1.6 starts with an `import` statement and then initializes the variable `curr` as an instance of the `CurrencyRates` class, and then initializes the variable `usd` with a set of USD currency rates.

The next code snippet initializes the currency exchange rate from `USD` to the `EUR` currency. The final code block initializes the variable `btc` as an instance of the `BtcConverter` class and then displays the latest Bitcoin price in `USD` currency. Now launch the code in Listing 1.6 and you will see the following output:

```
USD currency: {'EUR': 0.8880994671403198, 'JPY': 114.6714031971581,
'BGN': 1.7369449378330375, 'CZK': 21.611900532859682, 'DKK':
6.608081705150977, 'GBP': 0.7415452930728242, 'HUF': 316.50088809946715,
'PLN': 4.067850799289521, 'RON': 4.392984014209592, 'SEK':
9.275133214920071, 'CHF': 0.9213143872113678, 'ISK': 127.708703374778,
'NOK': 8.84884547069272, 'HRK': 6.685168738898757, 'RUB':
```

```
76.66412078152754, 'TRY': 13.378685612788633, 'AUD': 1.4092362344582594,
'BRL': 5.290586145648313, 'CAD': 1.2698934280639433, 'CNY':
6.36101243339254, 'HKD': 7.795648312611013, 'IDR': 14321.74955595027,
'INR': 74.77531083481351, 'KRW': 1202.1403197158081, 'MXN':
20.541385435168742, 'MYR': 4.185523978685613, 'NZD': 1.5126110124333927,
'PHP': 51.079928952042636, 'SGD': 1.349822380106572, 'THB':
33.195381882770874, 'ZAR': 15.242717584369451}

usd2eur: 0.8880994671403198
bcoin:   38515.2783
```

Listing 1.7 displays the contents of `convert_currency2.py` that illustrates how to display side-by-side countries and their currency conversion rates.

***LISTING 1.7: convert_currency2.py***

```
import pandas as pd

df = pd.DataFrame(data=['1000USD','2000EUR','3000EUR'],columns=['pri
ce'])
print("Dataset:")
print(df)
print()

cc = pd.DataFrame({'from':['EUR','USD'],'to':['USD','EUR'],'ra
te':[1.33,0.75]})
print("Conversion Table:")
print(cc)
print()
```

Listing 1.7 starts with an `import` statement to initialize the `Pandas` data frame `df` with three column titles, followed by the variable `cc` that contains currency conversion rates. The next code snippet displays the contents of `cc` in a tabular format. Now launch the code in Listing 1.7 and you will see the following output:

```
Dataset:
     price
0  1000USD
1  2000EUR
2  3000EUR

Conversion Table:
  from   to  rate
0  EUR  USD  1.33
1  USD  EUR  0.75
```

Listing 1.8 displays the contents of `mixed_currency.csv` and Listing 1.9 displays the contents of `convert_currency3.sh` that illustrates how to replace a "," with a "." to ensure that strings have a valid `USD` currency format.

***LISTING 1.8: mixed_currency.csv***

```
product1|1129.95
product2|2110,99
product3|1,130.95
product4|2,110.99
```

***LISTING 1.9: convert_currency3.sh***

```
filename="mixed_currency.csv"

echo "=> INITIAL CONTENTS OF CSV FILE:"
cat $filename
echo

echo "=> UPDATED CONTENTS OF CSV FILE:"
awk -F"|" '
BEGIN { modified=0 }
{
   comma  = index($2,",")
   period = index($2,".")
   OLD2=$2

   if(comma > 0 && period == 0) {
     gsub(/,/,".",$2)
     modified += 1
     #print "comma(s) but no period:", $2
   }
   else if( comma > period ) {
     # replace right-most "," with "Z"
     gsub(/,/,"Z",$2)
     # replace "." with ","
     gsub(/\./,",",$2)
     # replace "Z" with "."
     gsub(/Z/,".",$2)
     modified += 1
     #print "comma(s) before period:", $2
   }
   NEW2=$2
   printf("OLD: %18s NEW: %15s\n",OLD2, NEW2)
}
END { print "=> Modified lines:",modified }
' < mixed_currency.csv
```

Listing 1.9 starts by initializing the variable `filename` as `mixed_currency.csv` and then displays its contents. The next portion of Listing 1.9 is an `awk` script that initializes the variables `comma` and `period` with the index of a comma (",") and period (".") for every input line from the file `mixed_currency.csv`. Unlike other programming languages, there is no explicit loop keyword in the code: instead, it's an implicit aspect of the `awk` programming language (which might take some time to become accustomed to).

The next block of conditional code checks for the presence of a comma and the absence of a period: if so, then the `gsub()` function replaces the comma (",") with a period (".") in the second field, which is the numeric portion of each line in Listing 1.9, and the variable modified is incremented. For example, the input line `product3|2110,99` is processed by the conditional block and replaces the contents of $2—which is the second field—with the value `2110.99`.

The next portion of code checks for the presence of a comma and a period where the location of the comma is on the *right side* of the period: if so, then three substitutions are performed. First, the rightmost comma is replaced with the letter `Z`, after which the period is replaced with a comma, and then the `Z` is replaced with a period. Now launch the code in Listing 1.9 with the following code snippet:

```
./convert-currency3.sh
```

You will see the following output:

```
=> initial contents:
product1|1129.95
product2|2110,99
product3|1,130.95
product4|2,110.99

=> updated contents:
product1|1129.95
product2 2110.99
product3|1,130.95
product4|2,110.99
=> Modified lines: 1
```

## WORKING WITH DATES

The format for calendar dates varies among different countries, and this belongs to something called *localization* of data (not to be confused with `i18n`, which is data internationalization). Some examples of date formats are shown as follows (and the first four are probably the most common):

```
MM/DD/YY
MM/DD/YYYY
DD/MM/YY
DD/MM/YYYY
YY/MM/DD
M/D/YY
D/M/YY
YY/M/D
MMDDYY
DDMMYY
YYMMDD
```

If you need to combine data from datasets that contain different date formats, then converting the disparate date formats to a single common date format will ensure consistency.

Listing 1.10 displays the contents of `pandas_missing_dates.py` that illustrates how to display missing dates from a range of dates.

***LISTING 1.10: pandas_missing_dates.py***

```
import pandas as pd

# A dataframe from a dictionary of lists
data = {'Date': ['2022-01-18', '2022-01-20','2022-01-21','2022-01-24'],
        'Name': ['Joe', 'John', 'Jane', 'Jim']}
df = pd.DataFrame(data)

# Setting the Date values as index:
df = df.set_index('Date')

# to_datetime() converts string format to a DateTime object:
df.index = pd.to_datetime(df.index)

start_d="2022-01-18"
end_d="2022-01-25"

# display dates that are not in the sequence:
print("MISSING DATES BETWEEN",start_d,"and",end_d,":")
dates = pd.date_range(start=start_d, end=end_d).difference(df.index)

for date in dates:
  print("date:",date)
print()
```

Listing 1.10 starts with an `import` statement and then initializes the variable `data` as a dictionary of lists, which is used to initialize the `Pandas` data frame `df`. The next code snippet sets the field `Date` as the index, followed by converting the strings in `df` to `DateTime` objects, as shown here:

```
df.index = pd.to_datetime(df.index)
```

Then the variables `start_d` and `end_d` are initialized as string-based dates that represent a date range, after which the variable `dates` is initialized with a set of missing dates. The final loop displays the missing dates. Now launch the code in Listing 1.10 and you will see the following output:

```
MISSING DATES BETWEEN 2022-01-18 and 2022-01-25 :
date: 2022-01-19 00:00:00
date: 2022-01-22 00:00:00
date: 2022-01-23 00:00:00
date: 2022-01-25 00:00:00
```

Listing 1.11 displays the contents of `multiple_dates.csv` and Listing 1.12 displays the contents of `pandas_misc1.py` that determines the unique years in Listing 1.11.

***LISTING 1.11: multiple_dates.csv***

```
"dates","values"
2021-01-31,40
2021-02-28,45
2021-03-31,56
2021-04-30,NaN
```

```
2022-05-31,NaN
2022-06-30,140
2022-07-31,95
2022-08-31,40
2023-09-30,55
2023-10-31,NaN
2023-11-15,65
```

***LISTING 1.12: pandas_misc1.py***

```
df = pd.read_csv('multiple_dates.csv', parse_dates=['dates'])
print("df:")
print(df)
print()

the_years = df['dates']
year_list = set(the_years)

arr1 = np.array([])
for long_year in year_list:
  year = str(long_year)
  short_year = year[0:4]
  arr1 = np.append(arr1,short_year)

unique_years = set(arr1)
print("unique_years:")
print(unique_years)
print()

unique_arr = np.array(pd.DataFrame.from_dict(unique_years))
print("unique_arr:")
print(unique_arr)
print()
```

Listing 1.12 starts with an `import` statement and then initializes the `Pandas` data frame `df` with the contents of the `CSV` file `multiple_dates.csv`. After initializing the variable `the_years` with the `dates` feature in `df`, and also initializing `year_list` with the distinct values in the variable `the_years`, a loop extracts the year value for each date and appends this value to the `NumPy` array `arr1`.

Next, `unique_years` is populated with the distinct values in the variable `arr1` and then displayed. The final code snippet initializes the variable `unique_arr` with the unique years from the dictionary `unique_years` and the result is displayed. Now launch the code in Listing 1.12 and you will see the following output:

```
MISSING DATES BETWEEN 2022-01-18 and 2022-01-25
df:
        dates  values
0  2021-01-31    40.0
1  2021-02-28    45.0
2  2021-03-31    56.0
3  2021-04-30     NaN
4  2022-05-31     NaN
5  2022-06-30   140.0
```

```
6  2022-07-31    95.0
7  2022-08-31    40.0
8  2023-09-30    55.0
9  2023-10-31     NaN
10 2023-11-15    65.0

unique_years:
{'2022', '2023', '2021'}

unique_arr:
[['2022']
 ['2023']
 ['2021']]
```

Listing 1.13 displays the contents `standard_formats.csv` and Listing 1.14 displays the contents of `switching_date_formats.sh` that illustrates how to display missing dates from a range of dates.

***LISTING 1.13: standard_dates.csv***

```
2021-01-31
2021-02-28
2022-04-30
2022-05-31
2023-10-31
2023-11-15
```

***LISTING 1.14: switch_date_formats.sh***

```
import pandas as pd

file="standard_dates.csv"

echo "first output:"
cat $file | awk -F"-" ' { print $3 $2 $1 }'
echo

echo "second output:"
cat $file | awk -F"-" ' { print $3,$2,$1 }'
echo

echo "third output:"
cat $file | awk -F"-" ' { print $3 "-" $2 "-" $1 }'
```

Listing 1.14 starts by initializing the variable `file` with `standard_dates.csv` whose contents are shown in Listing 1.13. The remaining portion of Listing 1.14 consists of three short code blocks, each of which contains an `awk` command that specifies a hyphen ("-") as the delimiter for the dates. The values of `$1`, `$2`, and `$3` are the three hyphen delimited values in each input line. For example, `$1`, `$2`, and `$3` equal 2021, 01, and 31, respectively, for the first input line `2021-01-31`.

The first `awk` command displays the date fields in the order `$3`, `$2`, and `$1`. The second `awk` command also displays the date fields in the order `$3`, `$2`, and `$1`, along with a comma delimiter.

The third `awk` command also displays the date fields in the order `$3`, `$2`, and `$1`, along with a hyphen ("-") delimiter. Note that if you want to display the date fields in the order month, day, year, use the following statement: `print $2,$3,$1`. Now launch the code in Listing 1.14 and you will see the following output:

```
first output:
31012021
28022021
30042022
31052022
31102023
15112023

second output:
31 01 2021
28 02 2021
30 04 2022
31 05 2022
31 10 2023
15 11 2023

third output:
31-01-2021
28-02-2021
30-04-2022
31-05-2022
31-10-2023
15-11-2023
```

## SPLITTING AND SCALING DATA

As you now know, cleaning data tasks include checking for invalid data, duplicate data, and missing data. Additional data cleaning tasks can involve currency values and properly formatted dates.

After completing such tasks, you often need to resize the range of values for individual features, which is sometimes called "squashing" the data. The following subsections discuss data normalization and data standardization, as well as the importance of splitting data before performing any type of normalization.

Before you read any of the subsections, keep in mind that data normalization is different from database normalization: the latter pertains to various techniques for structuring database tables so that they conform to so-called "normal forms." Perform an Internet search for detailed information regarding database normalization.

### Why Normalize Data?

Normalization adjusts values of features in a dataset in order to establish a "level playing field," which is to say that the values in all the features are in the same general range. As an example of applying normalization to a dataset, suppose you to want to classify articles of clothing with these features (as well as others):

- the price is in the range of USD 5 to USD 250
- the number of distinct colors range from 1 to 5

Notice that if you scale the price range by dividing each price by 50 then the resulting set of prices are in the same range as the feature pertaining to the colors. Although the price range is a floating point number and the color can be assigned an integer value, the most important detail is that the values are in the same range.

The preceding example might seem slightly contrived because the scaled values have the same lower and upper bound, neither of which is a requirement. If the price values are in the range between 5 and 10 then the algorithm that you choose will perform better than the range between 5 and 250. The key point is to ensure that the range of values in different features differs by less than an order of magnitude (i.e., a multiple of 10).

## Split Before Normalizing Data

Normalization adjusts data to handle outliers. One technique is min-max normalization that first subtracts the mean from every value in a dataset, and then divides those values by the maximum value minus the minimum value. The result is a set of values that are between 0 and 1.

Although you might see code samples that perform normalization after other transformations have been applied to a column in a dataset, it's important to perform normalization as the first step. The rationale for doing so is that the training data and the test data will both be in the range between 0 and 1, which in turn prevents data leakage (discussed earlier in this chapter).

## Scaling Numeric Data via Normalization

A range of values can vary significantly and it's important to note that they often need to be scaled to a smaller range, such as values in the range `[-1,1]` or `[0,1]`, which you can do via the `tanh` function or the `sigmoid` function, respectively.

For example, measuring a person's height in terms of meters involves a range of values between 0.50 meters and 2.5 meters (in the vast majority of cases), whereas measuring height in terms of centimeters ranges between 50 centimeters and 250 centimeters: these two units differ by a factor of 100. A person's weight in kilograms generally varies between 5 kilograms and 200 kilograms, whereas measuring weight in grams differs by a factor of 1,000. Distances between objects can be measured in meters or in kilometers, which also differ by a factor of 1,000.

In general, use units of measure so that the data values in multiple features belong to a similar range of values. In fact, some machine learning algorithms require scaled data, often in the range of `[0,1]` or `[-1,1]`. In addition to the `tanh` and `sigmoid` function, there are other techniques for scaling data, such as "standardizing" data (think Gaussian distribution) and "normalizing" data (linearly scaled so that the new range of values is in `(0,1)`).

The following examples involve a floating point variable `X` with different ranges of values that will be scaled so that the new values are in the interval `[0,1]`.

- Example 1: if `X` is in the range [0,2], then `X`/2 is in the range [0,1].
- Example 2: if `X` is in the range [3,6], then `X`-3 is in the range [0,3], and (`X`-3)/3 is in the range [0,1].
- Example 3: if is `X` is in the range [-10,20], then `X` +10 is in the range [0,30], and (`X` +10)/30 is in the range of [0,1].

In general, suppose that `X` is a random variable whose values are in the range `[a,b]`, where `a < b`. You can scale the data values to the range `[0,1]` by performing two steps:

```
Step 1: X-a is in the range [0,b-a]
Step 2: (X-a)/(b-a) is in the range [0,1]
```

If X is a random variable that has the values {x1, x2, x3, . . . , xn}, then the formula for normalization involves mapping each xi value to (xi – min)/(max – min), where min is the minimum value of X and max is the maximum value of X.

As a simple example, suppose that the random variable X has the values {-1,0,1}. Then min and max are 1 and -1, respectively, and the normalization of {-1, 0, 1} is the set of values {(-1-(-1))/2, (0-(-1))/2, (1-(-1))/2}, which equals {0, 1/2, 1}.

## Scaling Numeric Data to the Range [a,b]

Given a set of numeric values X = {x1, x2, x3, . . . , xn} , with a minimum value of xmin and a maximum value of xmax, suppose that we want to scale these values so that they are between a and b (a < b). We can perform this transformation in two steps:

```
step 1:  xi => (xmax-xi)/(xmax-xmin)* (b-a) = vi
step 2:  vi => vi+a = wi
```

The values vi in step 1 are in the range [0,b-a] because each vi is the product of a rational number between 0 and 1 and the number b-a. Step 2 adds the value a to each vi, so the resulting set of numbers {w1, w2, . . . , wn} are in the range [a-b]. Here is a simple example:

```
X = [2, 4, 6, 10]
xmin = 2, xmax=10, a = 16, b=24
{vi} = {(10-2)/8, (10-4)/8, (10-6)/8, (10-10)/8}} * 8
     = {1, 3/4, 1/2, 0} * 8
     = {8, 6, 4, 0}
{wi} = {24, 22, 20, 16}
```

## Scaling Numeric Data via Standardization

The standardization technique involves finding the mean mu and the standard deviation sigma, and then mapping each xi value to (xi – mu)/sigma. Recall the following formulas:

```
mu = [SUM (x) ]/n
variance(x) = [SUM (x – xbar)*(x-xbar)]/n
sigma = sqrt(variance)
```

As a simple illustration of standardization, suppose that the random variable X has the values {-1, 0, 1}. Then mu and sigma are calculated as follows:

```
mu       = (SUM xi)/n = (-1 + 0 + 1)/3 = 0

variance = [SUM (xi- mu)^2]/n
         = [(-1-0)^2 + (0-0)^2 + (1-0)^2]/3
         = 2/3

sigma    = sqrt(2/3) = 0.816 (approximate value)
```

Hence, the standardization of {-1, 0, 1} is {-1/0.816, 0/0.816, 1/0.816}, which in turn equals the set of values {-1.2254, 0, 1.2254}.

As another example, suppose that the random variable X has the values {-6, 0, 6}. Then mu and sigma are calculated as follows:

```
mu       = (SUM xi)/n = (-6 + 0 + 6)/3 = 0

variance = [SUM (xi- mu)^2]/n
         = [(-6-0)^2 + (0-0)^2 + (6-0)^2]/3
         = 72/3
         = 24

sigma    = sqrt(24) = 4.899 (approximate value)
```

Hence, the standardization of {-6,0,6} is {-6/4.899, 0/4.899, 6/4.899}, which in turn equals the set of values {-1.2247, 0, 1.2247}.

In the preceding two examples, the mean equals 0 in both cases but the variance and standard deviation are significantly different. One other point: the normalization of a set of values *always* produces a set of numbers between 0 and 1.

On the other hand, the standardization of a set of values can generate numbers that are less than -1 and greater than 1: this will occur when sigma is less than the minimum value of every term |mu - xi|, where the latter is the absolute value of the difference between mu and each xi value. In the preceding example, the minimum difference equals 1, whereas sigma is 0.816, and therefore the largest standardized value is greater than 1.

## The StandardScaler Class

Listing 1.15 displays the contents of standard_scaler.py that illustrates how to use the StandardScaler class to scale values in a dataset.

***LISTING 1.15: standard_scaler.py***

```
from scikit-learn.preprocessing import StandardScaler

# initialize X_train, X_test, and y_train here:
# include your code here

ssx = StandardScaler()
X_train = ssx.fit_transform(X_train)
X_test  = ssx.transform(X_test)

ssy = StandardScaler()
y_train = ssy.fit_transform(y_train)
```

One detail to keep in mind: some machine learning algorithms do not require you to scale data, such as the following:

- AdaBoost
- Decision Trees
- Naive Bayes
- Random Forests

On the other hand, the following machine learning algorithms require you to explicitly scale data, such as the following:

- kNN (k nearest neighbors)
- Linear Regression
- Logistic Regression
- Neural Networks

## Scaling Numeric Data via Robust Standardization

The *robust standardization* technique is a variant of standardization that computes the mean `mu` and the standard deviation `sigma` based on a subset of values. Specifically, use only the values that are between the 25th percentile and 75th percentile, which ignores the first and fourth quartiles that might contain outliers. Let's define the following variables:

```
X25 = 25th percentile
X75 = 75th percentile
XM = mean of {Xi} values
XR = robust standardization
```

Then `XR` is computed according to the following formula:

```
XR = (Xi - XM)/(X75 - X25)
```

The preceding technique is also called `IQR`, which is an acronym for *interquartile range*, and you can see a sample calculation here:

*https://en.wikipedia.org/wiki/Interquartile_range*

## Deciding How to Scale Data

Scaling the values of features can improve the quality and predictive power of a model, which would otherwise be biased toward features with larger values. Feature scaling can be performed via normalization or standardization of features. Although we often assume that data is normally distributed, obviously there are exceptions.

Hence, try to ascertain the distribution of the data in features before deciding whether to use either standardization or normalization. For example, if a given feature appears to be uniformly distributed, use normalization (`MinMaxScaler` in `scikit-learn`).

By contrast, if the feature is approximately Gaussian, use standardization (`StandardScaler`). Again, note that whether you employ normalization or standardization, these are also approximative methods and are bound to contribute to the overall error of the model.

Keep in mind that the purpose of normalization is to scale values so that they lie in the interval `[0,1]`, whereas the purpose of standardization is to have a mean as 0 and standard deviation of 1. Although normalization is well suited for non-Gaussian distributions, it's sensitive to outliers.

## SUMMARY

This chapter started with an explanation of datasets, a description of data wrangling, and details regarding various types of data.

You also learned how to process inconsistent categorical data and how to map categorical data to numeric values. In addition, you learned about several types of dependencies among features, such as homoskedasticity and heroskedasticity, collinearity, variance inflation factor (VIF), multicollinearity, and correlation.

Furthermore, you learned how to process data that involves currency as well as date-related data. Then you learned about techniques for scaling numeric data, such as normalization and standardization, as well as the StandardScaler class in SkLearn.

CHAPTER 2

# OUTLIER AND ANOMALY DETECTION

This chapter explores outliers and anomalies, various types of algorithms that are available, and some `Python`-based code samples for detecting them.

The first part of this chapter discusses the relationship among fraud, anomalies, and outliers, along with `Python` code samples that illustrate how to find outliers with `NumPy` and `Pandas`, and also by calculating z-scores. The second section discusses fraud detection, which includes forty-one types of fraud.

## WORKING WITH OUTLIERS

In simplified terms, an *outlier* is an abnormal data value that is outside the range of "normal" values in a dataset. For example, a person's height in centimeters is typically between 30 centimeters and 250 centimeters. Hence, a data point (e.g., a row of data in a spreadsheet) with a height of 5 centimeters or a height of 500 centimeters is an outlier. The consequences of these outlier values are unlikely to involve a significant financial or physical loss (though they could adversely affect the accuracy of a trained model).

Outliers in a dataset are significantly larger or smaller than the inliers in a dataset. Outliers exist because of data variability or experimental errors or erroneous measurements. In addition, outliers can create issues during statistical analysis, such as adversely affecting the value of the mean and the standard deviation.

Three types of outliers are explained here:

*https://pub.towardsai.net/the-7-stages-of-preparing-data-for-machine-learning-dfe454da960b*

### Outliers Versus Data Drift

Data drift was discussed previously, and it refers to data that does not fit the pattern of previously observed data. In fact, data drift can involve data values that are significantly different and might be inadvertently classified as outliers.

This form of misclassification can occur in other scenarios, such as credit card purchases whereby a person who is living in San Francisco suddenly makes credit card purchases in

New York City. A one-time purchase could be an anomaly (i.e., a stolen credit card), or it could be a purchase made during a short stopover en route to a vacation in another city or country (i.e., a Type I error). A business trip or a vacation in New York City would probably involve a larger set of credit card purchases, and therefore normal purchases instead of credit card theft.

Consider a more extreme case: a customer on a business trip in New York City has his credit card stolen and credit card purchases are then made in San Francisco. The latter might escape detection because the customer lives in San Francisco (i.e., a Type II error). However, if multiple credit card purchases are made simultaneously in San Francisco and New York City during the same period of time, there is a greater risk of anomalous behavior because a spouse making credit card purchases with a card that is linked to the same bank account would have a different credit card number.

Incidentally, credit card companies *do* provide a telephone menu option to "notify us of any upcoming business or travel plans," which can help reduce the possibility of Type I or Type II errors associated with credit card purchases.

## Outlier Detection/Removal

Although the decision to keep or drop outliers is your decision to make, there are some techniques available that help you detect outliers in a dataset. Here is a short list of techniques, along with a very brief description and links for additional information:

- IQR
- z-score
- trimming
- winsorizing
- minimum covariance determinant
- local outlier factor
- Huber and Ridge
- isolation forest (tree-based algorithm)

The *IQR (interquartile range)* algorithm detects data points that are outside of 1.5 times of an interquartile range that either lie above the 3rd quartile or lie below the 1st quartile. Such points can be considered outliers.

The *z-score* for data points involves subtracting the mean and then dividing by the standard deviation:

```
z-score = (X-mean)/std
```

In general, z-scores that are greater than 3 are considered outliers, but you can adjust this to a value (e.g., 2.5 or 2) that is more suitable for your dataset.

Perhaps *trimming* is the simplest technique (apart from dropping outliers), which involves removing rows whose feature value is in the upper 5% range or the lower 5% range. *Winsorizing* the data is an improvement over trimming: set the values in the top 5% range equal to the maximum value in the 95th percentile, and set the values in the bottom 5% range equal to the minimum in the 5th percentile.

The *Minimum Covariance Determinant* is a covariance-based technique, and a `Python`-based code sample that uses this technique is downloadable here:

*https://scikit-learn.org/stable/modules/outlier_detection.html*

Two other techniques involve the *Huber* and the *Ridge* classes, both of which are included as part of `Sklearn`. Huber error is less sensitive to outliers because it's calculated via linear loss, similar to MAE (Mean Absolute Error). A code sample that compares Huber and Ridge is downloadable here:

*https://scikit-learn.org/stable/auto_examples/linear_model/plot_huber_vs_ridge.html*

You can also explore the Theil-Sen estimator and RANSAC that are "robust" against outliers, and additional information is here:

*https://scikit-learn.org/stable/auto_examples/linear_model/plot_theilsen.html*

*https://en.wikipedia.org/wiki/Random_sample_consensus*

Four algorithms for outlier detection are discussed here:

*https://www.kdnuggets.com/2018/12/four-techniques-outlier-detection.html*

One other scenario involves "local" outliers. For example, suppose that you use `kMeans` (or some other clustering algorithm) and determine that a value is an outlier with respect to one of the clusters. While this value is not necessarily an "absolute" outlier, detecting such a value might be important for your use case.

## Incorrectly Scaled Values Versus Outliers

You already know that an outlier is a value that is significantly different from the other values for a given feature. Now suppose that a numeric feature has a set of values in the range [90,100], but the correct range of values for this feature is [9,10]. Notice that the incorrect values do not contain any outliers, and also that those values can easily be scaled to the range [0,1] using the technique described in Chapter 1.

However, suppose that you are not a domain expert for the data in this dataset, so you do not realize that the initial values are out of range for that feature. As a result, you proceed to scale these data values so that they are in the range [0,1]. Although it's possible to train a model with this scaled dataset, the newly scaled values (as well as the initial values) are incorrect. Unfortunately, errors can arise when you perform other operations with this data, such as calculating the correlation between this feature and some other feature in the dataset. Hence, it's important to have domain knowledge in order to detect and rectify this type of error.

## Other Outlier Techniques

If you want to explore additional techniques for detecting outliers, the following techniques are also available:

- modified z-score
- MAD (median absolute deviation)
- Tukey's boxplot
- Carling median rule

In brief, the modified z-score provides a more fine-trained set of values that sometimes detects outliers that are not detected via a standard z-score. The MAD technique uses a median-based technique (instead of mean and variance values) that is less sensitive to outliers, which is better suited for non-normal distributions. Tukey's boxplot uses quartile values, whereas the Carling median rule uses median values. Perform an online search for more details, formulas, and code samples regarding the outlier techniques in the preceding bullet list.

## FINDING OUTLIERS WITH NUMPY

Although we have not discussed the `NumPy` library, we will only use the `NumPy array()` method, the `mean()` method, and the `std()` method in this section, all of which have intuitive functionality. Listing 2.1 displays the contents of `numpy_outliers1.py` that illustrates how to use `NumPy` methods to find outliers in an array of numbers.

***LISTING 2.1: numpy_outliers1.py***

```
import numpy as np

arr1 = np.array([2,5,7,9,9,40])
print("values:",arr1)

data_mean = np.mean(arr1)
data_std  = np.std(arr1)
print("data_mean:",data_mean)
print("data_std:" ,data_std)
print()

multiplier = 1.5
cut_off = data_std * multiplier
lower = data_mean - cut_off
upper = data_mean + cut_off
print("lower cutoff:",lower)
print("upper cutoff:",upper)
print()

outliers = [x for x in arr1 if x < lower or x > upper]
print('Identified outliers: %d' % len(outliers))
print("outliers:",outliers)
```

Listing 2.1 starts by defining a `NumPy` array of numbers and then calculates the mean and standard deviation of those numbers. The next block of code initializes two numbers that represent the upper and lower values that are based on the value of the cut_off variable. Any numbers in the array `arr1` that lie to the left of the lower value or to the right of the upper value are treated as outliers.

The final section of code in Listing 2.1 initializes the variable `outliers` with the numbers that are determined to be outliers, and those values are printed. Now launch the code in Listing 2.1 and you will see the following output:

```
values: [ 2  5  7  9  9 40]
data_mean: 12.0
data_std: 12.754084313139327
```

```
lower cutoff: -7.131126469708988
upper cutoff: 31.13112646970899

Identified outliers: 1
outliers: [40]
```

The preceding code sample specifies a hard-coded value in order to calculate the upper and lower range values.

Listing 2.2 is an improvement in that you can specify a set of values from which to calculate the upper and lower range values, and the new block of code is shown in bold.

***LISTING 2.2: numpy_outliers2.py***

```
import numpy as np

arr1 = np.array([2,5,7,9,9,40])
print("values:",arr1)

data_mean = np.mean(arr1)
data_std  = np.std(arr1)
print("data_mean:",data_mean)
print("data_std:" ,data_std)
print()

multipliers = np.array([0.5,1.0,1.5,2.0,2.5,3.0])
for multiplier in multipliers:
  cut_off = data_std * multiplier
  lower, upper = data_mean - cut_off, data_mean + cut_off
  print("=> multiplier:  ",multiplier)
  print("lower cutoff:",lower)
  print("upper cutoff:",upper)

  outliers = [x for x in df['data'] if x < lower or x > upper]
  print('Identified outliers: %d' % len(outliers))
  print("outliers:",outliers)
  print()
```

Listing 2.2 contains a block of new code that initializes the variable `multipliers` as an array of numeric values that are used for finding outliers. Although you will probably use a value of 2.0 or larger on a real dataset, this range of numbers can give you a better sense of detecting outliers. Now launch the code in Listing 2.2 and you will see the following output:

```
values: [ 2  5  7  9  9 40]
data_mean: 12.0
data_std: 12.754084313139327

lower cutoff: -7.131126469708988
upper cutoff: 31.13112646970899

Identified outliers: 1
outliers: [40]
=> multiplier:   0.5
lower cutoff: 5.622957843430337
upper cutoff: 18.377042156569665
```

```
Identified outliers: 3
outliers: [2, 5, 40]

=> multiplier:   1.0
lower cutoff: -0.7540843131393267
upper cutoff: 24.754084313139327
Identified outliers: 1
outliers: [40]

=> multiplier:   1.5
lower cutoff: -7.131126469708988
upper cutoff: 31.13112646970899
Identified outliers: 1
outliers: [40]

=> multiplier:   2.0
lower cutoff: -13.508168626278653
upper cutoff: 37.50816862627865
Identified outliers: 1
outliers: [40]

=> multiplier:   2.5
lower cutoff: -19.88521078284832
upper cutoff: 43.88521078284832
Identified outliers: 0
outliers: []

=> multiplier:   3.0
lower cutoff: -26.262252939417976
upper cutoff: 50.26225293941798
Identified outliers: 0
outliers: []
```

## FINDING OUTLIERS WITH PANDAS

Although we discuss `Pandas` in Chapter 3, the code in this section only involves a very simple `Pandas` data frame, the `mean()` method, and the `std()` method.

Listing 2.3 displays the contents of `pandas_outliers1.py` that illustrates how to use `Pandas` to find outliers in an array of numbers.

***LISTING 2.3: pandas_outliers1.py***

```
import pandas as pd

df = pd.DataFrame([2,5,7,9,9,40])
df.columns = ["data"]

print("=> complete data set:")
print(df)
print()

data_mean = df['data'].mean()
data_std  = df['data'].std()
print("=> data_mean:",data_mean)
```

```
print("=> data_std: ",data_std)
print()

multiplier = 1.5
cut_off = data_std * multiplier
lower, upper = data_mean - cut_off, data_mean + cut_off
print("=> lower cutoff:",lower)
print("=> upper cutoff:",upper)
print()

# outliers: method #1
outliers = [x for x in df['data'] if x < lower or x > upper]
print('=> Identified outliers: %d' % len(outliers))
print("=> outliers (#1):",outliers)
print()

# outliers: method #2
outliers = [x for x in df['data'] if x < lower or x > upper]
outliers = df[(df.data < lower) | (df.data > upper)]
print('=> Identified outliers: %d' % len(outliers))
print("=> outliers (#2):",outliers)
print()

# keep the inliers and drop the outliers:
df = df[(df.data > lower) & (df.data < upper)]
print("=> inliers without outliers:")
print(df)
print()
```

Listing 2.3 starts by defining a `Pandas` data frame and then calculates the mean and standard deviation of those numbers. The next block of code initializes two numbers that represent the upper and lower values that are based on the value of the `cut_off` variable. Any numbers in the data frame that lie to the left of the lower value or to the right of the upper value are treated as outliers.

The final section of code in Listing 2.3 initializes the variable `outliers` with the numbers that are determined to be outliers by means of a `Python` comprehension, and those values are printed, whereas the second technique accomplishes the same result without a `Python` comprehension. Now launch the code in Listing 2.3 and you will see the following output:

```
=> complete data set:
   data
0     2
1     5
2     7
3     9
4     9
5    40

=> data_mean: 12.0
=> data_std:  13.971399357258385

=> lower cutoff: -8.957099035887577
=> upper cutoff: 32.95709903588758
```

```
=> Identified outliers: 1
=> outliers (#1): [40]

=> Identified outliers: 1
=> outliers (#2):    data
5    40

=> inliers without outliers:
   data
0     2
1     5
2     7
3     9
4     9
```

The preceding code sample specifies a hard-coded value in order to calculate the upper and lower range values.

Listing 2.4 is an improvement in that you can specify a set of values from which to calculate the upper and lower range values, and the new block of code is shown in bold.

***LISTING 2.4: pandas_outliers2.py***

```
import pandas as pd

#df = pd.DataFrame([2,5,7,9,9,40])
#df = pd.DataFrame([2,5,7,8,42,44])

df = pd.DataFrame([2,5,7,8,42,492])
df.columns = ["data"]
print("=> data values:")
print(df['data'])

data_mean = df['data'].mean()
data_std  = df['data'].std()
print("=> data_mean:",data_mean)
print("=> data_std:" ,data_std)
print()

multipliers = [0.5,1.0,1.5,2.0,2.5,3.0]
for multiplier in multipliers:
  cut_off = data_std * multiplier
  lower, upper = data_mean - cut_off, data_mean + cut_off
  print("=> multiplier:  ",multiplier)
  print("lower cutoff:",lower)
  print("upper cutoff:",upper)

  outliers = [x for x in df['data'] if x < lower or x > upper]
  print('Identified outliers: %d' % len(outliers))
  print("outliers:",outliers)
  print()
```

Listing 2.4 contains a block of new code that initializes the variable `multipliers` as an array of numeric values that are used for finding outliers. Although you will probably use a value of 2.0 or larger on a real dataset, this range of numbers can give you a better sense of detecting outliers. Now launch the code in Listing 2.4 and you will see the following output:

```
=> data values:
0      2
1      5
2      7
3      8
4     42
5    492
Name: data, dtype: int64
=> data_mean: 92.66666666666667
=> data_std: 196.187325448579

=> multiplier:   0.5
lower cutoff: -5.42699605762283
upper cutoff: 190.76032939095617
Identified outliers: 1
outliers: [492]

=> multiplier:   1.0
lower cutoff: -103.52065878191233
upper cutoff: 288.85399211524566
Identified outliers: 1
outliers: [492]

=> multiplier:   1.5
lower cutoff: -201.6143215062018
upper cutoff: 386.9476548395352
Identified outliers: 1
outliers: [492]

=> multiplier:   2.0
lower cutoff: -299.7079842304913
upper cutoff: 485.0413175638247
Identified outliers: 1
outliers: [492]

=> multiplier:   2.5
lower cutoff: -397.80164695478084
upper cutoff: 583.1349802881142
Identified outliers: 0
outliers: []

=> multiplier:   3.0
lower cutoff: -495.8953096790703
upper cutoff: 681.2286430124036
Identified outliers: 0
outliers: []
```

## Calculating Z-Scores to Find Outliers

The `z-score` of a set of numbers is calculated by standardizing those numbers, which involves (a) subtracting their mean from each number, and (b) dividing by their standard deviation. Although you can perform these steps manually, the `Python SciPy` library simplifies the steps involved. If need be, you can install this package with the following command:

```
pip3 install scipy
```

Listing 2.5 displays the contents of `outliers_zscores.py` that illustrates how to find outliers in an array of numbers. As you will see, this code sample relies on convenience methods from `Numpy`, `Pandas`, and `SciPy`.

***LISTING 2.5: outliers_zscores.py***

```
import numpy as np
import pandas as pd
from scipy import stats

arr1 = np.array([2,5,7,9,9,40])
print("values:",arr1)

df = pd.DataFrame(arr1)

zscores = np.abs(stats.zscore(df))
print("z scores:")
print(zscores)
print()

upper = 2.0
lower = 0.5
print("=> upper outliers:")
print(zscores[np.where(zscores > upper)])
print()

print("=> lower outliers:")
print(zscores[np.where(zscores < lower)])
print()
```

Listing 2.5 starts with several `import` statements, followed by initializing the variable `arr1` as a `NumPy` array of numbers, and then displaying the values in `arr1`. The next code snippet initializes the variable `df` as a data frame that contains the values in the variable `arr1`.

Next, the variable `zscores` is initialized with the z-scores of the elements of the df data frame, as shown here:

```
zscores = np.abs(stats.zscore(df))
```

The next section of code initializes the variables `upper` and `lower`, and the z-scores whose values are less than the value of `lower` or greater than the value `upper` are treated as outliers, and those values are displayed. Now launch the code in Listing 2.5 and you will see the following output:

```
values: [ 2  5  7  9  9 40]
z scores:
[[0.78406256]
 [0.54884379]
 [0.39203128]
 [0.23521877]
 [0.23521877]
 [2.19537517]]

=> upper outliers:
[2.19537517]
```

```
=> lower outliers:
[0.39203128 0.23521877 0.23521877]
```

## FINDING OUTLIERS WITH SKLEARN (OPTIONAL)

This section is optional because the code involves the `EllipticEnvelope` class in `sklearn.covariance`, which we do not cover in this book. However, it's still worthwhile to peruse the code and compare this code with earlier code samples for finding outliers.

Listing 2.6 displays the contents of `elliptic_envelope_outliers.py` that illustrates how to use `Pandas` to find outliers in an array of numbers.

***LISTING 2.6: elliptic_envelope_outliers.py***

```
# pip3 install sklearn
from sklearn.covariance import EllipticEnvelope
import numpy as np

# Create a sample normal distribution:
Xdata = np.random.normal(loc=5, scale=2, size=10).reshape(-1, 1)
print("Xdata values:")
print(Xdata)
print()

# instantiate and fit the estimator:
envelope = EllipticEnvelope(random_state=0)
envelope.fit(Xdata)

# create a test set:
test = np.array([0, 2, 4, 6, 8, 10, 15, 20, 25, 30]).reshape(-1, 1)
print("test values:")
print(test)
print()

# predict() returns 1 for inliers and -1 for outliers:
print("envelope.predict(test):")
print(envelope.predict(test))
```

Listing 2.6 starts with several `import` statements and then initializes the variable `Xdata` as a column vector of random numbers from a Gaussian distribution. The next code snippet initializes the variable envelope as an instance of the `EllipticEnvelope` from `Sklearn` (which will determine if there are any outliers in `Xdata`) and is then trained on the data values in `Xdata`.

The next portion of Listing 2.6 initializes the variable `test` as a column vector, much like the initialization of `Xdata`. The final portion of Listing 2.6 makes a prediction on the values in the variable test and also displays the results: the value -1 indicates an outlet. Now launch the code in Listing 2.6 and you will see the following output:

```
Xdata values:
[[5.21730452]
 [5.49182377]
 [2.87553776]
 [4.20297013]
 [8.29562026]
```

```
 [5.78097977]
 [4.86631006]
 [5.47184212]
 [4.77954946]
 [8.66184028]]

test values:
[[ 0]
 [ 2]
 [ 4]
 [ 6]
 [ 8]
 [10]
 [15]
 [20]
 [25]
 [30]]

envelope.predict(test):
[-1  1  1  1  1 -1 -1 -1 -1 -1]
```

## FRAUD DETECTION

According to one estimate, worldwide fraud amounts to more than five trillion dollars, which is discussed in the following link:

*https://www.crowe.com/global/news/fraud-costs-the-global-economy-over-us$5-trillion*

Earlier sections in this chapter discussed how outliers differ from inliers in terms of their value, frequency, location, or some combination. On the other hand, inliers are common occurrences, which is to say, there is nothing unusual about the values for inliers. An outlier draws attention to the possibility of fraud but does not necessarily indicate that fraud has occurred.

An anomaly is also an outlier that is a more serious type of outlier: there is a greater chance that this type of outlier is also fraud. By way of analogy, consider a traffic light consisting of green (go), yellow (caution), and red (stop). An outlier can be in any of the following ranges:

- between green and red
- between green and yellow
- between yellow and red

As such, there are different levels of caution involved with anomalies and outliers. Specifically, an anomaly belongs to the third category, whereas a "benign" outlier (which is *not* an anomaly) is in the second category. Moreover, the collection of all types of outliers is in the first category. With the preceding points in mind, here is a short list of various types of fraud:

- credit card fraud
- payroll fraud
- insurance fraud

Although there is no single method for always determining fraud, there are some techniques for detecting potentially fraudulent transactions. For example, if you encounter a suspicious event for a customer, calculate the following values for that customer:

- total purchase amount for this day
- number of transactions for this day
- time of day for each transaction
- number of locations
- addresses of those locations

Now compare the values in the preceding list with the customer daily transaction patterns to see if there is a likely case of fraud. In case you're interested, the following link contains a list of forty-one types of fraud, along with techniques for fraud prevention:

*https://www.i-sight.com/resources/41-types-of-fraud-and-how-to-detect-and-prevent-them/*

## TECHNIQUES FOR ANOMALY DETECTION

In simplified terms, an anomaly is an outlier that has a more serious nature, such as credit card fraud, insurance fraud, or payroll fraud. As a simple example of a potential anomaly, suppose that someone who lives in California suddenly makes a credit card purchase in New York. If the person is on vacation (or a business trip), then the purchase is an outlier (it's "outside" the typical purchasing pattern), but it's not an issue. However, if that person was in California when the credit card purchase was made, then it's most likely to be credit card fraud, as well as an anomaly.

Unfortunately, there is no simple way to *decide* how to deal with anomalies and outliers in a dataset. Although you can drop rows that contain outliers, keep in mind that doing so might deprive the dataset—and therefore the trained model—of valuable information. You can try modifying the data values (described later), but again, this might lead to erroneous inferences in the trained model.

Another possibility is to train a model with the dataset that contains anomalies and outliers, and then train a model with a dataset from which the anomalies and outliers have been removed. Compare the two results and see if you can infer anything meaningful regarding the anomalies and outliers. With the preceding points in mind, here is a list of various techniques and algorithms for anomaly detection:

- LOF
- HBOS
- PyOD
- Numeric Outlier (IQR)
- z-score
- DBSCAN
- isolation forest

The *local outlier factor* (LOF) technique is an unsupervised technique that calculates a local anomaly score via the kNN (k nearest neighbor) algorithm. LOF is a density-based algorithm that is well-suited for finding local anomalies. Documentation and short code samples that use LOF are here:

*https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.LocalOutlierFactor.html*

*https://towardsdatascience.com/outlier-detection-theory-visualizations-and-code-a4fd39de540c*

*Anomaly detection with histogram-based outlier score* (HBOS) is an algorithm that constructs a histogram for a given random variable. Moreover, the histogram elements are normalized so that their height is between 0 and 1. The HBOS value is determined by calculating the sum of the logarithm of the outlier score for all the features, and you can find the formula here:

*https://www.dfki.de/fileadmin/user_upload/import/6431_HBOS-poster.pdf*

*Python outlier detection* (PyOD) is another open-source `Python`-based library for anomaly detection. PyOD actually computes anomaly detection scores for more than twenty anomaly detection algorithms (e.g., AutoEncoder, kNN, and PCA, among others). The anomaly scores are displayed in a tabular format so that you can quickly discern which algorithms have the best performance for a given dataset.

*Numeric outlier* is straightforward outlier detection technique that operates in a feature space consisting of a single dimension. This algorithm uses the IQR method to calculate the Q1 value and the Q3 value, which are the first and third quartile values, respectively. Any value that is outside the interval [Q1,Q3] is considered an outlier.

*Z-score* is a technique that uses a Gaussian distribution as the basis for the distribution of the data in a dataset, and the outliers are simply the points that lie in either extreme ("tail") of the distribution. A threshold value—often the value 3—is used to compare with the normalized data values, and those whose value is greater than 3 (or less than -3) are treated as outliers.

*Density-based spatial clustering of applications with noise* (DBSCAN) is a clustering-based technique for identifying outliers that classifies data points in three different groups: core points, border points or noise points, which are determined based on whether or not they have more than or fewer than MinPoints in a neighborhood of radius epsilon. Note that MinPoints and epsilon are hyperparameters in this algorithm, as well as the chosen distance measure. A code sample with DBSCAN is accessible here:

*https://donernesto.github.io/blog/outlier-detection-with-dbscan/*

An *isolation forest* (useful for large datasets) calculates an isolation number for each data point, which is the number of "cuts" that are required to "detach" a point from all other points in the dataset. For example, leaf nodes in a tree have an isolation number equal to 1 because their only connection is the edge to their respective parent. An example of an isolation forest is accessible here: *https://quantdare.com/isolation-forest-algorithm*

## SUMMARY

This chapter started with an explanation of outliers, which comprise a superset of anomalies (the latter are more significant and more problematic). You also learned about fraud detection, which includes forty-one different types of fraud. In addition, you learned about various algorithms for detecting and removing outliers, such as LOF, HBOS, PyOD, z-score, DBSCAN, and isolation forest.

CHAPTER 3

# DATA CLEANING TASKS

This chapter discusses data cleaning tasks involving datasets that contain various types of data, such as date formats, telephone numbers, and currency, all of which can have different formats. In addition, many of the code samples in this chapter reference techniques that are discussed in previous chapters of this book. This chapter provides a natural segue into chapter 4 that discusses data wrangling in various code samples.

The first part of this chapter briefly describes data cleaning, followed by examples of data cleaning tasks for data in the `MySQL` database table. Specifically, you will see how to replace `NULL` values with 0, how to replace `NULL` values with an average value, how to replace multiple values with a single value, how to handle mismatched attribute values, and how to convert strings to date values.

The second part of this chapter shows you how to use the `sed` command line utility to replace multiple delimiters in a `CSV` file with a single delimiter. You will also see how to use the `awk` command line utility to restructure a `CSV` file to create a file whose rows have the same number of fields.

The third part of this chapter shows you how to use the `awk` command line utility to process `CSV` files that have a variable number of columns. The `awk` command is a self-contained programming language, with a truly impressive capability for processing text files. If you are unfamiliar with the `awk` command, please read the appendix that contains an assortment of code samples that use the `awk` utility.

The fourth part of this chapter contains `awk`-based shell scripts that show you how to convert a list of phone numbers to the same format as well as a list of date formats to the same format.

This chapter assumes that you have read the data cleaning examples that are in previous chapters. Moreover, Chapter 7 contains a section with additional code samples involving data cleaning tasks.

## WHAT IS DATA CLEANING?

*Data cleaning*, also called data cleansing, is the task of ensuring that the contents of a dataset are complete, correct, and typically without duplicates. Hence, the focus of data cleaning is on individual files instead of combining or transforming data from two or more files. Data cleaning

is often performed before any data transformation is performed. In some cases, data cleaning must also be performed after a data transformation.

For example, suppose that a CSV file contains employee-related data and a MySQL table also contains employee-related data, both of which have been cleansed of inconsistencies and duplicates have been removed. However, after exporting the table data to a CSV file that is merged with the first CSV file, it's possible that there are duplicates that must then be removed.

Incidentally, there are several techniques for determining the values to replace empty fields, and the choice of techniques can range from making intuitively obvious changes to more subtle factors. Sometimes you can specify the mean or the median for missing values, but in other cases you need a more sophisticated technique. For example, suppose that a dataset with 1,000 rows consists of two types of patients: those who are healthy (the majority) as well as patients who have cancer. Obviously you want the number of sick patients to be as low as possible, which means that the dataset is fundamentally imbalanced.

Unfortunately, machine learning algorithms can produce inaccurate results with imbalanced datasets. Moreover, generating synthetic data whose feature values are based on the mean or the median is probably risky. A better technique is called SMOTE, which generates data values that are close to values that appear in rows of the original dataset.

As another example, the format for dates, currency, and decimal numbers varies among different countries. Examples of date formats include YYYY/MM/DD, MM/DD/YYYY, and DD/MM/YYYY (as well as other possible date formats). Incidentally, YYYY/MM/DD is an ISO standard for numeric dates.

Number formats involve a comma "," for the thousands position and a decimal "." for decimal values in the United States (e.g., $1,234.56), whereas Europe uses the opposite order for numbers (e.g., 1.234,56). Depending on the dataset in question, data cleaning can also involve working with an assortment of dates, currencies, and decimal numbers to ensure that all values have the same format.

Moreover, if two CSV files contain different date formats and you need to create a single CSV file that is based on the date columns, then there will be some type of conversion process that could be one of the following:

- convert the first date format to the second date format, or
- convert the second date format to the first date format, or
- convert both date formats to a third date format.

In the case of financial data, you are likely to also encounter different currencies, which involves a conversion rate between a pair of currencies. Since currency conversion rates fluctuate, you need to decide the exchange rate to use for the data, which can be:

- the exchange rate during the date that the CSV files were generated
- the current currency exchange rate
- some other mechanism

## Data Cleaning for Personal Titles

Sometimes a "brute force" solution is also the simplest solution, particularly when strings are involved. For example, a person's title can be misspelled in myriad ways that involve many pesky

little variations, so it's good to know how to perform this task in a manner that's simple, intuitive, and easy to extend with additional cases. For example, consider how you would replace each string in the following list with either "Mr","Ms", or "Mrs" before you look at the following potential solution:

```
titles = ['mr.','MR','MR.','mister','Mister','Ms','Ms.', 'Mr', 'Mr.',
'mr','MS','MS.','ms','ms.','Mis','miss','miss.','Mrs','Mrs.','mrs',
'mrs.','Madam','madam','ma"am']
```

While it's possible to solve this task with conditional logic that uses if/else code blocks, such an approach involves a lengthy code block. A simpler and easier solution involves the **in** keyword, as shown here:

```
mr_dict={}
ms_dict={}
mrs_dict={}

titles = ['mr.','MR','MR.','mister','Mister','Ms','Ms.', 'Mr', 'Mr.',
'mr','MS','MS.','ms','ms.','Mis','miss','miss.','Mrs','Mrs.','mrs',
'mrs.','Madam','madam','ma"am']

for title in ['Mr','Mr.','mr','mr.','MR','MR.','mister','Mister']:
  mr_dict[title] = "Mr"

for title in ['Ms','Ms.','MS','MS.','ms','ms.','Mis','miss','miss.']:
  ms_dict[title] = "Ms"

for title in ['Mrs','Mrs.','mrs','mrs.','Madam','madam','ma"am']:
  mrs_dict[title] = "Mrs"

print("Mr dictionary: ",mr_dict)
print()
print("Ms dictionary: ",ms_dict)
print()
print("Mrs dictionary:",mrs_dict)
```

Now launch the preceding code block and you will see the following output:

```
Mr dictionary:  {'Mr': 'Mr', 'Mr.': 'Mr', 'mr': 'Mr', 'mr.': 'Mr', 'MR':
'Mr', 'MR.': 'Mr', 'mister': 'Mr', 'Mister': 'Mr'}

Ms dictionary:  {'Ms': 'Ms', 'Ms.': 'Ms', 'MS': 'Ms', 'MS.': 'Ms', 'ms':
'Ms', 'ms.': 'Ms', 'Mis': 'Ms', 'miss': 'Ms', 'miss.': 'Ms'}


Mrs dictionary: {'Mrs': 'Mrs', 'Mrs.': 'Mrs', 'mrs': 'Mrs', 'mrs.':
'Mrs', 'Madam': 'Mrs', 'madam': 'Mrs', 'ma"am': 'Mrs'}
```

The previous solution makes it very easy to maintain, debug, and extend because the only change that's required is adding a new string in the appropriate location. Moreover, the given solution does not require any additional loops or regular expressions. In the event that you need a new category, such as "Sir," define the `Python` dictionary `sir_dict` and then add a new code snippet, as shown here:

```
if title in ['Sir','sir','Sire','sire','Yessir','yessir']:
  sir_dict[title] = "Sir"
```

## DATA CLEANING IN SQL

This section contains several subsections that perform data cleaning tasks in SQL. Note that it's not mandatory to perform these tasks in SQL: another option is to read the contents of a database table into a Pandas data frame and then use Pandas methods to achieve the same result.

However, this section illustrates how to perform the following data cleaning tasks that affect an attribute of a database table:

- replace NULL with 0
- replace NULL with the average value
- replace multiple values into a single value
- handle data type mismatch
- convert a string date to a date format

### Replace NULL With 0

This task is very straightforward, which you can perform with either of the following SQL statements:

```
SELECT ISNULL(column_name, 0 ) FROM table_name
OR
SELECT COALESCE(column_name, 0 ) FROM table_name
```

### Replace NULL Values With Average Value

This task involves two steps: first find the average of the non-NULL values of a column in a database table, and then update the NULL values in that column with the value that you found in the first step. Listing 3.1 displays the contents of the SQL script replace_null_values.sql that performs this pair of steps.

***LISTING 3.1: replace_null_values.sql***

```
USE mytools;
DROP TABLE IF EXISTS temperatures;
CREATE TABLE temperatures (temper INT, city CHAR(20));

INSERT INTO temperatures VALUES(78,'sf');
INSERT INTO temperatures VALUES(NULL,'sf');
INSERT INTO temperatures VALUES(42,NULL);
INSERT INTO temperatures VALUES(NULL,'ny');
SELECT * FROM temperatures;

SELECT @avg1 := AVG(temper) FROM temperatures;
update temperatures
set temper = @avg1
where ISNULL(temper);
SELECT * FROM temperatures;
```

```
-- initialize city1 with the most frequent city value:
SELECT @city1 := (SELECT city FROM temperatures GROUP BY city ORDER
BY COUNT(*) DESC LIMIT 1);

-- update NULL city values with the value of city1:
update temperatures
set city = @city1
where ISNULL(city);
SELECT * FROM temperatures;
```

Listing 3.1 creates and populates the table `temperatures` with several rows, and then initializes the variable `avg1` with the average temperature in the `temper` attribute of the `temperatures` table. Now launch the code in Listing 3.1 and you will see the following output:

```
+--------+------+
| temper | city |
+--------+------+
|     78 | sf   |
|   NULL | sf   |
|     42 | NULL |
|   NULL | ny   |
+--------+------+
4 rows in set (0.000 sec)

+----------------------+
| @avg1 := AVG(temper) |
+----------------------+
|         60.000000000 |
+----------------------+
1 row in set, 1 warning (0.000 sec)

Query OK, 2 rows affected (0.001 sec)
Rows matched: 2  Changed: 2  Warnings: 0

+--------+------+
| temper | city |
+--------+------+
|     78 | sf   |
|     60 | sf   |
|     42 | NULL |
|     60 | ny   |
+--------+------+
4 rows in set (0.000 sec)

+-------------------------------------------------------------------+
| @city1 := (SELECT city FROM temperatures GROUP BY city ORDER BY
COUNT(*) DESC LIMIT 1) |
+-------------------------------------------------------------------+
| sf
|
+-------------------------------------------------------------------+
1 row in set, 1 warning (0.000 sec)

Query OK, 1 row affected (0.000 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
+--------+------+
| temper | city |
+--------+------+
|     78 | sf   |
|     60 | sf   |
|     42 | sf   |
|     60 | ny   |
+--------+------+
4 rows in set (0.000 sec)
```

## REPLACE MULTIPLE VALUES WITH A SINGLE VALUE

An example of coalescing multiple values in an attribute involves replacing multiple strings for the state of New York (such as `new_york`, `NewYork`, `NY`, `New_York`, etc.) with `NY`. Listing 3.2 displays the contents of `reduce_values.sql` that performs this pair of steps.

***LISTING 3.2: reduce_values.sql***

```
use mytools;
DROP TABLE IF EXISTS mytable;
CREATE TABLE mytable (str_date CHAR(15), state CHAR(20), reply
CHAR(10));

INSERT INTO mytable VALUES('20210915','New York','Yes');
INSERT INTO mytable VALUES('20211016','New York','no');
INSERT INTO mytable VALUES('20220117','Illinois','yes');
INSERT INTO mytable VALUES('20220218','New York','No');
SELECT * FROM mytable;

-- replace yes, Yes, y, Ys with Y:
update mytable
set reply = 'Y'
where upper(substr(reply,1,1)) = 'Y';
SELECT * FROM mytable;

-- replace all other values with N:
update mytable
set reply = 'N' where upper(substr(reply,1,1)) != 'Y';
SELECT * FROM mytable;
```

Listing 3.2 creates and populates the table `mytable`, and then replaces the variants of the word yes with the letter `Y` in the reply attribute. The final portion of Listing 3.2 replaces any string that does *not* start with the letter `Y` with the letter `N`. Launch the code in Listing 3.2 and you will see the following output:

```
+----------+----------+-------+
| str_date | state    | reply |
+----------+----------+-------+
| 20210915 | New York | Yes   |
| 20211016 | New York | no    |
| 20220117 | Illinois | yes   |
| 20220218 | New York | No    |
+----------+----------+-------+
4 rows in set (0.000 sec)
```

```
Query OK, 2 rows affected (0.001 sec)
Rows matched: 2  Changed: 2  Warnings: 0

+----------+----------+-------+
| str_date | state    | reply |
+----------+----------+-------+
| 20210915 | New York | Y     |
| 20211016 | New York | no    |
| 20220117 | Illinois | Y     |
| 20220218 | New York | No    |
+----------+----------+-------+
4 rows in set (0.000 sec)

Query OK, 2 rows affected (0.001 sec)
Rows matched: 2  Changed: 2  Warnings: 0

+----------+----------+-------+
| str_date | state    | reply |
+----------+----------+-------+
| 20210915 | New York | Y     |
| 20211016 | New York | N     |
| 20220117 | Illinois | Y     |
| 20220218 | New York | N     |
+----------+----------+-------+
4 rows in set (0.001 sec)
```

## HANDLE MISMATCHED ATTRIBUTE VALUES

This task involves two steps: first find the average of the non-`NULL` values of a column in a database table, and then update the `NULL` values in that column with the value that you found in the first step.

Listing 3.3 displays the contents of `type_mismatch.sql` that performs this pair of steps.

***LISTING 3.3: type_mismatch.sql***

```
USE mytools;
DROP TABLE IF EXISTS emp_details;
CREATE TABLE emp_details (emp_id CHAR(15), city CHAR(20), state
CHAR(20));

INSERT INTO emp_details VALUES('1000','Chicago','Illinois');
INSERT INTO emp_details VALUES('2000','Seattle','Washington');
INSERT INTO emp_details VALUES('3000','Santa Cruz','California');
INSERT INTO emp_details VALUES('4000','Boston','Massachusetts');
SELECT * FROM emp_details;

select emp.emp_id, emp.title, det.city, det.state
from employees emp join emp_details det
WHERE emp.emp_id = det.emp_id;

--required for earlier versions of MySQL:
--WHERE emp.emp_id = cast(det.emp_id as INT);
```

Listing 3.3 creates and populates the table `emp_details`, followed by a `SQL JOIN` statement involving the tables `emp` and `emp_details`. Although the `emp_id` table is defined as an `INT` type and a `CHAR` type, respectively, in the tables `emp` and `emp_details`, the code works as desired. However, in earlier versions of `MySQL`, you need to use the built-in `CAST()` function in order to convert a `CHAR` value to an `INT` value (or vice versa), as shown in the commented-out code snippet:

```
--WHERE emp.emp_id = cast(det.emp_id as INT);
```

Now launch the code in Listing 3.3 and you will see the following output:

```
+--------+------------+---------------+
| emp_id | city       | state         |
+--------+------------+---------------+
| 1000   | Chicago    | Illinois      |
| 2000   | Seattle    | Washington    |
| 3000   | Santa Cruz | California    |
| 4000   | Boston     | Massachusetts |
+--------+------------+---------------+
4 rows in set (0.000 sec)
+--------+--------------------+------------+---------------+
| emp_id | title              | city       | state         |
+--------+--------------------+------------+---------------+
|   1000 | Developer          | Chicago    | Illinois      |
|   2000 | Project Lead       | Seattle    | Washington    |
|   3000 | Dev Manager        | Santa Cruz | California    |
|   4000 | Senior Dev Manager | Boston     | Massachusetts |
+--------+--------------------+------------+---------------+
4 rows in set (0.002 sec)
```

## CONVERT STRINGS TO DATE VALUES

Listing 3.4 displays the contents of `str_to_date.sql` that illustrates how to populate a date attribute with date values that are determined from another string-based attribute that contains strings for dates.

***LISTING 3.4: str_to_date.sql***

```
use mytools;
DROP TABLE IF EXISTS mytable;
CREATE TABLE mytable (str_date CHAR(15), state CHAR(20), reply
CHAR(10));

INSERT INTO mytable VALUES('20210915','New York','Yes');
INSERT INTO mytable VALUES('20211016','New York','no'););
INSERT INTO mytable VALUES('20220117','Illinois','yes'););
INSERT INTO mytable VALUES('20220218','New York','No'););

SELECT * FROM mytable;

-- 1) insert date-based feature:
ALTER TABLE mytable
```

```
ADD COLUMN (real_date DATE);
SELECT * FROM mytable;

-- 2) populate real_date from str_date:
UPDATE mytable t1
        INNER JOIN mytable t2
             ON t1.str_date = t2.str_date
SET t1.real_date = DATE(t2.str_date);
SELECT * FROM mytable;

-- 3) Remove unwanted features:
ALTER TABLE mytable
DROP COLUMN str_date;
SELECT * FROM mytable;
```

Listing 3.4 creates and populates the table `mytable` and displays the contents of this table. The remainder of Listing 3.4 consists of three `SQL` statements, each of which starts with a comment statement that explains its purpose.

The first `SQL` statement inserts a new column `real_date` of type `DATE`. The second `SQL` statement populates the `real_date` column with the values in the `str_date` column that have been converted to a date value via the `DATE()` function. The third `SQL` statement is optional: it drops the `str_date` column if you wish to do so. Now launch the code in Listing 3.4 and you will see the following output:

```
+----------+----------+-------+
| str_date | state    | reply |
+----------+----------+-------+
| 20210915 | New York | Yes   |
| 20211016 | New York | no    |
| 20220117 | Illinois | yes   |
| 20220218 | New York | No    |
+----------+----------+-------+
4 rows in set (0.000 sec)

Query OK, 0 rows affected (0.007 sec)
Records: 0  Duplicates: 0  Warnings: 0

+----------+----------+-------+-----------+
| str_date | state    | reply | real_date |
+----------+----------+-------+-----------+
| 20210915 | New York | Yes   | NULL      |
| 20211016 | New York | no    | NULL      |
| 20220117 | Illinois | yes   | NULL      |
| 20220218 | New York | No    | NULL      |
+----------+----------+-------+-----------+
4 rows in set (0.002 sec)

Query OK, 4 rows affected (0.002 sec)
Rows matched: 4  Changed: 4  Warnings: 0
```

```
+----------+----------+-------+------------+
| str_date | state    | reply | real_date  |
+----------+----------+-------+------------+
| 20210915 | New York | Yes   | 2021-09-15 |
| 20211016 | New York | no    | 2021-10-16 |
| 20220117 | Illinois | yes   | 2022-01-17 |
| 20220218 | New York | No    | 2022-02-18 |
+----------+----------+-------+------------+
4 rows in set (0.000 sec)

Query OK, 0 rows affected (0.018 sec)
Records: 0  Duplicates: 0  Warnings: 0

+----------+-------+------------+
| state    | reply | real_date  |
+----------+-------+------------+
| New York | Yes   | 2021-09-15 |
| New York | no    | 2021-10-16 |
| Illinois | yes   | 2022-01-17 |
| New York | No    | 2022-02-18 |
+----------+-------+------------+
4 rows in set (0.000 sec)
```

## DATA CLEANING FROM THE COMMAND LINE (OPTIONAL)

This section is marked "optional" because the solutions to tasks involve an understanding of some Unix-based utilities. Although this book only contains details about the awk utility, you can find online tutorials with examples regarding the other utilities (such as the tr and sed utility).

This section contains several subsections that perform data cleaning tasks that involve the "sed" and "awk" utilities:

- replace multiple delimiters with a single delimiter (sed)
- restructure a dataset so all rows have the same column count (awk)

Keep in mind the following point about these examples: they must be performed from the command line before they can be processed in a `Pandas` data frame.

### Working With the sed Utility

This section contains an example of how to use the `sed` command line utility to replace different delimiters with a single delimiter for the fields in a text file. You can use the same code for other file formats, such as `CSV` files and `TSV` files.

Keep in mind that this section does not provide any details about `sed` beyond the code sample in this section. However, after you read the code—it's a one-liner—you will understand how to adapt that code snippet to your own requirements (i.e., how to specify different delimiters).

Listing 3.5 displays the contents of `delimiter1.txt` and Listing 3.6 displays the contents of `delimiter1.sh` that replaces all delimiters with a comma (",").

***LISTING 3.5: delimiter1.txt***

```
1000|Jane:Edwards^Sales
2000|Tom:Smith^Development
3000|Dave:Del Ray^Marketing
```

***LISTING 3.6: delimiter1.sh***

```
cat delimiter1.txt | sed -e 's/:/,/' -e 's/|/,/' -e 's/\^/,/'
```

Listing 3.6 starts with the `cat` command line utility, which sends the contents of the file `delimiter1.txt` "standard output," which is the screen (by default). However, in this example the output of this command becomes the input to the `sed` command because of the pipe ("|") symbol.

The `sed` command consists of three parts, all of which are connected by the "-e" switch. You can think of "-e" as indicating "there is more processing to be done" by the `sed` command. In this example, there are three occurrences of "-e", which means that the `sed` command will be invoked three times.

The first code snippet is `'s/:/,/'`, which translates into "replace each semicolon with a comma." The result of this operation is passed to the next code snippet, which is `'s/|/,/'`. This code snippet translates into "replace each pipe symbol with a comma." The result of this operation is passed to the next code snippet, which is `'s/\^/,/'`. This code snippet translates into "replace each caret symbol ("^") with a comma." The result of this operation is sent to standard output, which can be redirected to another text file. Launch the code in Listing 3.6 and you will see the following output:

```
1000,Jane,Edwards,Sales
2000,Tom,Smith,Development
3000,Dave,Del Ray,Marketing
```

Three comments to keep in mind. First, the snippet contains a backslash because the caret symbol ("^") is a meta character, so we need to "escape" this character. The same is true for other meta characters, such as "$" and ".".

Second, you can easily extend the `sed` command for each new delimiter that you encounter as a field separator in a text file: simply follow the pattern that you see in Listing 3.6 for each new delimiter.

Third, redirect the output of `delimiter1.sh` to the text file `delimiter2.txt` by launching the following command:

```
./delimiter1.sh > delimiter2.txt
```

If an error occurs in the preceding code snippet, make sure that `delimiter1.sh` is executable by invoking the following command:

```
chmod 755 delimiter1.sh
```

This concludes the example involving the sed command line utility, which is a very powerful utility for processing text files. Check online for articles and blog posts if you want to learn more about the sed utility.

## WORKING WITH VARIABLE COLUMN COUNTS

This section shows you how to use the awk command line utility in order to "pad" rows in a CSV file with NaN values so that all records have the same number of columns.

Listing 3.7 displays the contents variable_columns.csv and Listing 3.8 displays the contents of variable_columns.sh that uses the awk utility to pad the number of columns in a CSV file.

***LISTING 3.7: variable_columns.csv***

```
10,20,30
10,20,30,40
10,20,30,40,50,60
```

***LISTING 3.8: variable_columns.sh***

```
filename="variable_columns.csv"

cat $filename | awk -F"," '
BEGIN { colCount = 6 }
{
  printf("%s", $0)
  for(i=NF; i<colCount; i++) {
     printf(",NaN")
  }
  print ""
}
'
```

Listing 3.8 initializes the variable filename with the name of the CSV file for this code sample. The next snippet is an awk script that initializes colCount with the value 6 in the BEGIN block: this value is the largest number of columns in any row of the CSV file.

The next block of code displays the contents of the current line, followed by a loop that prints a comma-separated list of NaN values to ensure that the output line contains 6 columns. For instance, if a row has 4 columns, then NaN will be printed twice. The print statement after the loop ensures that the next line from the input file starts on a new line instead of the current output line. Launch the code in Listing 3.8 and you will see the following output:

```
10,20,30,NaN,NaN,NaN
10,20,30,40,NaN,NaN
10,20,30,40,50,60
10,20,NaN,NaN,NaN,NaN
10,20,30,40,NaN,NaN
```

One limitation of Listing 3.8 is that the maximum number of columns must be specified in the BEGIN block. Listing 3.9 removes this constraint by scanning the entire file to determine the maximum number of columns in the CSV file.

***LISTING 3.9: variable_columns2.sh***

```
filename="variable_columns.csv"

cat $filename | awk -F"," '
BEGIN {
  maxColCount = 0;

  ##############################################
  # maxColCount = # of fields in the longest row
  ##############################################
  while(getline line < ARGV[1]) {
    colCount = split(line,data)
    if(maxColCount < length(data)) {
       maxColCount = length(data)
    }
  }
}
{
  # print current input line:
  printf("%s", $0)

  # pad with NaN (if necessary):
  for(j=NF; j<maxColCount; j++) {
     printf(",NaN")
  }
  print ""
}
' $filename
```

Listing 3.9 initializes the variable `filename` with the name of the `CSV` file for this code sample. The next portion of Listing 3.9 is an `awk` script that contains a while loop to process the input lines from the `CSV` file. The variable `maxColCount` is initialized with 0 in the `BEGIN` block, and when this loop has completed, its value will be the maximum number of columns of the lines in the input file.

The next portion of Listing 3.9 is the same as the loop in Listing 3.8, which prints the lines of text and pads them with `NaN`, whenever it's necessary to do so.

Launch the code in Listing 3.9 and you will see the following output:

```
10,20,30,NaN,NaN,NaN
10,20,30,40,NaN,NaN
10,20,30,40,50,60
10,20,NaN,NaN,NaN,NaN
10,20,30,40,NaN,NaN
```

## TRUNCATING ROWS IN CSV FILES

The previous section showed you how to use the `awk` command line utility in order to "pad" rows from a `CSV` file with the string `NaN` so that all records have the same number of columns, whereas this section shows you how to truncate the rows in a `CSV` file to display only the number of columns that are in the shortest row.

Listing 3.10 displays the contents of `variable_columns3.sh` that uses the `awk` utility to display a subset of columns in a `CSV` file.

***LISTING 3.10: variable_columns3.sh***

```
filename="variable_columns.csv"

cat $filename | awk -F"," '
BEGIN {
  colCount = 0; minColCount = 9999

  ###############################################
  # minColCount = # of fields in the shortest row
  ###############################################

  while(getline line < ARGV[1]) {
    colCount = split(line,data)
    if(minColCount > length(data)) {
       minColCount = length(data)
    }
  }
}
{
  # perform for each input line:
  for(j=1; j<=minColCount; j++) {
     printf("%s,",$j)
  }
  print ""
}
' $filename
```

Listing 3.10 is very similar to Listing 3.9: after initializing the variable `filename` with the name of the `CSV` file for this code sample, the `awk` script finds the number of columns in the row with the fewest columns from the `CSV` file.

The variable `minColCount` is initialized with 9999 in the `BEGIN` block, and when this loop has completed, its value will be the minimum number of columns of the lines in the input file.

The next portion of Listing 3.10 is a loop that prints the first `minColCount` columns in each line of the input file. Launch the code in Listing 3.10 and you will see the following output:

```
10,20,
10,20,
10,20,
10,20,
10,20,
```

## GENERATING ROWS WITH FIXED COLUMNS WITH THE AWK UTILITY

The code sample in this section contains an `awk` script that processes a space-delimited input string `CSV` file and generates output in which all rows have the same number of columns (with the possible exception of the final output row). Listing 3.11 displays the contents `FixedFieldCount1.sh` that illustrates how to use the `awk` utility in order to split a string into rows that contain three strings.

***LISTING 3.11: FixedFieldCount1.sh***

```
echo "=> pairs of letters:"
echo "aa bb cc dd ee ff gg hh"
echo

echo "=> split on multiple lines:"
echo "aa bb cc dd ee ff gg hh"| awk '
BEGIN { colCount = 3 }
{
  for(i=1; i<=NF; i++) {
     printf("%s ", $i)
     if(i % colCount == 0) { print "" }
  }
  print ""
}
'
```

Listing 3.11 displays the contents of a string, and then provides this string as input to the `awk` command. The main body of Listing 3.11 is a loop that iterates from 1 to `NF`, where `NF` is the number of fields in the input line, which in this example equals 8. The value of each field is represented by `$i`: `$1` is the first field, `$2` is the second field, and so forth. Note that `$0` is the contents of the *entire* input line (which is used in the next code sample).

Next, if the value of `i` (which is the field *position*, not the contents of the field) is a multiple of 3, then the code prints a linefeed. Launch the code in Listing 3.11 and you will see the following output:

```
=> pairs of letters:
aa bb cc dd ee ff gg hh

=> split on multiple lines:
aa bb cc
dd ee ff
gg hh
```

Listing 3.12 displays the contents of `employees.txt` and Listing 3.13 displays the contents of `FixedFieldCount2.sh` that illustrates how to use the `awk` utility in order to ensure that all the rows have the same number of columns.

***LISTING 3.12: employees.txt***

```
jane:jones:SF:
john:smith:LA:
dave:smith:NY:
sara:white:CHI:
>>>none:none:none<<<:
jane:jones:SF:john:
smith:LA:
dave:smith:NY:sara:white:
CHI:
```

***LISTING 3.13: FixedFieldCount2.sh***

```
cat employees.txt | awk -F":" '{printf("%s", $0)}' | awk -F':' '
BEGIN { colCount = 3 }
{
  for(i=1; i<=NF; i++) {
     printf("%s#", $i)
     if(i % colCount == 0) { print "" }
  }
}
'
```

Notice that the code in Listing 3.13 is almost identical to the code in Listing 3.11: the code snippet that is shown in bold removes the "\n" character from its input that consists of the contents of `employees.txt`.

In case you need to be convinced, launch the following code snippet from the command line:

```
cat employees.txt | awk -F":" '{printf("%s", $0)}'
```

The output of the preceding code snippet is shown here:

```
jane:jones:SF:john:smith:LA:dave:smith:NY:sara:white:CHI:>>>none:none:no
ne<<<:jane:jones:SF:john:smith:LA:dave:smith:NY:sara:white:CHI:
```

The reason that the "\n" has been removed in the preceding output is because of this code snippet:

```
printf("%s", $0)
```

If you want to retain the "`\n`" linefeed character after each input line, then replace the preceding code snippet with this snippet:

```
printf("%s\n", $0)
```

We have now reduced the task in Listing 3.13 to the same task as Listing 3.11, which is why we have the same awk-based code block. Now launch the code in Listing 3.13 and you will see the following output:

```
1000,Jane,Edwards,Sales
jane#jones#SF#
john#smith#LA#
dave#smith#NY#
sara#white#CHI#
>>>none#none#none<<<#
jane#jones#SF#
john#smith#LA#
dave#smith#NY#
Sara#white#CHI#
```

## CONVERTING PHONE NUMBERS

Listing 3.14 displays the contents of `phone_numbers.txt` that contains (mostly fictitious) phone numbers with different formats.

***LISTING 3.14: phone_numbers.txt***

```
1234567890
234 4560987
234 456 0987
212 555-1212
212-555-1212
(123)5551212
(456)555-1212
(789)555 1212
1-1234567890
1 234 4560987
1-234 456 0987
1 212 555-1212
1-212-555-1212
1 (123)5551212
1-(456)555-1212
1 (789)555 1212
011-1234567890
033 234 4560987
039-234 456 0987
034 212 555-1212
081-212-555-1212
044 (123)5551212
049-(456)555-1212
052 (789)555 1212
```

Listing 3.15 displays the contents of `phone_numbers.sh` that illustrates how to remove non-digit characters from the phone numbers in Listing 3.14 so that they have the same format.

***LISTING 3.15: phone_numbers.sh***

```
FILE="country_codes.csv"

#cat phone_numbers.txt |tr -d '()' | sed -e 's/ //g' -e 's/-//g' | awk
-F" " '
cat phone_numbers.txt |sed -e 's/[()]//g' -e 's/ //g' -e 's/-//g'| awk
-F" " '
{
   line_len = length($0)

   if(line_len == 10) {
     inter = ""
     area  = substr($0,0,3)
     xchng = substr($0,3,3)
     subsc = substr($0,7,4)
     printf("%s,%s,%s\n",area,xchng,subsc)
   } else if(line_len == 11) {
     inter = substr($0,0,1)
     area  = substr($0,1,3)
     xchng = substr($0,4,3)
     subsc = substr($0,8,4)
     printf("%s,%s,%s,%s\n",inter, area,xchng,subsc)
   } else if(line_len == 13) {
```

```
      inter = substr($0,0,3)
      area  = substr($0,3,3)
      xchng = substr($0,6,3)
      subsc = substr($0,10,4)
      printf("%s,%s,%s,%s\n",inter, area,xchng,subsc)
    } else {
      print "invalid format: ",$0
    }
}
'
```

Listing 3.15 initializes the variable `FILE` with the name of the `CSV` file that contains the three-digit international codes for a set of countries. The next code snippet is a pipe-delimited sequence of commands that starts by redirecting the contents of the file phone_numbers.txt to the sed command that removes all left parentheses, right parentheses, hyphens ("-"), and multiple occurrences of a whitespace.

The output from the `sed` command is redirected to an `awk` script that processes input lines of length 10, 11, and 13 that contain phone numbers that lack an international code, contain a single digit international code, and contain a three-digit international code, respectively. Any input line that has a different length is considered invalid. Now launch the code in Listing 3.15 and you will see the following output:

```
123,345,7890
234,445,0987
234,445,0987
212,255,1212
212,255,1212
123,355,1212
456,655,1212
789,955,1212
1,112,345,7890
1,123,445,0987
1,123,445,0987
1,121,255,1212
1,121,255,1212
1,112,355,1212
1,145,655,1212
1,178,955,1212
011,112,345,7890
033,323,445,0987
039,923,445,0987
034,421,255,1212
081,121,255,1212
044,412,355,1212
049,945,655,1212
052,278,955,1212
```

## CONVERTING NUMERIC DATE FORMATS

This section shows you how to convert date formats to a common format of the form `MM-DD-YYYY`. Before delving into the code, the following list contains sample formats for the month, day, and year of a date:

```
• yy:   two-digit year             (ex: 22)
• yyyy: four-digit year            (ex: 2022)
• m:    one-digit month            (ex: 4)
• mm:   two-digit month            (ex: 04)
• mmm:  three letters for month    (ex: Apr)
• mmmm: month spelled in full      (ex: April)
• d:    one-digit day of the month (ex: 2)
• dd:   two-digit day of the month (ex: 02)
• ddd:  three letter day of week   (ex: Sat)
• dddd: day spelled in full        (ex: Saturday)
```

Listing 3.16 displays the contents of `dates.txt` that contains fictitious dates in various formats, and the strings in bold have invalid formats.

***LISTING 3.16: dates.txt***

```
03/15/2021
3/15/2021
3/15/21
03/5/2021
3/5/2021
3/5/21
3/5/212
3/5/21z
```

Listing 3.17 displays the contents of `dates.sh` that shows how to remove non-digit characters from the dates in Listing 3.16 so that they have the same format.

***LISTING 3.17: dates.sh***

```
cat dates.txt | awk -F"/" '
{
   DATE_FORMAT="valid"
   # step 1: extract the month
   if($0 ~ /^[0-9]{2}/) {
     month = substr($0,1,2)
     #print "normal month: ",month
   } else if($0 ~ /^[0-9]\//) {
     month = "0" substr($0,1,1)
     #print "short month: ",month
   } else {
     DATE_FORMAT="invalid"
   }

   if(DATE_FORMAT="valid") {
     # step 2: extract the day
     if( $0 ~ /^[0-9][0-9]\/[0-9][0-9]/ ) {
       day = substr($0,4,2)
       #print "normal day: ",day
     } else if ( $0 ~ /^[0-9][0-9]\/[0-9]\// ) {
       day = "0" substr($0,4,1)
       #print "short day: ",day
     } else {
       DATE_FORMAT="invalid"
     }
```

```
    }

    if(DATE_FORMAT="valid") {
      # step 3: extract the year
      if( $0 ~ /^[0-9][0-9]\/[0-9][0-9]\/[0-9][0-9][0-9][0-9]$/ ) {
        year = substr($0,7,4)
        #print "normal year: ",year
      } else if ( $0 ~ /^[0-9][0-9]\/[0-9]\/[0-9][0-9]\/$/ ) {
        year = "20" substr($0,7,2)
        #print "short year: ",year
      }
    } else {
      DATE_FORMAT="invalid"
    }

    if(DATE_FORMAT="valid") {
      printf("=> $0: %s MM/DD/YYYY: %s-%s-%s\n",$0,month,day,year)
    } else {
      print "invalid format: ",$0
    }
}
'
```

Listing 3.17 contains code that might seem daunting if you are unfamiliar with regular expressions. Let's try to demystify the code, starting with the RE `[0-9]` that represents any single digit. The initial caret symbol in `^[0-9]\/` indicates that a digit must appear in the leftmost (i.e., first) position. Next, the RE `^[0-9]\/` indicates that a "/" must follow the single digit. Thus, the string `3/` matches the RE but the strings `03/`, `B3/`, and `AB/` do not match the RE.

Now let's examine that RE `^[0-9][0-9]\/` represents any string that starts with two digits and then a "/" character. Thus, the string `03/` matches the RE but the strings `3/`, `B3/`, and `AB/` do not match the RE.

Next, the RE `^[0-9][0-9]\/[0-9][0-9]` represents any string that starts with two digits, followed by a "/" character, and then two more digits. Thus, the string `03/15` matches the RE but the strings `3/5`, `03/5`, and `3/15` do not match the RE.

Finally, the RE `^[0-9][0-9]\/[0-9][0-9]\/[0-9][0-9][0-9][0-9]$` matches a string that:

- starts with two digits
- followed by a "/"
- another two digits
- followed by "/"
- four more digits
- no additional characters

As you can see, Listing 3.17 contains REs that match strings containing "short" months, days, and years. Now launch the code in Listing 3.17 and you will see the following output:

```
=> $0: 03/15/2021 MM/DD/YYYY: 03-15-2021
=> $0: 3/15/2021 MM/DD/YYYY: 03-15-2021
=> $0: 3/15/21 MM/DD/YYYY: 03-15-2021
```

```
=> $0: 03/5/2021 MM/DD/YYYY: 03-05-2021
=> $0: 3/5/2021 MM/DD/YYYY: 03-05-2021
=> $0: 3/5/21 MM/DD/YYYY: 03-05-2021
=> $0: 3/5/212 MM/DD/YYYY: 03-05-2021
=> $0: 3/5/21Z MM/DD/YYYY: 03-05-2021
```

Why are the preceding pair of lines (that are shown in bold) displayed as valid dates, even though the anchor meta character "$" appears in the two REs for extracting the year? Unfortunately, the latter two REs appear after the initial REs that extract the day value, and those two REs do *not* check for invalid year formats.

Listing 3.18 displays the contents of `dates2.sh` that detects invalid dates, uses a shorthand notation for multiple digits, and removes extraneous print-related statements.

***LISTING 3.18: dates2.sh***

```
cat dates.txt | awk -F"/" '
{
   DATE_FORMAT="valid"

   # step 1: check for invalid formats
   if($0 ~ /[A-Za-z]/) {
     print "invalid characters: ",$0
     DATE_FORMAT="invalid"
   } else if($0 ~ /\/[0-9]{3}$/) {
     print "invalid format: ",$0
     DATE_FORMAT="invalid"
   }

   if(DATE_FORMAT="valid") {
     # step 2: extract the month
     if($0 ~ /^[0-9][0-9]/) {
       month = substr($0,1,2)
     } else if($0 ~ /^[0-9]\//) {
       month = "0" substr($0,1,1)
     } else {
       DATE_FORMAT="invalid"
     }
   }

   if(DATE_FORMAT="valid") {
     # step 3: extract the day
     if( $0 ~ /^[0-9]{2}\/[0-9]{2}/ ) {
       day = substr($0,4,2)
     } else if ( $0 ~ /^[0-9]{2}\/[0-9]{2}\// ) {
       day = "0" substr($0,4,1)
     } else {
       DATE_FORMAT="invalid"
     }
   }

   if(DATE_FORMAT="valid") {
     # step 4: extract the year
     if( $0 ~ /^[0-9]{2}\/[0-9]{2}\/[0-9]{4}$/ ) {
```

```
      year = substr($0,7,4)
    } else if ( $0 ~ /^[0-9]{2}\/[0-9]\/[0-9]{2}\/$/ ) {
      year = "20" substr($0,7,2)
    }
  }

  if(DATE_FORMAT="valid") {
    printf("$0: %10s => %s-%s-%s\n",$0,month,day,year)
  } else {
    print "Date format invalid:",$0
  }
}
'
```

Listing 3.18 is very similar to contents of Listing 3.17 with some differences. The first difference is that Listing 3.18 checks for phone numbers that have alphabetic characters and reports them as having an invalid format. The second difference is the conditional code block that identifies any phone numbers that have three digits in the rightmost position.

The third difference is a shorthand way to specify multiple consecutive digits: [0-9]{2} matches any pair of consecutive digits, [0-9]{3} matches any occurrence of three consecutive digits, and so forth. Now launch the code in Listing 3.18 and you will see the following output:

```
$0: 03/15/2021 => 03-15-2021
$0:  3/15/2021 => 03-15-2021
$0:    3/15/21 => 03-15-2021
$0:  03/5/2021 => 03-15-2021
$0:   3/5/2021 => 03-15-2021
$0:     3/5/21 => 03-15-2021
invalid format:  3/5/212
$0:    3/5/212 => 03-15-2021
invalid characters:  3/5/21Z
$0:    3/5/21Z => 03-15-2021
```

## CONVERTING ALPHABETIC DATE FORMATS

This section shows you how to convert date formats to a common format of the form `DD-MON-YYYY`. Listing 3.19 displays the contents of `dates2.txt` that contains dates in various formats, and the strings in bold have invalid formats.

***LISTING 3.19: dates2.txt***

```
03/15/2021
04-SEP-2022
04-sep-  2022
04-sep-  22
05-OCT   2022
05-oct 2022
05-oct 22
06 JAN 2022
06 jan 2022
06 jnn 22
```

Listing 3.20 displays the contents of `dates3.sh` and illustrates how to remove non-digit characters from the dates in Listing 3.19 so that they have the same format.

***LISTING 3.20: dates3.sh***

```
cat dates2.txt  | tr -s ' ' |sed -e 's/- /-/g' -e 's/ /-/g' |awk -F"-" '
BEGIN {
   months["JAN"] = "JAN"
   months["FEB"] = "FEB"
   months["MAR"] = "MAR"
   months["APR"] = "APR"
   months["MAY"] = "MAY"
   months["JUN"] = "JUN"
   months["JUL"] = "JUL"
   months["AUG"] = "AUG"
   months["SEP"] = "SEP"
   months["OCT"] = "OCT"
   months["NOV"] = "NOV"
   months["DEC"] = "DEC"
}
{
   #Valid date formats (Oracle):
   #DD-MON-YY:   04-SEP-2022
   #DD-MON-YYYY: 05-OCT-22

   $0 = toupper($0)
   DATE_FORMAT="valid"

   # step 1: extract the day:
   if($0 ~ /^[0-9]\-/) {
     day = "0" substr($0,1,1)
   } else if($0 ~ /^[0-9][0-9]\-/) {
     day = substr($0,1,2)
   } else {
     DATE_FORMAT="invalid"
   }

   if(DATE_FORMAT="valid") {
     # step 2: extract the month:
     if( $0 ~ /^[0-9]{2}\-[A-Z]{3}/ ) {
       month = substr($0,4,3)
     } else {
       DATE_FORMAT="invalid"
     }
   }

   if(DATE_FORMAT="valid") {
     # step 3: extract the year:
     if( $0 ~ /^[0-9]{2}\-[A-Z]{3}\-[0-9]{2}$/ ) {
       year = "20" substr($0,8,2)
     } else if( $0 ~ /^[0-9]{2}\-[A-Z]{3}\-[0-9]{4}/ ) {
       year = substr($0,8,4)
     } else {
       DATE_FORMAT="invalid"
```

```
      }
   }

   if(DATE_FORMAT="valid") {
   if(months[month] == month) {
      printf("%12s => %s-%s-%s\n",$0,month,day,year)
   } else {
      printf("Invalid month:  %s-%s-%s\n",month,day,year)
   }
   } else {
     print "Date format invalid:",$0
   }
}
'
```

Listing 3.20 starts with a pipe-delimited sequence of commands that redirects the contents of the file `dates2.txt` to the `tr` command that removes multiple consecutive occurrences of whitespaces.

Next, the output from the `tr` command is redirected to the `sed` command that removes any whitespaces that follow the hyphen ("-") character, and then replaces any whitespaces with a hyphen ("-").

The output from the `sed` command is redirected to an `awk` script that initializes an array with the three letter abbreviations of the months of the year. This array is referenced later in the code in order to detect any invalid month formats.

The main portion of the `awk` script is similar to the contents of Listing 3.27 for detecting various date formats in order to initialize the day, month (as a three-letter value), and year for each input line. Now launch the code in Listing 3.20 and you will see the following output:

```
04-SEP-2022 => SEP-04-2022
04-SEP-2022 => SEP-04-2022
04-SEP-22   => SEP-04-2022
05-OCT-2022 => OCT-05-2022
05-OCT-2022 => OCT-05-2022
05-OCT-22   => OCT-05-2022
06-JAN-2022 => JAN-06-2022
06-JAN-2022 => JAN-06-2022
Invalid month: JNN-06-2022
```

## WORKING WITH DATE AND TIME DATE FORMATS

Now that you have seen code samples with dates of the form `MM:DD:YYYY`, this section handles dates of the form `YYYY:MM:DD:HH:MM:SS`. Although the `awk`-based code sample in this section is much longer than all other code samples in this book, it's based on techniques that you have already seen in previous date-related code samples. If you do not need to delve into the details of this code sample at this time, feel free to skip this section with no loss of continuity.

Single digit values for all fields (except the year) are considered valid, and handling all the possible combinations becomes quite tedious. However, the code sample in this section shows you a shortcut that involves dynamically modifying the input line so that it's progressively padded as the input string is processed in a left-to-right fashion.

To make sure this is clear, the string `3/15/22 5:5:44` is a valid date, and as the code checks for the validity of the individual date-related fields, the preceding string is successively modified as follows:

```
3/15/22 5:5:44
03/15/22 5:5:44
03/15/2022 5:5:44
03/15/2022 5:5:44
03/15/2022 05:5:44
03/15/2022 05:05:44
```

As you can see, the final row in the preceding list of dates is a "fully padded" date. This approach significantly reduces the number of patterns that need to be checked in order to determine whether or not a date has a valid format.

Listing 3.21 displays the contents of `dates-times.txt` that contains dates, and Listing 3.22 shows you the contents of `date-times-padded.sh`, which is an `awk`-based shell script that processes the dates in Listing 3.21.

***LISTING 3.21: date-times.txt***

```
2/30/2020 15:05:44
2/30/2020 15:05:44
3/15/22 15:5:44
4/29/23 15:5:4
5/16/24 5:05:44
6/17/25 5:5:44
7/18/26 5:5:4
8/19/24 115:05:44
```

Notice that the first and last lines in Listing 3.21 are invalid: the first row contains a value of 30 for February, which is invalid for any year, and the last row contains an invalid value for the hour.

Listing 3.22 displays the contents of `date-times-padded.sh` that determines which dates in Listing 3.21 have the format `YYYY:MM:DD:HH:MM:SS`.

***LISTING 3.22: date-times-padded.sh***

```
cat date-times.txt | awk -F"/" '
function check_leap_year(year) {
  ########################################
  # A year is a leap year if the following:
  # 1) it is a multiple of 4 AND
  # 2) a century must be a multiple of 400
  # => 2000 is a leap year but 1900 is not.
  ########################################

  result = 0 # 0: non-leap year 1: leap year
  if((year % 4) == 0) {
    if((year % 100) == 0) {
      if((year % 400) == 0) {
        return 1 # leap year
      } else {
```

```
        return 0 # non-leap year
      }
    } else {
      return 1 # leap year
    }
  } else {
    return 0 # non-leap year
  }
}

BEGIN {
  count = 1
  for(i=0;i<12;i++) {
    months[i] = 31
  }

  # 30 days: april,june,september,november
  months[1]  = 28;
  months[3]  = 30;
  months[5]  = 30;
  months[8]  = 30;
  months[10] = 30;

  #for(i=0;i<12;i++) {
  #  print "months[",i,"]:",months[i]
  #}
}
{
   ###################################
   # valid month format:
   # [0-9], [1][0-2]
   #
   # valid day format:
   # [0-9], [1][0-9], [3][0-1]
   #
   # valid year format:
   # [0-9]{2}, [2][0-9][0-9][0-9]
   #
   # Additional comments:
   # 1) 30 versus 31 days in a month
   # 2) check february:  28 vs 29 days
   ###################################

   # sample format:
   # MM/DD/YYYY HH:MM:SS
   print "=> #" count " PROCESSING:",$0

   VALID_DATE="true"
   split($0,day_time," ")
   date_part = day_time[1]
   time_part = day_time[2]

   # step 1: extract the month
   if(date_part ~ /^[0-9]\//) {
```

```
  month = "0" substr(date_part,1,1)
  #print "short month: ",month
  # insert a "0" in the date part:
  date_part = "0" substr(date_part,1,1) substr(date_part,2)
  #print "*** new date_part:",date_part
} else if(date_part ~ /^[0-9]{2}/) {
  month = substr(date_part,1,2)
  #print "normal month: ",month
} else {
  print "Cannot find month"
  VALID_DATE="false"
}

if(VALID_DATE == "true") {
  # step 2: extract the day: #03/15/2021 15:05:44
  #print "checking for day:",date_part
  if( date_part ~ /^[0-9]{2}\/[0-9]{2}/ ) {
    day = substr(date_part,4,2)
    #print "1normal day: ",day
  } else if ( date_part ~ /^[0-9]{2}\/[0-9]/ ) {
    day = "0" substr(date_part,4,1)
    #print "1short day: ",day
    date_part = substr(date_part,1,3) "0" substr(date_part,4)
    #print "*** 2new date_part:",date_part
  } else if ( date_part ~ /^[0-9]{1}\/[0-9]{2}/ ) {
    day = substr(date_part,3,2)
    #print "2normal day: ",day
  } else if ( date_part ~ /^[0-9]{1}\/[0-9]/ ) {
    day = "0" substr(date_part,2,1)
    #print "2short day: ",day
  } else {
    print "Cannot find day"
    VALID_DATE="false"
  }
}

if(VALID_DATE == "true") {
  # step 3: extract the year: #03/15/2021 15:05:44
  #print "date part:",date_part  # 03/15/2021
  #print "time part:",time_part  # 15:05:44

  if( date_part ~ /^[0-9]{2}\/[0-9]{2}\/[0-9]{4}/ ) {
    year = substr(date_part,7,4)
    #print "normal year: ",year
  } else if ( date_part ~ /^[0-9]{2}\/[0-9]{2}\/[0-9]{2}/ ) {
    year = "20" substr(date_part,7,2)
    #print "1short year: ",year
    date_part = substr(date_part,1,6) "20" substr(date_part,7)
    #print "*** 3new date_part:",date_part
  } else {
    print "Cannot find year"
    VALID_DATE="false"
  }
}
```

```
if(VALID_DATE == "true") {
  #print "step 4 time_part:",time_part
  # step 4: extract the hours: #15:05:44
  if(time_part ~ /^[0-9]{2}:/) {
    hours = substr(time_part,1,2)
    #print "normal hours: ",hours
  } else if(time_part ~ /^[0-9]:/) {
    hours = "0" substr(time_part,1,1)
    #print "short hours: ",hours
    time_part = "0" substr(time_part,1)
    #print "*** 3new time_part:",time_part
  } else {
    print "no matching hours"
    VALID_DATE = "false"
  }
}

if(VALID_DATE == "true") {
  # step 5: extract the minutes: #15:5:44
  if( time_part ~ /^[0-9]{2}:[0-9]{2}/ ) {
    minutes = substr(time_part,4,2)
    #print "normal minutes: ",minutes
  } else if ( time_part ~ /^[0-9]{2}:[0-9]:/ ) {
    minutes = "0" substr(time_part,4,1)
    #print "short minutes: ",minutes
    time_part = substr(time_part,1,3) "0" substr(time_part,4)
    #print "*** 4new time_part:",time_part
  } else {
    print "no matching minutes"
    VALID_DATE = "false"
  }
}

if(VALID_DATE == "true") {
  # step 6: extract the seconds: #15:05:44
  if( time_part ~ /^[0-9]{2}:[0-9]{2}:[0-9]{2}/ ) {
    seconds = substr(time_part,7,2)
    #print "normal seconds: ",seconds
  } else if( time_part ~ /^[0-9]{2}:[0-9]{2}:[0-9]{1}/ ) {
    seconds = "0" substr($0,7,1)
    #print "short seconds: ",seconds
    time_part = substr(time_part,1,6) "0" substr(time_part,7)
    #print "*** 5new time_part:",time_part
  } else {
    print "no matching seconds"
    VALID_DATE = "false"
  }
}

if(VALID_DATE == "true") {
  result = check_leap_year(year)
  if(result == 1) {
     #print "found leap year:",year
     # is day <= 29?
     if( day <= 29) {
```

```
          print "=> VALID DAY/TIME FORMAT: ",$0
        } else {
          print "*** Leap year day out of bounds:",$0
        }
      } else {
         #print "found non-leap year:",year
         # is day <= 28?
         if( day <= 28) {
           print "=> VALID DAY/TIME FORMAT: ",$0
         } else {
           print "*** Non-leap year day out of bounds:",$0
         }
      }
   } else {
     print "invalid day/time format: ",$0
   }

   print "----------------\n"
   count += 1
}
'
```

Now launch the code in Listing 3.22 and you will see the following output:

```
=> #1 PROCESSING: 2/30/2020 15:05:44
*** Leap year day out of bounds: 2/30/2020 15:05:44
----------------

=> #2 PROCESSING: 3/15/22 15:5:44
=> VALID DAY/TIME FORMAT:  3/15/22 15:5:44
----------------

=> #3 PROCESSING: 4/29/23 15:5:4
*** Non-leap year day out of bounds: 4/29/23 15:5:4
----------------

=> #4 PROCESSING: 5/16/24 5:05:44
=> VALID DAY/TIME FORMAT:  5/16/24 5:05:44
----------------

=> #5 PROCESSING: 6/17/25 5:5:44
=> VALID DAY/TIME FORMAT:  6/17/25 5:5:44
----------------

=> #6 PROCESSING: 7/18/26 5:5:4
=> VALID DAY/TIME FORMAT:  7/18/26 5:5:4
----------------

=> #7 PROCESSING: 8/19/24 115:05:44
no matching hours
invalid day/time format:  8/19/24 115:05:44
----------------
```

In case you're interested, the companion files contain `date-times-padded2.sh` that enhances `date-times-padded.sh` to provide addition information, as shown here:

```
INVALID DATES:
8/19/24 115:05:44
2/30/2020 15:05:44
4/29/23 15:5:4
---------------
VALID DATES:
6/17/25 5:5:44
5/16/24 5:05:44
7/18/26 5:5:4
3/15/22 15:5:44

YEARS IN VALID DATES:
2022
2024
2025
2026
MONTHS IN VALID DATES:
03
05
06
07
DAYS IN VALID DATES:
15
16
17
18
```

## WORKING WITH CODES, COUNTRIES, AND CITIES

This section shows you how to use the awk utility to manage CSV files that contain information about countries, cities, and telephone codes. The international telephone codes are available here:

*https://www.internationalcitizens.com/international-calling-codes/*

Listing 3.23 displays the contents of `country_codes.csv` that contains the international prefix for several countries. For simplicity, the three-digit prefix has been left-padded with 0, because some countries have a two-digit prefix and other countries have a three-digit prefix.

***LISTING 3.23: country_codes.csv***

```
001,usa
033,france
034,spain
039,italy
044,uk
049,germany
052,mexico
081,japan
```

Listing 3.24 displays the contents of add_country_codes.sh that illustrates how to increment the three-digit international prefix for a set of countries. Note that you won't need this code sample beyond this section: it's included here to show you how easily you can manipulate the numeric values in a CSV file with the awk utility. If need be, you can easily adapt this code to work with other CSV files.

***LISTING 3.24: add_country_codes.sh***

```
FILE="country_codes.csv"

echo "=> display code and country:"
awk -F"," '
BEGIN { code = 10000; incr = 10000 }
{
   printf("%s,%s,%d\n", $1, $2, code)
   code += incr
}
' < $FILE
echo "-----------------"
echo

echo "=> increment country code:"
awk -F"," '
BEGIN { code = 1000; incr = 1000 }
{
   printf("%d,%s\n", $1 + code, $2)
   code += incr
}
' < $FILE
```

Listing 3.24 initializes the variable FILE with the name of the CSV file that contains country codes and abbreviations for several countries. Next, an awk script contains a BEGIN section with a loop that adds 10000 to each country code. For example, the following two code snippets display the "before" and "after" contents of an input line:

```
001,usa,10000
1001,usa
```

Now launch the code in Listing 3.24 and you will see the following output:

```
=> display code and country:
001,usa,10000
033,france,20000
034,spain,30000
039,italy,40000
044,uk,50000
049,germany,60000
052,mexico,70000
081,japan,80000
-----------------
```

```
=> increment country code:
1001,usa
2033,france
3034,spain
4039,italy
5044,uk
6049,germany
7052,mexico
8081,japan
```

Listing 3.25 displays the contents of `countries_cities.csv` in which each row consists of a country and a list of cities in that country.

***LISTING 3.25: countries_cities.csv***

```
italy,firenze,milano,roma,venezia
france,antibe,nice,paris,st_jeannet
germany,berlin,frankfurt
spain,barcelona,madrid
england,liverpool,london,manchester
mexico,mexico_city,tijuana
```

Listing 3.26 displays the contents of `split_countries_cities.sh` and illustrates how to display a list of cities that belong to each country in `countries_cities.csv`.

***LISTING 3.26: split_countries_codes.sh***

```
FILE="countries_cities.csv"

awk -F"," '
{
   printf("=> CITIES in %s:\n",$1)
   for(i=2; i<=NF; i++) {
        printf("%s\n", $i)
   }
}
' < $FILE
```

Listing 3.26 initializes the variable `FILE` with the name of the `CSV` file that contains country codes and abbreviations for several countries. Next, an `awk` script contains a loop that displays the abbreviation of each city that is listed in the `CSV` file `countries_cities.csv`. Launch the code in Listing 3.26 and you will see the following output:

```
=> display code and country:
=> CITIES in italy:
firenze
milano
roma
venezia
=> CITIES in france:
antibe
nice
```

```
paris
st_jeannet
=> CITIES in germany:
berlin
frankfurt
=> CITIES in spain:
barcelona
madrid
=> CITIES in england:
liverpool
london
manchester
=> CITIES in mexico:
mexico_city
tijuana
Zzzz
```

Listing 3.27 displays the contents of `countries_cities2.csv` in which each row consists of a country and a list of cities in that country.

***LISTING 3.27: countries_cities2.csv***

```
italy,firenze,milano,roma,venezia
france,antibe,nice,paris,st_jeannet
germany,berlin,frankfurt
spain,barcelona,madrid
england,liverpool,london,manchester
mexico,mexico_city,tijuana
usa,chicago,illinois,denver,colorado,seattle,washington,vancouver,
washington
can,vancouver,bc,edmonton,calgary,hamilton,ontario
```

Listing 3.28 displays the contents of `split_countries_cities2.sh` and illustrates how to display a list of cities that belong to each country in `countries_cities2.csv`.

***LISTING 3.28: split_countries_codes2.sh***

```
FILE="countries_cities2.csv"

awk -F"," '
BEGIN { incr = 1000 }
{
   if( $1 !~ /#/ ) {
     printf("=> CITIES in %s:\n",$1)
     for(i=2; i<=NF; i++) {
          printf("%s\n", $i)
     }
     print("------------\n")
   } else {
     #printf("=> CITIES in %s:\n",$1)
     printf("=> CITIES in %s:\n",substr($1,2))
     for(i=2; i<=NF; i+=2) {
```

```
            printf("%s,%s\n", $i,$(i+1))
        }
        print("------------\n")
    }
}
' < $FILE
```

Listing 3.28 initializes the variable `FILE` with the name of the `CSV` file that contains country codes and abbreviations for several countries. Note that this `CSV` file differs from a similar `CSV` file with countries and country codes. Specifically, this `CSV` file contains rows that have either city/province pairs (Canada) or city/state pairs (USA).

Next, an `awk` script contains a `BEGIN` section with conditional logic and two blocks of code. The first block of code is for rows that start with a "#" symbol, which indicates that the row contains either city/province pairs (Canada) or city/state pairs (USA). An example of such a row is here:

```
#usa,chicago,illinois,denver,colorado,seattle,washington,vancouver,washi
ngton
```

Note that the use of a "#" symbol is simply a convenient way to differentiate these rows from the rows in the second code block. One alternative is to specify one CSV file for the rows that are processed in this code block and a different CSV file for the rows that are processed in the second code block.

As you can see in Listing 3.28, the first field $1 of such rows contains the abbreviation of a country, and subsequent pairs contain the location of a city in the associated country. For example, $2 and $3 consist of a city/province pair or a city/state pair; similarly for $4 and $5, for $6 and $7, and so forth.

The second block of code processes rows whose countries do not have a province or state designation for each city. An example of such a row is here:

```
italy,firenze,milano,roma,venezia
```

The code in this block processes each column sequentially and displays all of them on separate output lines. Now launch the code in Listing 3.28 and you will see the following output:

```
=> CITIES in italy:
firenze
milano
roma
venezia
------------

=> CITIES in france:
antibe
nice
paris
st_jeannet
------------

=> CITIES in germany:
```

```
berlin
frankfurt
------------

=> CITIES in spain:
barcelona
madrid
------------

=> CITIES in england:
liverpool
london
manchester
------------

=> CITIES in mexico:
mexico_city
tijuana
------------

=> CITIES in usa:
chicago,illinois
denver,colorado
seattle,washington
vancouver,washington
------------

=> CITIES in canada:
vancouver,bc
edmonton,calgary
hamilton,ontario
------------
```

At this point let's summarize the files that we have examined thus far and what we have accomplished:

- `country_codes.csv`
- `add_country_codes.sh`
- `countries_cities.csv`
- `split_country_cities.sh`
- `countries_cities2.csv`
- `split_country_cities2.sh`

We have a `csv` file whose rows are country code + country name combinations. We have `csv` files with countries and cities that belong to those countries, one of which contains a list of cities for each country. The other `csv` file contains three types of rows:

- country and list of cities
- country and a list of city/province pairs
- country and a list of city/state pairs

## DATA CLEANING ON A KAGGLE DATASET

If you want to see data cleaning performed on an online dataset, you're in luck because this section contains an `awk` script that performs data cleaning on the following dataset:

*https://www.kaggle.com/fehmifratpolat/marketing-report*

The `awk` script follows most of the data cleaning steps that are performed in `R` that are discussed in this article:

*https://towardsdatascience.com/cleaning-and-preparing-marketing-data-in-r-prior-to-machine-learning-or-analysis-ec1a12079f1*

Listing 3.29 displays the contents of `convert_marketing.sh` that shows you how to perform various data cleaning steps, as indicated in the comments in the code.

***LISTING 3.29: convert_marketing.sh***

```
# Kaggle dataset: https://www.kaggle.com/fehmifratpolat/marketing-report

# step 1: extract the 1st, 3rd, 4th, and 7th fields:
cat mark1.csv |awk -F";" '{print $1 ":" $3 ":" $4 ":" $7}' >mark2.csv

# step 2: replace Not tracked" with "direct":
# peri-Co-od;salesChannel;platformcode;marketingInvestment
# date;channel;platformcode;;spend
cat mark2.csv |awk -F":" '
{
   $3 = tolower($3)

   if( $3 ~ /Not tracked/ ) {
      $3 = "direct"
   } else if( $3 ~ /unpaid/ ) {
      $3 = "organic"
   } else if( $3 ~ /Silverpop/ ) {
      $3 = "email"
   }

   if( $4 ~ /,/ ) {
      split($4,arr1,",")
      $4 = arr1[1] "." arr1[2]
   }
   print $1 ":" $2 ":" $3 ":" $4
}
' >mark3.csv

# step 3: replace YYYYDDMM with YYYY-MM-YY format:
cat mark3.csv |awk -F":" '
{
   year  = substr($1,1,4)
   day   = substr($1,5,2)
   month = substr($1,7,2)

   printf("%s-%s-%s:%s:%s:%s\n",year,month,day,$2,$3,$4)
}
```

```
' >mark4.csv

# step 4: calculate subtotals based on $1+$2+$3:
cat mark4.csv |awk -F":" '
{
   fields=$1FS$2FS$3;subtotals[fields] += $4
}
# step 5: display values of subtotals:
END{
   total = 0
   for (i in subtotals) {
     printf("Subtotal for %44s => %-8d\n", i, subtotals[i])
     total += subtotals[i]
   }
   printf("TOTAL REVENUE: => %44d\n",total)
}
' >mark5.csv
```

Although it's not necessary to split the code into five awk scripts, it's easier to follow the code, and besides, you will also have access to the intermediate files that you can inspect to see their contents.

Listing 3.29 starts with step 1 that extracts the 1st, 3rd, 4th, and 7th fields from the CSV file mark1.csv and redirects the output to the CSV file mark2.csv, as shown here:

```
cat mark1.csv |awk -F";" '{print $1 ":" $3 ":" $4 ":" $7}' >mark2.csv
```

Step 2 in Listing 3.29 replaces occurrences of the string "Not tracked" with "direct", the string "unpaid" with "organic", and the string "Silverpop" with email in $3. In addition, the "," that appears in $4 is replaced by a ".", after which a new string is printed that contains the modifications to $3 and $4. Each output line is redirected to the CSV file mark3.csv.

Step 3 in Listing 3.29 replaces `YYYYDDMM` dates with a `YYYY-MM-YY` format, as shown here, after which the output is redirected to the CSV file mark4.csv:

```
year  = substr($1,1,4)
day   = substr($1,5,2)
month = substr($1,7,2)
printf("%s-%s-%s:%s:%s:%s\n",year,month,day,$2,$3,$4)
```

Step 4 in Listing 3.29 is probably the most interesting code snippet in this code sample. The concatenation of the first three fields, separated by the default field separator, form an index for the array subtotals. Each time this combination appears in mark4.csv, the corresponding entry in the subtotals array is incremented with the value in $4, as shown here:

```
fields=$1FS$2FS$3;subtotals[fields] += $4
```

After the body of this awk script has completed execution, an END block (which is labeled step 5) displays the values in the `subtotals` array, along with the grand total, as shown here:

```
total = 0
for (i in subtotals) {
  printf("Subtotal for %44s => %-8d\n", i, subtotals[i])
  total += subtotals[i]
}
```

After the preceding code has completed execution, the output is redirected to the CSV file mark5.csv.

Now launch the code in Listing 3.29 that generates a set of intermediate `CSV` files and a final `CSV` file named mark5.csv. If you want to see the first five lines from each of the intermediate files `mark1.csv, ..., mark5.csv` that are generated in Listing 3.29, execute a shell script that contains the following code:

```
for f in `ls mark*csv`
do
  echo "file: $f"
  head -5 $f
done
```

The output of the preceding code block is shown here:

```
file: mark1.csv
periodCode;reportGranularity;salesChannel;platformCode;channelCode;
tagCodes;marketingInvestment;impressions;clicks;visits;conversions;
deliveries;currencyCode;appliedAttributionModel;periodStartDate
20200102;Daily;online;Not tracked;notset;;0;0;0;0;16;14;CZK;lastTou
ch;2020-01-02T00:00:00.0000000
20200103;Daily;online;Not tracked;notset;;0;0;0;0;13;13;CZK;lastTou
ch;2020-01-03T00:00:00.0000000
20200104;Daily;online;Not tracked;notset;;0;0;0;0;6;6;CZK;lastTou
ch;2020-01-04T00:00:00.0000000
20200105;Daily;online;Not tracked;notset;;0;0;0;0;1;1;CZK;lastTou
ch;2020-01-05T00:00:00.0000000

file: mark2.csv
periodCode:salesChannel:platformCode:marketingInvestment
20200102:online:Not tracked:0
20200103:online:Not tracked:0
20200104:online:Not tracked:0
20200105:online:Not tracked:0

file: mark3.csv
periodCode:salesChannel:platformcode:marketingInvestment
20200102:online:not tracked:0
20200103:online:not tracked:0
20200104:online:not tracked:0
20200105:online:not tracked:0

file: mark4.csv
peri-Co-od:salesChannel:platformcode:marketingInvestment
2020-02-01:online:not tracked:0
2020-03-01:online:not tracked:0
2020-04-01:online:not tracked:0
2020-05-01:online:not tracked:0

file: mark5.csv
Subtotal for                    2020-12-02:online:organic => 0
Subtotal for                2020-31-01:online:not tracked => 0
Subtotal for 2020-10-01:online:facebookbusinessadsmanager => 130
```

```
Subtotal for                        2020-06-01:online:sklik => 914
Subtotal for                      2020-24-03:online:rtbhouse => 0
TOTAL REVENUE: =>                                     1227857
```

## SUMMARY

This chapter started with a brief description of data cleaning, followed by examples of replacing `NULL` values with numeric values, replacing multiple values with a single value, and converting strings to date values.

Next, you saw how to use the `sed` command line utility to replace multiple delimiters in a `CSV` file with a single delimiter. You also saw how to use the `awk` command line utility to restructure a `CSV` file to create a file whose rows have the same number of fields.

In addition, you saw how to perform various tasks in `SQL`, and then how to perform these tasks in a `Pandas` data frame.

CHAPTER 4

# DATA WRANGLING

This chapter is primarily about data wrangling, along with code samples that leverage technical concepts that you learned in previous chapters in this book.

The first part of this chapter briefly discusses the term "data wrangling," which means different things to different people, so it's worthwhile establishing some context for this term. Although the terminology regarding data wrangling is specific to this book and not necessarily standard terminology, the distinction can be helpful when you are involved in discussions that necessitate finer-grained terminology.

The second part of this chapter contains examples of data cleaning with `CSV` files that contain fields that are split on two separate lines. There are several such variants that you will see, whose solutions involve `Pandas` or the `csv` command line utility. In this section you will learn how to convert an ill-formatted `CSV` file (containing a *quoted* field that is split on two lines) into a proper `CSV` file. However, this code sample is marked as optional because the solution involves a rather lengthy `awk`-based shell script.

The third part of this chapter introduces a project that involves creating a `CSV` file that contains an event schedule for musical bands and their future concert performances. This project emulates a scenario that consists of multiple files in various formats, such as `XML` documents, text data, `CSV` files, and data in a `MySQL` database table. The goal is to transform the files and combine them in order to create a single `CSV` file.

## WHAT IS DATA WRANGLING?

*Data wrangling* means different things to different people. Some online articles suggest that data wrangling involves a set of steps, and other articles suggest that data wrangling involves converting datasets from one format to a different format. The following link describes data wrangling as a six-step process:

*https://en.wikipedia.org/wiki/Data_wrangling*

In addition to the steps outlined in the preceding link, data wrangling can also involve the following tasks (performed in code samples in Chapter 7):

- transforming datasets from one format into another format (convert)
- creating new datasets from subsets of columns in existing datasets (extract)

As you can see, the preceding steps differentiate between converting data to a different format versus extracting data from multiple datasets to create new datasets. The conversion process can be considered a data cleaning task if only the first step is performed; that is, there is no extraction step.

In the previous chapter you learned about data cleaning, along with a collection of Python-based code samples and Unix shell scripts to perform data cleaning tasks. By contrast, data wrangling involves multiple steps that can include transforming one or more files. The following subsection provides a more detailed distinction between data cleaning and data transformation.

### Data Transformation: What Does This Mean?

In general, data cleaning involves a single data source (not necessarily in a CSV format), with some type of modification to the content of the data source (e.g., filling missing values, changing date formats, etc.), without creating a second data source.

For example, suppose that the data source is a MySQL table called employees that contains employee-related information. After data cleaning tasks on the employees table are completed, the result will still be named the employees table. In database terminology, data cleaning is somewhat analogous to executing a SQL statement that involves a SELECT on a single table.

On the other hand, if two CSV files contain different date formats and you need to create a single CSV file that is based on the date columns, then there will be some type of conversion process that could be one of the following:

- convert the first date format to the second date format, or
- convert the second date format to the first date format, or
- convert both date formats to a third date format

In the case of financial data, you are likely to also encounter different currencies, which involves a conversion rate between a pair of currencies. Since currency conversion rates fluctuate, you need to decide the exchange rate to use for the data, which can be:

- the exchange rate during the date that the CSV files were generated
- the current currency exchange rate
- some other mechanism

In addition, you might also need to convert the CSV files to XML documents, where the latter might be required to conform to an XML Schema, and perhaps also conform to XBRL, which is a requirement for business reporting purposes:

*https://en.wikipedia.org/wiki/XBRL*

As mentioned earlier, data transformation can involve two or more data sources in order to create yet another data source whose attributes are in the required format. Here are four scenarios of data transformation with just two data sources A and B, where data from A and from B are combined in order to create data source C, where A, B, and C can have different file formats:

- all attributes in A and all attributes in B
- all attributes in A and a subset of attributes in B
- a subset of the attributes in A and all attributes in B
- a subset of the attributes in A and a subset of attributes in B

In database terminology, data transformation is somewhat analogous to executing a SQL statement that involves a SELECT on two or more database tables with a JOIN clause. Such SQL statements typically involve a subset of columns from each database table, which would correspond to selecting a subset of the features in the data sources.

Keep in mind there is also the scenario involving the *concatenation* of two or more data sources. If all data sources have the same attributes, then their concatenation is straightforward, but you might also need to check for duplicate values. For example, if you want to load multiple CSV files into a database table that does not allow duplicates, then one solution involves concatenating the CSV files from the command line and then excluding the duplicate rows.

The preceding concatenation task can be easily performed. Suppose that you have the following set of CSV files (all of which have the same attributes) emp1.csv, emp2.csv, . . . , emp100.csv. Concatenate these files and exclude duplicate rows by creating a shell script called concat.sh with the following contents:

```
for f in `ls emp*csv`
do
  cat $f
done
```

Now make sure that the shell script is executable:

```
chmod +x concat.sh
```

Now launch the shell script concat.sh that redirects its output to the sort command and then redirects its output to the uniq command, with the final output redirected to the CSV file all_emps.csv:

```
./concat.sh | sort | uniq > all_emps.csv
```

At this point you *might* be able to load the CSV file all_emps.csv into a MySQL table, provided that that table is not a "child" table of another table, with a CONSTRAINT condition on the rows of the child table. For example, a customers table typically has a one-to-many relationship with a purchase_orders table, which means that you cannot insert rows in the latter table if they do not have an associated customer row in the customers table.

When in doubt, try loading the CSV file into an appropriate table: if you encounter any errors, make the necessary corrections until the loading process is successful.

This concludes the discussion regarding the first section for data wrangling, and the next several sections contain additional data cleaning code samples, after which you will see data wrangling code samples.

## CSV FILES WITH MULTI-ROW RECORDS

This section shows you how to work with CSV files that contain records that are split across two lines.

Listing 4.1 displays the contents of `multi_line.csv` that will be reformatted so that each record occupies a single line: one solution involves `Pandas` and another solution involves the `CSV` library.

***LISTING 4.1: multi_line.csv***

```
id,fname,lname
1,donna,rhymes?
2,dave,"jones$"
3,john,van jones.#
4,stevie,"ray
vaughn#"
```

## Pandas Solution (1)

Listing 4.2 displays the contents of `pandas_multi_line.py` that reformats the contents of `multi_line.csv` so that each record occupies a single row.

***LISTING 4.2: pandas_multi_line.py***

```
import sys

import pandas as pd

filename = 'multi_line.csv'
data = pd.read_csv(filename)
print("data:")
print(data)
```

Listing 4.2 starts with an `import` statement and then initializes the variable `filename` with the name of the `CSV` file for this code sample. The next snippet populates the `Pandas` data frame `data` and then displays its contents. Launch the code in Listing 4.2 and you will see the following output:

```
data:
   id   fname          lname
0   1   donna        rhymes?
1   2    dave         jones$
2   3    john   van jones.#
3   4  stevie  ray\nvaughn#
4   5   ralph       emerson#
```

As you can see, the preceding output contains multiple metacharacters. If need be, you can create a one-line sed command to replace the metacharacters with one metacharacter of your choice: check Chapter 6 for a similar task that involves the sed command.

Notice that the output from Listing 4.2 is not a true `CSV` file. However, we can generate a CSV file, as shown in the next section.

## Pandas Solution (2)

Listing 4.3 displays the contents of `pandas_multi_line2.py` that reformats the contents of `multi_line.csv` so that each record occupies a single row and saves the data as a `CSV` file.

***LISTING 4.3: pandas_multi_line2.py***

```
import sys

import pandas as pd

filename = 'multi_line.csv'
data = pd.read_csv(filename)

# remove the "\n" character from the data:
columns=["id","fname","lname"]
data2 = data[columns].replace('\\n',' ', regex=True)

# save to a CSV file:
data2.to_csv("multi_line2.csv",index=False)
```

Listing 4.3 starts with an `import` statement and then initializes the variable `filename` with the name of the `CSV` file for this code sample. The next snippet populates the `Pandas` data frame `data` and then initializes the `Pandas` data frame `data2` with the contents of `data` without the line feed characters. The final code snippet saves `data2` to the `CSV` file `multi_line2.csv`. Launch the code in Listing 4.3 and you will see a new file `multi_line2.csv` whose contents are displayed here:

```
id,fname,lname
1,donna,rhymes?
2,dave,jones$
3,john,van jones.#
4,stevie,ray vaughn#
5,ralph,emerson#
```

As you can see, the preceding output contains multiple meta characters that can be replaced with the same column delimiter using the same technique in the `sed`-based `delimiters1.sh` shell script.

## CSV Solution

In case you have not already done so, install the `CSV Python` library with this command:

```
pip3 install csv
```

Listing 4.4 displays the contents of `csv_multi_line.py` that reformats the contents of `multi_line.csv` so that each record occupies a single row.

***LISTING 4.4: csv_multi_line.py***

```
import sys


import csv

filename = 'multi_line.csv'

with open(filename, newline='', encoding='utf-8') as f:
  data = csv.reader(f, delimiter=',', quotechar='"',
                    quoting=csv.QUOTE_MINIMAL)
  for line in data:
    print("line:",line)
```

Listing 4.4 starts with an `import` statement and then initializes the variable `filename` with the name of the `CSV` file for this code sample. The next snippet opens the `CSV` file and initializes the `Pandas` data frame `data` with its contents. The final code block contains a loop that iterates through the rows of the data frame and prints their contents. Launch the code in Listing 4.4 and you will see the following output:

```
line: ['id', 'fname', 'lname']
line: ['1', 'donna', 'rhymes?']
line: ['2', 'dave', 'jones$']
line: ['3', 'john', 'van jones.#']
line: ['4', 'stevie', 'ray\nvaughn#']
line: ['5', 'ralph', 'emerson#']
```

## CSV FILES, MULTI-ROW RECORDS, AND THE AWK COMMAND

This section shows you how to work with `CSV` files that contain records that are defined in multiple lines and records that are separated by a blank line.

Listing 4.5 displays the contents of `multi_line.csv` that will be reformatted so that each record occupies a single line: one solution involves `Pandas` and another solution involves the `CSV` library.

***LISTING 4.5: multi_line_records.csv***

```
nancy jones
marketing dept
san francisco

dave smith
sales dept
chicago

steve anderson
marketing dept
seattle
```

Listing 4.6 displays the contents of `multi_line_records.sh` that reformats the contents of `multi_line.csv` so that each record occupies a single row.

***LISTING 4.6: multi_line_records.sh***

```
# Records are separated by blank lines.
input_file="multi_line_records.csv"

awk '
BEGIN { RS = "" ; FS = "\n" }
{
  printf("%s,%s,%s\n", $1, $2, $3)
}
' < $input_file
```

Listing 4.6 starts by initializing the variable `input_file` with the name of the `CSV` file with this code sample. The next portion of Listing 4.6 contains an `awk` script that prints the contents of each input line as a comma-separated line, which can be redirected to a `CSV` file. Launch the code in Listing 4.6 and you will see the following output:

```
nancy jones,marketing dept,san francisco
dave smith,sales dept,chicago
steve anderson,marketing dept,seattle
```

## QUOTED FIELDS SPLIT ON TWO LINES (OPTIONAL)

This section is marked optional primarily because it involves a custom `CSV` file that you might not encounter in your daily routine. Specifically, the `CSV` file contains multiple rows with a single field as well as a quoted field that is split on two lines. Moreover, the shell script in this section is an `awk`-based solution that is also the most complex `awk` example in this book. If you are new to the `awk` command, please read at least a portion of the appendix that contains `awk`-based code samples, which will facilitate your understanding of the code in this section.

The `awk` utility is a very powerful utility for processing text files through intricate and precise conditional logic. In particular, you can easily extract substrings of fields, perform compound conditional logic that involves adjacent fields, and also remove line feed characters from the rows in text files.

By now you probably realize that the `awk` utility is extremely useful for data cleaning as well as wrangling that involves transforming `CSV` files (or general text files) with irregular structures into formats that are then easily handled through `Pandas`. Consider the ease with which `awk` calculates subtotals, in the final example in Chapter 6, with a *single* line of code. For some people the `awk` is a "go to" command line utility that performs preprocessing steps on practically any type of text file or `CSV` file.

One point to keep in mind is that portions of Listing 4.7 are quite intricate and even nonintuitive, which tends to occur when you need a character-by-character analysis as well as a context dependency that involves maintaining and checking "state" information about previous characters as well as the current character, both of which occur in this code sample.

Listing 4.7 displays the contents of `multi_line3.csv` and Listing 4.8 displays the contents of `split_quotes.sh` that reformats the contents of `multi_line3.csv` so that each record contains exactly three fields.

***LISTING 4.7: multi_line3.csv***

```
id,fname,lname
1,
donna,
rhymes?
2,dave,"jones$"
3,john,van jones.#
4,stevie,"ray
vaughn#"
5,ralph,emerson#
```

***LISTING 4.8: split_quotes.sh***

```
# Records are separated by blank lines
# PART 1: create a single string
interfile1="temp_single_line1.csv"
interfile2="temp_single_line2.csv"
interfile3="temp_single_line3.csv"
csv_file="multi_line3.csv"

cat $csv_file | awk -F"," '
BEGIN { count = 1; colCount = 3 }
{
  printf("%s", $0)
  if( $0 !~ /,$/ ) {
     printf(",")
  } else {
    if( $0 !~ /^[0-9]/ ) {
      if( count % colCount == 0) {
        printf(",")
      }
      count += 1
    }
  }
}' > $interfile1

# id,fname,lname,1,donna,rhymes?,2,dave,"jones$",3,john,van jones.#,4,
stevie,"ray,vaughn#",5,ralph,emerson#,
# PART 2: replace patterns like "abc,def" with "abcZdef"
cat $interfile1 | awk -F"," '
{
  for(i=1; i<=NF; i++) {
    if( $i ~ /^"/ && $(i+1) ~ /"$/ ) {
       printf("%sZ",$i)
    } else {
       printf("%s,",$i)
    }
  }
```

```
}
' > $interfile2

# PART 3: ensure that each line contains three fields
cat $interfile2 | awk -F',' '
BEGIN { start_quote = 0; end_quote = 0; quote = 0; colCount = 3 }
{
  for(i=1; i<=NF; i++) {
     if($i ~ /^"/ && $i ~ /"$/ ) {
       printf("%s",$i)
     }
     else if($i !~ /^"/ && $i !~ /"$/ ) {
       printf("%s,", $i)
     }
     else {
        if($i ~ /^"/ ) {
          printf("%s,",$i)
          quote = 1
          start_quote = 1
        } else {
          if($i ~ /"$/ ) {
            printf("%s",$i)
            quote = 1
            end_quote = 1
          }
        }
     }

     if( quote == 0) {
       if(i % colCount == 0) {
         print ""
       }
     } else {
       if(start_quote == 1 && end_quote == 1) {
         quote = 0
         start_quote = 0
         end_quote = 0
       }
     }
  }
}
' > $interfile3

cat $interfile3 | sed -e 's/Z/,/' -e '/^,,/d' -e 's/,$//'
```

Listing 4.8 initializes three variables with the names of three `CSV` files of the form `temp_single_linex.csv`, where x is replaced with 1, 2, and 3, as well as initializing the variable `csv_file` with the `CSV` file `multi_line3.csv`.

Next, Listing 4.8 contains ("PART 1") an `awk` script with a `BEGIN` block that prints each input line, and then performs the following conditional logic. If the input line does *not* terminate with a "," then a "," is printed. Otherwise, if the input line does *not* start with a digit and the value of the count is a multiple of 3, then a "," is printed as shown here:

```
if( $0 !~ /^[0-9]/ ) {
  if( count % colCount == 0) {
    printf(",")
  }
  count += 1
}
```

As you can see, the preceding code block is reminiscent of the logic in the shell script `delimiter1.sh` in chapter 6.

The next portion of Listing 4.8 ("PART 2") is another awk script whose purpose is to replace a nested "," with the letter `Z`. The choice of the letter `Z` is arbitrary, and you can replace it with another character or string that does not appear in the `CSV` file.

The preceding logic detects a nested "," by setting the variables `start_quote` and `end_quote` to the value 1 whenever a field contains a start quote and end quote, respectively. The code for making the preceding determination involves checking the contents of the current field with the contents of the *next* field, as shown here:

```
if( $i ~ /^"/ && $(i+1) ~ /"$/ ) {
```

As you can see, the preceding code snippet checks if field `i` *starts* with a quote (") and also if field `(i+1)` *ends* with a quote (), in which case the letter `Z` is printed.

The next portion of Listing 4.8 ("PART 3") is an `awk` script whose purpose is to print output lines that contain three fields. Fields that contain no quotes or fields that contain balanced quotes are printed as-is. Other fields are checked for start quotes and end quotes. Whenever the loop variable ii is a multiple of 3, a line feed is printed to start the next output line on a separate line. Additional conditional logic determines when to reset the variables `start_quote` and `end_quote` to 0.

The last portion of Listing 4.8 replaces the character `Z` with a comma (","), removes initial occurrences of ",," in each row, and removes any trailing "," character as shown here:

```
cat $interfile3 | sed -e 's/Z/,/' -e '/^,,/d' -e 's/,$//'
```

Launch the code in Listing 4.8 and you will the following output:

```
id,fname,lname
1,donna,rhymes?
2,dave,"jones$"
3,john,van jones.#
4,stevie,"ray,vaughn#"
5,ralph,emerson#
```

## OVERVIEW OF THE EVENTS PROJECT

As you saw in the introduction for this chapter, this section contains a project to generate a `CSV` file that contains the details of the concerts (including the dates and locations) pertaining to various music bands that are scheduled to perform in different countries.

Although this project might appear to be straightforward, in actuality there are some tedious details, such as which programming language (Unix shell programming, `Python`, `Java`, or perhaps other languages) to adopt for transforming each data source into a `CSV` file.

## Why This Project?

Before delving into the details, you might be pondering this legitimate question: why create a project that involves significant complexity? The intent is to simulate a scenario in which you will be required to work with text files that are in different formats, which might be file formats that are unfamiliar to you, and perhaps even file formats that you wish could be avoided (yes, it does happen).

From another perspective, this project is *intended* to take you out of your comfort zone, for the purpose of enhancing your technical skills. As such, the nature of the project is of secondary importance: if you prefer, you could easily replace music bands with soccer clubs from countries throughout the world, or chess tournaments involving chess players from various countries (you get the idea).

Returning to the project: the transformation from the source files to `CSV` files is performed via a mixture of shell scripts, `Python` scripts, and `Java` code. As you learn how to convert each source file into a `CSV`-based file using different technologies, you can assess their suitability for future tasks. For example, the `Python` code and the shell script code that solve the same task have approximately the same length, but you need to take into account the readability and maintainability of the code.

By way of comparison, the `Python` code for converting an `XML` document to a `CSV` file is much shorter than the corresponding `Java` code, which is one of several factors involved in deciding which technology to use for converting the various files to a common `CSV` format.

Indeed, there are multiple solutions that are possible for this project, and the "best" implementation is the one that works best for your requirements. Perhaps the central idea of this project is to illustrate a key aspect of data wrangling: how to transform data sources in order to create a new data source that is derived from the initial set of data sources.

For your convenience, this book contains an appendix chapter that discusses the `sed` command line utility, and another appendix that discusses the `awk` command line utility, both of which contain short code samples that illustrate how to leverage the functionality of `sed` and `awk`.

## Project Tasks

This project requires several `CSV` files to maintain information such as countries, cities, venues, and dates for concert events. Some of the tasks are listed as follows:

- Step 1: Populate CSV file with countries and international codes
- Step 2: Populate CSV file with list of cities per country
- Step 3: Generate city codes based on country codes
- Step 4: Generate SQL file to populate city_code table in MySQL
- Step 5: Convert band-related information to a CSV file
- Step 6: Generate COE (calendar of events) as a CSV file

The code samples that perform each of the preceding tasks are listed here:

- Step 1: manual process
- Step 2: construct_city_codes.py (Python)

- Step 3: city_codes_sql.sh (awk script)
- Step 4: BandsToCSV.java (Java program)
- Step 5: generate_coe.py (Python)
- Step 6: generate_project.sh (shell script)

In some cases there is more than one code sample to perform a given task. For example, step 4 can also be performed with the `Python` file `bands_to_csv.py`.

The following are sample data or code fragment generated in each of the preceding steps, as well as the steps that are performed manually:

- `001,usa`
- `039,100,firenze`
- `france,antibe,nice,paris,st_jeannet`
- `INSERT INTO city_codes VALUES ( 039,100,firenze );`
- `100,The Data Wranglers,Dave Pallai`

Just to be sure it's clear: most of the steps in this project must be performed in the order that they are shown in the preceding list. The exception is step 4, which can be at any point after step 3.

## Generate Country Codes

Among the various ways to generate a unique id for a country, a simple way to do so involves identifying each country by its international prefix for telephone numbers, which will be unique. In this project, `country_codes.csv` is created manually by performing an online lookup for links that contain this information. For example, Listing 4.9 displays the contents of `country_codes.csv` with the country names and their corresponding international codes for this project.

***LISTING 4.9: country_codes.csv***

```
001,usa
033,france
034,spain
039,italy
044,uk
049,germany
052,mexico
081,japan
```

## Prepare List of Cities in Countries

Listing 4.10 displays the contents of `countries_cities.csv` with country names and a list of cities that belong to those countries.

***LISTING 4.10: countries_cities.csv***

```
italy,firenze,milano,roma,venezia
france,antibe,nice,paris,st_jeannet
germany,berlin,frankfurt
spain,barcelona,madrid
uk,liverpool,london,manchester
mexico,mexico_city,tijuana
```

## GENERATING CITY CODES FROM COUNTRY CODES: AWK

Listing 4.11 displays the contents of `construct_city_codes.sh` that illustrates how to generate a unique code for every city that belongs to a given country.

***LISTING 4.11: construct_city_codes.sh***

```
FILE1="country_codes.csv"
FILE2="countries_cities.csv"
rm -f city_codes.csv

awk -F"," '
BEGIN {
   FS=","; OUT_FILE="city_codes.csv";
   prev_name = ""; city_name = "";
   start_code=100; curr_code = 100;
   city_code = ""; incr_code = 100

   printf("=> BEGIN-PROCESSING: %s\n",ARGV[1])
   while(getline line < ARGV[1]){
     #format: country-code,country-name
     split(line,data)
     code = data[1]
     country = data[2]
     #printf("code: %s country: %s\n",code,country)
     country_code[country] = code
     code_country[code] = country
   }
   close(ARGV[1])
}
########################################
# read each line from countries_cities.csv
# split each line into a list of cities
# generate a city code that contains:
# country_code, curr_code, city_name
# sample line in countries_cities.csv:
# italy,firenze,milano,roma,venezia
########################################
{
   #sample: italy,firenze,milano,roma,venezia
   split($0,data)
   code    = data[1]
   country = data[2]

   #printf(" code = %s country = %s\n", code, country)
   #printf("=> CHECK CODE FOR: x %s x\n", country_code[code])

   if(country_code[country] == "") {
      printf("=> MAIN-CITIES in %s:\n",$0)
      curr_code = start_code
      for(i=2; i<=NF; i++) {
         city_code = country_code[$1] "_" curr_code "_" $i
         printf("=> constructed city_code:%s\n",city_code)
         print city_code >> OUT_FILE
```

```
         curr_code += incr_code
      }

      if(curr_name == "") {
        curr_name = $1
      } else {
        if(curr_name != $1) {
          # processing a different country:
          prev_name = curr_name
          curr_name = $1
          curr_code = start_code
        }
      }

      if(city_code == 0) city_code = start_code
   } else {
      #printf("=> SKIPPING INVALID LINE: %s\n",$0)
   }
}
' $FILE1 $FILE2
```

Listing 4.11 initializes the variables `IN_FILE1` and `IN_FILE2` with the names of two `CSV` files `country_codes.csv` and `countries_cities.csv`, followed by an awk script that initializes several string variables and numeric variables that are explained later.

Next, Listing 4.11 contains an `awk` script with a `BEGIN` block that contains a loop that reads the contents of the file `country_codes.csv` whose rows contain a country name and the associated three-digit code for the country. These pairs of values are used to populate the dictionaries `country_code` and `code_country`. The keys for the `country_code` dictionary are the names of the countries, and the values are the three-code numeric codes, whereas the keys and values for the `code_country` dictionary are reversed.

The next block of code is the main execution block of the `awk` script, which involves conditional logic and a large block of code. The reason for the conditional logic is that `awk` will process both input files that are specified at the end of the script regardless of whether or not they are processed in the `BEGIN` block:

```
}
' $IN_FILE1 $IN_FILE2
```

Due to the nature of `awk`, Listing 4.11 processes `IN_FILE1` *twice*: once in the `BEGIN` block and again in the main execution block (is this a quirk of `awk` or just a normal feature?).

Hence, the code in the main block must *skip* the rows in `IN_FILE1` and process *only* the rows in `IN_FILE2`. The way to skip `IN_FILE1` is very simple: the rows in `IN_FILE1` are of the form `country_code,country_abbreviation` (such as `001,usa`), and the code `001` does not have an entry in the `country_code` dictionary. Consequently, only the rows in the `CSV` file `IN_FILE2` satisfy this condition:

```
if(country_code[country] == "")
```

The code inside this `if` statement contains a loop that constructs a city code for each city that is listed in each row of `IN_FILE2`. The key idea is that a city code for each city is constructed by concatenating its country code, a generated number that's based on the value of `curr_code`, and

the name of the city itself. For example, the following line specifies Italy (whose country code is 039) as well as five cities:

```
italy,firenze,milano,roma,venezia
```

Consequently, the city code for each of the cities in Italy in the preceding row are as follows:

```
039_100_firenze
039_200_milano
039_300_roma
039_400_venezia
```

Now launch the code in Listing 4.11 and you will see the following output:

```
=> BEGIN-PROCESSING: country_codes.csv
=> MAIN-CITIES in italy,firenze,milano,roma,venezia:
=> constructed city_code:039_100_firenze
=> constructed city_code:039_200_milano
=> constructed city_code:039_300_roma
=> constructed city_code:039_400_venezia
=> MAIN-CITIES in france,antibe,nice,paris,st_jeannet:
=> constructed city_code:033_100_antibe
=> constructed city_code:033_200_nice
=> constructed city_code:033_300_paris
=> constructed city_code:033_400_st_jeannet
=> MAIN-CITIES in germany,berlin,frankfurt:
=> constructed city_code:049_100_berlin
=> constructed city_code:049_200_frankfurt
=> MAIN-CITIES in spain,barcelona,madrid:
=> constructed city_code:034_100_barcelona
=> constructed city_code:034_200_madrid
=> MAIN-CITIES in uk,liverpool,london,manchester:
=> constructed city_code:044_100_liverpool
=> constructed city_code:044_200_london
=> constructed city_code:044_300_manchester
=> MAIN-CITIES in mexico,mexico_city,tijuana:
=> constructed city_code:052_100_mexico_city
=> constructed city_code:052_200_tijuana
```

Listing 4.12 displays the contents of `city_codes.csv` that is generated from the code in Listing 4.11.

***LISTING 4.12: city_codes.csv***

```
039_100_firenze
039_200_milano
039_300_roma
039_400_venezia
033_100_antibe
033_200_nice
033_300_paris
033_400_st_jeannet
049_100_berlin
049_200_frankfurt
```

```
034_100_barcelona
034_200_madrid
044_100_liverpool
044_200_london
044_300_manchester
052_100_mexico_city
052_200_tijuana
```

## GENERATING CITY CODES FROM COUNTRY CODES: PYTHON

Listing 4.13 displays the contents of `construct_city_codes2.py` that illustrates how to generate a unique code for every city that belongs to a given country.

***LISTING 4.13: construct_city_codes2.py***

```
import csv

FILE1="country_codes.csv"
FILE2="countries_cities.csv"

OUTFILE1="python_city_codes.txt"
OUTFILE2="python_city_codes.csv"
OUTFILE3="python_city_codes.sql"

prev_name  = ""
city_name  = ""
curr_name  = ""
start_code = 100
curr_code  = 100
city_code1 = ""
city_code2 = ""
incr_code  = 100
code_country = {}
country_code = {}

# STEP 1: read file with code and country values:
print("=> BEGIN-PROCESSING:",FILE1)
with open(FILE1, mode ='r')as file:
  # reading the CSV file
  csvFile = csv.reader(file)

  # displaying the contents of the CSV file
  for line in csvFile:
    code    = line[0]
    country = line[1]
    country_code[country] = code
    code_country[code] = country

# STEP 2: read file countries and cities:
########################################
# read each line from countries_cities.csv
# split each line into a list of cities
```

```
# generate a city code that contains:
# country_code, curr_code, city_name
# sample line in countries_cities.csv:
# italy,firenze,milano,roma,venezia
########################################

print("=> BEGIN-PROCESSING:",FILE2)
#sample: italy,firenze,milano,roma,venezia

with open(FILE2, mode ='r')as file:
  # reading the CSV file
  csvFile = csv.reader(file)

  # open two output files:
  # create txt_writer1:
  txt_writer1 = open(OUTFILE1,'w')

  # create csv_writer2:
  csv_writer2 = open(OUTFILE2,'w')

  # create sql_writer3:
  sql_writer3 = open(OUTFILE3,'w')

  for line in csvFile:
    country = line[0]
    code = country_code[country]
    #print("full line:",line)
    #print(" code = ",code," country = ",country)

    #print("=> MAIN-CITIES in:",line)
    curr_code = start_code
    column_count = len(line)
    for i in range(2,column_count):
      city_code1 = str(code) + "_" +
                   str(curr_code) + "_" + str(line[i])

      city_code2 = str(code) + "," +
                   str(curr_code) + "," + str(line[i])
      print("=> constructed city_code1:",city_code1)

      # generate SQL statement from city code:
      sql_stmt = 'INSERT INTO city_codes VALUES ('
      sql_stmt += str(code)+','+str(curr_code)+',"'+line[i]+'");\n'

      # print SQL statement to SQL file:
      sql_writer3.write(sql_stmt)

      # write city code to CSV file:
      txt_writer1.write(city_code1+"\n")
      #curr_code += incr_code

      # write city code to TXT file:
      csv_writer2.write(city_code2+"\n")
      curr_code += incr_code
```

```
    if(curr_name == None):
      curr_name = line[1]
    else:
      if(curr_name != line[1]):
        # processing a different country:
        prev_name = curr_name
        curr_name = line[1]
        curr_code = start_code

    if(city_code1 == ""): city_code1 = start_code
    if(city_code2 == ""): city_code2 = start_code

# STEP 3: close the city_code file:
#city_code_file.close()
```

Listing 4.13 contains the `Python`-based counterpart to Listing 4.11 that contains an `awk`-based shell script for generating city codes. Since you are probably less familiar with shell programming than `Python`, Listing 4.11 contains a detailed explanation of the code, whereas a detailed explanation is omitted for the `Python` code. Since the flow of logic is identical, the main difference is the language-specific syntax, which you can compare in a line-by-line fashion. If you launch the code in Listing 4.13 you will see essentially the same output as Listing 4.12.

There is one significant difference between Listing 4.13 and Listing 4.12: the code snippets shown in bold in Listing 4.13 generate `SQL` statements for inserting a row into the `city_codes` table, which is not created in this listing (but you could add that code if you want to do so). Each `SQL` statement is appended to the file `python_city_codes.sql`, a portion of which is shown as follows:

```
INSERT INTO city_codes VALUES (039,100,"milano");
INSERT INTO city_codes VALUES (039,200,"roma");
INSERT INTO city_codes VALUES (039,300,"venezia");
INSERT INTO city_codes VALUES (033,100,"nice");
INSERT INTO city_codes VALUES (033,200,"paris");
```

Now that you have seen two solutions for generating city codes, you can use your "preferred" solution as a guide to help you navigate through the other solution.

## GENERATING SQL STATEMENTS FOR THE CITY_CODES TABLE

Listing 4.14 displays the contents of `city_codes_sql.sh` that creates the file `city_codes.sql` that contains `SQL` statements for creating the table `city_codes` with values from the `CSV` file `city_codes.csv`.

***LISTING 4.14: city_codes_sql.sh***

```
OUTFILE="city_codes.sql"
rm -f OUTFILE

echo "USE mytools;" > $OUTFILE
echo "DROP TABLE IF EXISTS city_code;" >> $OUTFILE
echo "CREATE TABLE city_codes (city_code CHAR(30));" >> $OUTFILE
```

```
for line in 'cat city_codes.csv'
do
  echo "INSERT INTO city_codes VALUES ( $line );"      >> $OUTFILE
done
```

Listing 4.15 displays the contents of `city_codes.sql` that is generated by the shell script `city_codes_sql.sh`.

***LISTING 4.15: city_codes.sql***

```
OUTFILE="city_codes.sql"
USE mytools;
DROP TABLE IF EXISTS city_code;

CREATE TABLE city_codes (city_code CHAR(30));

INSERT INTO city_codes VALUES ( 039_100_firenze );
INSERT INTO city_codes VALUES ( 039_200_milano );
INSERT INTO city_codes VALUES ( 039_300_roma );
INSERT INTO city_codes VALUES ( 039_400_venezia );
INSERT INTO city_codes VALUES ( 033_100_antibe );
INSERT INTO city_codes VALUES ( 033_200_nice );
INSERT INTO city_codes VALUES ( 033_300_paris );
INSERT INTO city_codes VALUES ( 033_400_st_jeannet );
INSERT INTO city_codes VALUES ( 049_100_berlin );
INSERT INTO city_codes VALUES ( 049_200_frankfurt );
INSERT INTO city_codes VALUES ( 034_100_barcelona );
INSERT INTO city_codes VALUES ( 034_200_madrid );
INSERT INTO city_codes VALUES ( 044_100_liverpool );
INSERT INTO city_codes VALUES ( 044_200_london );
INSERT INTO city_codes VALUES ( 044_300_manchester );
INSERT INTO city_codes VALUES ( 052_100_mexico_city );
INSERT INTO city_codes VALUES ( 052_200_tijuana );
```

## GENERATING A CSV FILE FOR BAND MEMBERS (JAVA)

Listing 4.16 displays the contents of `bands.xml` and Listing 4.17 displays the contents of `BandsToCSV.java` that illustrates how to convert the `XML` file to a `CSV` file.

***LISTING 4.16: bands.xml***

```
<bands>
  <band>
    <band_id>
      100
    </band_id>
    <band_name>
      The Data Wranglers
    </band_name>
    <band_member>
      Dave Pallai
```

```
    </band_member>
    <band_member>
      Jennifer Blaney
    </band_member>
  </band>
  <band>
    <band_id>
      200
    </band_id>
    <band_name>
      The Data Cleaners
    </band_name>
    <band_member>
      Raymond Reddington
    </band_member>
    <band_member>
      Scary Joe
    </band_member>
    <band_member>
      The Smoother
    </band_member>
  </band>
</bands>
```

***LISTING 4.17: BandsToCSV.java***

```
import csv
import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.Node;
import org.w3c.dom.NodeList;
import org.xml.sax.SAXException;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.InputStream;

public class BandsToCSV
{
   private static final String FILENAME = "bands.xml";

   public static void main(String[] args) {
      String filename = "bandmembers.csv";
      File outfile = null;
      FileWriter csvWriter = null;

      // 1) create a file object:
      outfile = new File(filename);

      // 2) create a FileWriter object:
      try {
```

```
  csvWriter = new FileWriter("bandmembers.csv");
} catch (IOException e) {
  System.out.println("An error occurred:");
  e.printStackTrace();
}

// 3) process the XML file:
DocumentBuilderFactory dbf =
                  DocumentBuilderFactory.newInstance();
try {
   DocumentBuilder db = dbf.newDocumentBuilder();
   Document doc = db.parse(new File(FILENAME));
   doc.getDocumentElement().normalize();
   System.out.println("ROOT: " +
               doc.getDocumentElement().getNodeName());

   NodeList bandList = doc.getElementsByTagName("band");

   for (int idx = 0; idx < bandList.getLength(); idx++) {
      Node node = bandList.item(idx);
      if (node.getNodeType() == Node.ELEMENT_NODE) {
         Element element = (Element) node;

         // 4) the id of the band:
         String band_id = element.getElementsByTagName("band_id")
                                 .item(0).getTextContent();
         band_id = band_id.replaceAll("\\s", "");
         band_id = band_id.replaceAll("\\n", "");
         System.out.println("band_id = "+band_id);

         // 5) the name of the band:
         String band_name =
                   element.getElementsByTagName("band_name")
                          .item(0).getTextContent();

         band_name = band_name.trim();
         band_name = band_name.replaceAll("\\n", "");
         System.out.println("band_name = "+band_name);

         // 6) the band members:
         NodeList memberList =
                    element.getElementsByTagName("band_member");

         for (int idx2=0; idx2<memberList.getLength(); idx2++)
         {
            Node memberNode = memberList.item(idx2);
            if (memberNode.getNodeType() == Node.ELEMENT_NODE)
            {
               Element element2 = (Element) memberNode;
               System.out.println("Current Element: " +
                 memberNode.getNodeName());

               String band_member =
```

```
                        memberList.item(idx2).getTextContent();

                    //band_member = band_member.replaceAll("\\s", "");
                    band_member = band_member.trim();
                    band_member = band_member.replaceAll("\\n", "");
                    System.out.println("band_member = "+band_member);

                    // 7) write a string to the CSV file:
                    csvWriter.write(band_id+","+
                                    band_name+","+
                                    band_member+"\n");
                }
            }
        }
      }
      csvWriter.close();
    } catch (ParserConfigurationException | SAXException | IOException e)
{
      e.printStackTrace();
    }
  }
}
```

Listing 4.17 starts with multiple `import` statements, and then defines the `Java` class `BandsToCSV` that contains a `main()` method that is divided into six sections. The first section creates the file object `outfile` and the second section creates the `FileWriter` object `csvWriter`.

The third section is the longest section and also contains the fourth and fifth sections. The first portion of the third section contains a `try/catch` block in which a `Document` object `doc` is constructed, which is the root node of the `XML` document that contains information about the music bands.

Next, the variable `bandList` is initialized, which represents a list of nodes, and each node in the list is information about one band. The next portion of section 3 is a for list that iterates through the items in the variable `bandList`. If each item is of `ELEMENT_NODE`, then section 4 is executed, which involves extracting the value of the `band_id` from the current node.

At this point section 5 is executed, which extracts the value of the `band_name` in the current node, followed by section 6 that consists of a `for` loop that extracts all the other band-related information from the current node. Note that the loop in section 6 contains conditional logic: if the current element is of type `ELEMENT_NODE`, the contents of the variable `band_member` are trimmed to remove whitespaces.

We have now reached section 7, which is the final section in Listing 4.17, and the location of the following code snippet that writes band-related information in `CSV` format:

```
csvWriter.write(band_id+","+band_name+","+band_member+"\n");
```

Now launch the code in Listing 4.17 and you will see the following output:

```
ROOT: bands
band_id = 100
band_name = The Data Wranglers
```

```
Current Element: band_member
band_member = Dave Pallai
Current Element: band_member
band_member = Jennifer Jones
Current Element: band_member
band_member = Raymond Reddington
band_id = 200
band_name = The Data Cleaners
Current Element: band_member
band_member = Heavy D
Current Element: band_member
band_member = Big Daddy Kane
```

Listing 4.18 displays the contents of the CSV file `bandmembers.csv` that is generated by the Java code in Listing 4.17.

***LISTING 4.18: bandmembers.csv***

```
100,The Data Wranglers,Dave Pallai
100,The Data Wranglers,Jennifer Blaney
100,The Data Wranglers,Raymond Reddington
200,The Data Cleaners,Scary Joe
200,The Data Cleaners,The Smoother
```

## GENERATING A CSV FILE FOR BAND MEMBERS (PYTHON)

Listing 4.19 displays the contents of `BandsToCSV.java` that illustrates how to generate a text file as well as a CSV file from the XML document `bandmembers.csv`.

***LISTING 4.19: bands_to_csv.py***

```
from xml.etree import cElementTree as ET

INFILE1="bands.xml"
OUTFILE1="python_bandmembers.txt"
OUTFILE2="python_bandmembers.csv"

# create txt_writer1:
txt_writer1 = open(OUTFILE1,'w')

# create csv_writer2:
csv_writer2 = open(OUTFILE2,'w')

band_id      = ""
band_name    = ""
band_member  = ""

xml_data = ET.parse(INFILE1).getroot()
for band in xml_data.iter('band'):
  for child in band:
    #print("child: ",child)
```

```
    print("name: ",child.tag)
    name = child.tag

    if(name == "band_id"):
      band_id = child.text
      band_id = band_id.strip()
      print("band_id =     ", band_id)
    elif(name == "band_name"):
      band_name = child.text
      band_name = band_name.strip()
      print("band_name =   ", band_name)
    elif(name == "band_member"):
      band_member = child.text
      band_member = band_member.strip()
      print("band_member = ", band_member)

      #the_member = child.text
      #the_member = the_member.strip()
      #print("the_member = ",the_member)

    if(band_id != "" and band_name != "" and band_member != ""):
      line1 = str(band_id) + "_" + band_name + "_" + band_member
      line2 = str(band_id) + "," + band_name + "," + band_member
      print("=> constructed line1:",line1)
      print("=> constructed line2:",line2)

      # write line to CSV file:
      txt_writer1.write(line1+"\n")

      # write line to TXT file:
      csv_writer2.write(line2+"\n")

      #band_id      = ""
      #band_name    = ""
      band_member  = ""
```

Listing 4.19 starts by initializing some variables for existing files and variables for output files, followed by initializing the variable `xml_data` as the root node of the `bands.xml` file that contains band-related information, as shown here:

```
xml_data = ET.parse(INFILE1).getroot()
```

The next portion of Listing 4.19 is a nested loop that iterates through the band elements, and for each band there is another loop that iterates through the elements of the current band. Next, conditional logic checks for the presence of any `band_id`, `band_name`, and `band_member` nodes, and initializes the values of corresponding variables.

The final `if` statement checks if the three variables are populated with values, and if so, constructs a `CSV`-based output row that is written to the `CSV` file; similarly, another text-based output row is constructed and written to the associated text file. Now launch the code in Listing 4.19 and you will see the following output:

```
name:  band_id
```

```
band_id =      100
name:  band_name
band_name =    The Data Wranglers
name:  band_member
band_member =  Dave Pallai
=> constructed line1: 100_The Data Wranglers_Dave Pallai
=> constructed line2: 100,The Data Wranglers,Dave Pallai
name:  band_member
band_member =  Jennifer Blaney
=> constructed line1: 100_The Data Wranglers_Jennifer Blaney
=> constructed line2: 100,The Data Wranglers,Jennifer Blaney
name:  band_id
band_id =      200
name:  band_name
band_name =    The Data Cleaners
name:  band_member
band_member =  Raymond Reddington
=> constructed line1: 200_The Data Cleaners_Raymond Reddington
=> constructed line2: 200,The Data Cleaners,Raymond Reddington
name:  band_member
band_member =  Scary Joe
=> constructed line1: 200_The Data Cleaners_Scary Joe
=> constructed line2: 200,The Data Cleaners,Scary Joe
name:  band_member
band_member =  The Smoother
=> constructed line1: 200_The Data Cleaners_The Smoother
=> constructed line2: 200,The Data Cleaners,The Smoother
```

Also check the directory that contains Listing 4.19 and you will see two new files:

```
python_bandmembers.txt
python_bandmembers.csv
```

## GENERATING A CALENDAR OF EVENTS (COE)

Listing 4.20 displays the contents of the CSV file `concert_dates.csv` that contains a short calendar of events: additional relevant details are stored in other CSV files that are created in this project.

***LISTING 4.20: concert_dates.csv***

```
100,039,vicenza,01/05/2022
100,039,vicenza,01/05/2022,50
100,033,paris,01/15/2022,80
100,044,london,02/02/2022,70
100,049,berlin,02/09/2022,75
100,081,tokyo,02/19/2022,100
200,039,vicenza,01/08/2022,50
200,033,paris,01/18/2022,70
200,044,london,02/05/2022,60
200,049,berlin,02/13/2022,80
200,081,tokyo,02/27/2022,100
```

Each row in Listing 4.20 contains details that have the following format:

```
band_code,country_code,city,date,price
```

The target CSV file concert_coe.csv will enhance the contents of concert_dates.csv so that each row will contain information with the following format:

```
band_code,band_name,country_code,country_name,city,date,price
```

Notice that the date format is dd/mm/yyyy and the price column is in USD (but not necessarily realistic ticket prices). The only new columns that we need to provide are the band_name and the country_name columns.

Listing 4.21 displays the contents of generate_coe.py that shows how to generate the CSV file concert_coe.csv by enhancing the CSV file concert_dates.csv with the two additional columns.

***LISTING 4.21: generate_coe.py***

```
import pandas as pd
import numpy as np

# 1) read the input file concert_dates.csv
# header line: band_code,country_code,city,date,price
# sample row:  100,039,vicenza,01/05/2022,50
concert_dates_file="concert_dates.csv"

# 2) read python_bandmembers.csv (band names and members)
# sample row:100_The Data Wranglers_Dave Pallai
band_members_file="python_bandmembers.csv"

# 3) read country_codes.csv (country and code)
# sample row: 039,italy
cc_codes_file="country_codes.csv"

# 4) create the output file calendar_coe.csv:
# header line: band_code,band_name,country_code,country_name,
city,date,price
# sample row:  The Data Wranglers,italy,vicenza,01/05/2022,50
outfile1="calendar_coe.csv"

# 5) populate three data frames:
concert1_df = pd.read_csv(concert_dates_file)
band2_df = pd.read_csv(band_members_file)
cc3_df  = pd.read_csv(cc_codes_file)

# 6) add column labels:
#existing row: 100,039,vicenza,01/05/2022,50
concert1_df.columns = ['band_code','country_code','City','Concert
Date','Tickets (USD)']
band2_df.columns = ['band_code','band_name','band_members']
cc3_df.columns   = ['country_code','country_name']
```

```
# 7) extract array of unique band codes and band names:
bands_unique_codes = band2_df.band_code.unique()
bands_unique_names = band2_df.band_name.unique()

# 8) create dictionary from unique band codes and band names:
bands_codes_names_dict = {}
for code, name in zip(bands_unique_codes,bands_unique_names):
  bands_codes_names_dict[code] = name

# 9) insert a column for "Band Name":
concert1_df['Band Name'] = concert1_df['band_code'].map(bands_codes_
names_dict)

# 10) create dictionary from unique country codes and names:
country_code_to_name_dict = {}
for index, row in cc3_df.iterrows():
  country_code   = row['country_code']
  country_name   = row['country_name']
  country_code_to_name_dict[country_code] = country_name

# 11) insert a column for country names:
concert1_df['Country'] = concert1_df['country_code'].map(country_code_
to_name_dict)

# 12) remove band codes and country codes:
del concert1_df['band_code']
del concert1_df['country_code']

print("*** Concert Details ***")
print(concert1_df)
print()

# 13) save concert details to CSV file:
concert1_df.to_csv(outfile1, index=False)
```

The first three sections of Listing 4.21 specify the names of the three input files `concert_dates,csv`, `python_bandmembers.csv`, and `country_codes.csv` that we need in order to generate the full calendar for the concerts.

The next portion specifies `calendar-coe.csv` as the name of the output file, along with the header and sample row in this file.

The next portion of Listing 4.21 populates three data frames with the data from the files listed in the first portion of this code sample. The next three sections add column names to the data frames, which would not be necessary if the input files had a header line.

The next portion of Listing 4.21 extracts the unique band codes and their corresponding band names in order to create a `Python` dictionary with code/name pairs, which we will use later in this code sample.

In a similar fashion, the next portion of Listing 4.21 extracts the unique country codes and their corresponding names in order to create a `Python` dictionary with `country_code/country_name` pairs, which we will also use later in this code sample.

The next code block inserts a column into `concert1_df` with the name of the band that corresponds to the band code, for each row in `concert1_df`. Another code block inserts a column into `concert1_df` with the name of the country that corresponds to the country code, for each row in `concert1_df`.

The next pair of `del` statements drop the `band_code` and `country_code` columns from the `concert1_df` data frame (an optional step).

Finally, the contents of the data frame `concert1_df` are displayed and then the contents of the data frame are saved as a `CSV` file. Now launch the code in Listing 4.21 and you will see the following output:

```
*** Concert Details ***
      City Concert Date  Tickets (USD)           Band Name  Country
0    paris   01/15/2022             80  The Data Wranglers   france
1   london   02/02/2022             70  The Data Wranglers       uk
2   berlin   02/09/2022             75  The Data Wranglers  germany
3    tokyo   02/19/2022            100  The Data Wranglers    japan
4  vicenza   01/08/2022             50   The Data Cleaners    italy
5    paris   01/18/2022             70   The Data Cleaners   france
6   london   02/05/2022             60   The Data Cleaners       uk
7   berlin   02/13/2022             80   The Data Cleaners  germany
8    tokyo   02/27/2022            100   The Data Cleaners    japan
```

## PROJECT AUTOMATION SCRIPT

Listing 4.22 displays the contents of `generate_project.sh` that automates the project creation steps that are discussed earlier in this chapter.

***LISTING 4.22: generate_project.sh***

```
echo "Assigning code to countries..."
#=> manual process

echo "Assigning code to cities..."
python3 construct_city_codes.py
#=> generates city_codes.csv

echo "Generate SQL for city codes..."
./city_codes_sql.sh
#=> generates city_codes.sql

echo "Converting personnel (XLSX) to CSV..."
python3 save_xlsx_to_csv.py
#=> generates personnel.csv

echo "Converting bands_list (XML) to CSV..."
# via Java:
java BandsToCSV
# via Python:
#python3 bands_to_csv.py
#=> generates bandmembers.csv
```

```
echo "Generating CSV calendar of events (COE)..."
python3 generate_coe.py
#=> generates complete calendar

#echo "Generating SQL script for COE..."
#exercise

#echo "Inserting COE into MySQL..."
#exercise
```

Listing 4.22 contains various executable files that were discussed earlier in this chapter, and they are invoked sequentially in order to automate the execution of the required tasks in the correct sequence. Now launch the code in Listing 4.22 and you will see the following output:

```
Assigning code to countries...
Assigning code to cities...
=> BEGIN-PROCESSING: country_codes.csv
=> BEGIN-PROCESSING: countries_cities.csv
=> constructed city_code1: 039_100_milano
=> constructed city_code1: 039_200_roma
=> constructed city_code1: 039_300_venezia
=> constructed city_code1: 033_100_nice
=> constructed city_code1: 033_200_paris
=> constructed city_code1: 033_300_st_jeannet
=> constructed city_code1: 049_100_frankfurt
=> constructed city_code1: 034_100_madrid
=> constructed city_code1: 044_100_london
=> constructed city_code1: 044_200_manchester
=> constructed city_code1: 052_100_tijuana
Generate SQL for city codes...
Contents of df1:
   Unnamed: 0 fname  lname                        Role gender  country
0           0  john  smith                VP Marketing      m      usa
1           1  jane  smith                   VP Sales      f   france
2           2  jack  jones          Event Coordinator      m   france
3           3  dave  stone              Event Planner      m    italy
4           4  sara  stein  Director Public Relations      f  germany
5           5  eddy  bower      Directory of Security      m    spain
Contents of df1 after drop:
  fname  lname                        Role gender  country
0  john  smith                VP Marketing      m      usa
1  jane  smith                   VP Sales      f   france
2  jack  jones          Event Coordinator      m   france
3  dave  stone              Event Planner      m    italy
4  sara  stein  Director Public Relations      f  germany
5  eddy  bower      Directory of Security      m    spain
Saving data frame to CSV file personnel.csv
Converting bands_list (XML) to CSV...
ROOT: bands
band_id = 100
band_name = The Data Wranglers
Current Element: band_member
band_member = Dave Pallai
Current Element: band_member
```

```
band_member = Jennifer Blaney
band_id = 200
band_name = The Data Cleaners
Current Element: band_member
band_member = Raymond Reddington
Current Element: band_member
band_member = Scary Joe
Current Element: band_member
band_member = The Smoother
Generating CSV calendar of events (COE)...
*** Concert Details ***
      City Concert Date  Tickets (USD)           Band Name  Country
0    paris   01/15/2022             80  The Data Wranglers   france
1   london   02/02/2022             70  The Data Wranglers       uk
2   berlin   02/09/2022             75  The Data Wranglers  germany
3    tokyo   02/19/2022            100  The Data Wranglers    japan
4  vicenza   01/08/2022             50   The Data Cleaners    italy
5    paris   01/18/2022             70   The Data Cleaners   france
6   london   02/05/2022             60   The Data Cleaners       uk
7   berlin   02/13/2022             80   The Data Cleaners  germany
8    tokyo   02/27/2022            100   The Data Cleaners    japan
```

## Project Follow-Up Comments

If you read the entire project up to this point in the chapter, consider it a job well done. Of course, there are variations in which you can assemble the files in this project to create the final `CSV` file that contains the calendar of events.

First, a general comment: the high-level description of a project is often much easier to understand than the implementation details: the latter can be very tedious and is also influenced by the technologies that you use to solve the tasks of a project. However, it's important to understand the project goals because they will assist you in maintaining a clear understanding of the purpose of the code that you write: you'll understand what needs to be done and why it needs to be done.

Second, this project attempts to strike a balance between code complexity and readability of the code; that is, avoiding unnecessary complexity. Keep in mind that this project involves (albeit somewhat contrived) a small dataset with virtually no data cleaning requirements. By contrast, real-world projects can involve time consuming and much more tedious data cleaning tasks that will make this project seem almost trivial. Nevertheless, hopefully you can avail yourself of the data cleaning code samples in this chapter as well as the previous chapter for other tasks.

Third, this project showed you how to convert files to different formats using `awk`-based shell scripts, `Python`, `Pandas`, and `Java`. In addition, you learned how to generate new files from existing files using these same technologies. Indeed, performing the transformations with the small datasets has probably given you a sense of the complexity to expect when you work with datasets that are hundreds or thousands of times larger than those in this project.

Fourth, you probably noticed that Listing 4.21 did not use all the `CSV` files that were created earlier in this chapter, such as the `CSV` file `python_city_codes.csv` that is created by the `Python` script `construct_city_codes.py`. The reason is twofold: (a) ensuring that Listing 4.21 is only one page of code, and (b) enabling you to modify the structure of some of the `CSV` files that will necessitate the inclusion of the `CSV` file `python_city_codes.csv` in your modified solution, in case you wish to do so.

Finally, there are various "commented out" `print()` statements in the various files for this project. Those statements can be very helpful for debugging purposes and also for understanding the purpose of the code blocks that contain those statements. However, *too many* `print() I` statements can result in an overload of details, so after you understand a particular block of code, consider commenting out those statements again when you analyze different code blocks. Moreover, you can copy/paste some of the output, as a comment block, into the files that generated the output, near the relevant code block.

## SUMMARY

This chapter started with an introduction to the term "data wrangling" and compared it with the term "data cleaning." Then you saw examples of data cleaning with `CSV` files that contain fields that are split on two separate lines. Next you learned how to convert an ill-formatted `CSV` file (containing a *quoted* field that is split on two lines) into a proper `CSV` file.

In addition, you saw a detailed project for creating a `CSV` file that contains an event schedule for musical bands and their future concert performances. This project emulates a scenario that consists of multiple files in various formats, such as `XML` documents, text data, `CSV` files, and data in a `MySQL` database table.

Finally, you learned how to convert existing files into `CSV`-based files using different tools, such as shell scripts, `Python` code, `Java` code, and `Pandas`-based code.

CHAPTER 5

# FEATURE SELECTION

This chapter explores various aspects of feature selection in machine learning, such as available feature selection techniques and how to decide which feature selection technique to use for a given dataset. You will also learn about algorithms that require scaled data, how to work with imbalanced datasets, and how to apply transformations to data in datasets.

The first part discusses techniques for feature selection, which involves selecting a subset of the features in a given dataset. Note that feature extraction is discussed in the next chapter.

The second part discusses techniques for processing imbalanced datasets, such as oversampling, undersampling, and a combination of both techniques. This section also describes `SMOTE` for generating synthetic data in tabular datasets.

## WHAT IS FEATURE SELECTION?

Feature selection is crucial for building efficient and effective machine learning models. It's important to note that the best feature selection method may vary depending on the nature of the data and the problem you're trying to solve.

Feature selection is an essential process in machine learning wherein you select those features in your data that contribute most to the prediction variable or output. The goal is to improve the model's performance, reduce overfitting, and reduce the computational cost. Feature selection is important for several reasons, some of which are listed as follows:

- reduces overfitting by removing redundant or irrelevant features
- improves accuracy by considering only relevant features
- reduces training time due to fewer data dimensions

### Three Types of Feature Selection Methods

There are several categories of feature selection methods (each containing multiple methods) that you can use in machine learning, some of which are listed as follows, followed by subsections that provide additional information:

- filter methods
- wrapper methods
- embedded methods

## FILTER METHODS

*Filter methods* select features based on general metrics like correlation. Filter methods for feature selection evaluate the relevance of the features based on their inherent characteristics and statistical properties. Moreover, they are independent of any machine learning algorithms.

In a sense, filter methods are foundational and are often the first step in the feature selection process. They are quick to compute and provide a good understanding of the data's characteristics, but they consider each feature individually, rather than in combination with others, which might lead to suboptimal feature sets in some cases. The following subsections discuss some popular filter methods for feature selection, along with Python code snippets.

### Variance Threshold

This method removes features whose variance is below a certain threshold (hence its name), which is based on the assumption that low variance features contain less information. Here is a sample code block that involves variance threshold:

```
from sklearn.feature_selection import VarianceThreshold

# Sample data
X = [[0, 1, 0.5], [0, 1, 0.7], [0, 1, 0.8], [0, 1, 0.9]]

# Create VarianceThreshold object
selector = VarianceThreshold(threshold=0.01)
X_new = selector.fit_transform(X)
```

### Chi-Squared Test

This test measures the dependence between stochastic variables. As such, this test is useful for categorical features to evaluate the likelihood that any observed difference between the categories arises by chance. Here is a sample code block that involves the chi-squared test:

```
from sklearn.datasets import load_iris
from sklearn.feature_selection import SelectKBest, chi2

X, y = load_iris(return_X_y=True)
X_new = SelectKBest(chi2, k=2).fit_transform(X, y)
```

### ANOVA F-test

This method is used for numerical input and categorical output. It estimates the degree of linear dependency between two random variables. Here is a sample code block that involves an F-test:

```
from sklearn.datasets import load_iris
from sklearn.feature_selection import SelectKBest, f_classif
```

```
X, y = load_iris(return_X_y=True)
X_new = SelectKBest(f_classif, k=2).fit_transform(X, y)
```

### Mutual Information

This technique measures the dependency between two variables, and its value is zero if and only if two random variables are independent. Here is a code block that uses mutual information:

```
from sklearn.datasets import load_iris
from sklearn.feature_selection import SelectKBest, mutual_info_classif

X, y = load_iris(return_X_y=True)
X_new = SelectKBest(mutual_info_classif, k=2).fit_transform(X, y)
```

### Correlation Coefficient

Features that are highly correlated with the target can be preferred, whereas features that are correlated with each other instead of the target are candidates for removal. Here is a code block that removes a correlated feature:

```
import pandas as pd

# Sample data
df = pd.DataFrame({
    'A': [1, 2, 3, 4],
    'B': [1.1, 2.1, 3.1, 4.1],
    'C': [5, 6, 7, 8],
    'target': [0, 1, 0, 1]
})

# Calculate correlations
correlations = df.corr()['target'].abs().sort_values(ascending=False)
selected_features = correlations.index[1:]  # Excluding the target
itself
```

Another major category of feature selection techniques involves wrapper methods, which is the topic of the next section.

## WRAPPER METHODS

*Wrapper methods* use a model to evaluate the importance of features based on the performance of a specific machine learning algorithm. These methods search for the best subset of features by training the model on various feature subsets and selecting the best subset of features.

Keep in mind that some of these methods, such as `Boruta` (discussed later in this chapter) and the `SequentialFeatureSelector` from `mlxtend`, require additional libraries to be installed. Moreover, wrapper methods can be computationally expensive, especially for high-dimensional datasets, as they involve training a model multiple times.

The following subsections discuss some popular wrapper methods for feature selection, along with `Python` code blocks that you can use to create complete code samples for your use cases.

### Recursive Feature Elimination (RFE)

RFE is a method that uses recursion in order to rank features based on the given estimator's importance. A code block that uses RFE is shown as follows:

```
from sklearn.datasets import make_classification
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

X,y = make_classification(n_samples=1000,n_features=100,n_informative=2)
estimator = LogisticRegression(max_iter=500)
selector = RFE(estimator, n_features_to_select=5, step=1)
X_new = selector.fit_transform(X, y)
```

### Recursive Feature Elimination With Cross-Validation (RFECV)

RFECV extends RFE by using cross-validation to select the best number of features, as shown in the following code block:

```
from sklearn.datasets import make_classification
from sklearn.feature_selection import RFECV
from sklearn.linear_model import LogisticRegression

X, y = make_classification(n_samples=1000, n_features=100,
n_informative=5)
estimator = LogisticRegression(max_iter=500)
selector = RFECV(estimator, step=1, cv=5)
X_new = selector.fit_transform(X, y)
```

### Sequential Feature Selection (SFS)

SFS is a family of algorithms that adds or removes features based on a criterion, as shown in the following code block:

```
from mlxtend.feature_selection import SequentialFeatureSelector
from sklearn.neighbors import KNeighborsClassifier

X, y = make_classification(n_samples=1000, n_features=25,
n_informative=3)
knn = KNeighborsClassifier(n_neighbors=4)
sfs = SequentialFeatureSelector(knn, k_features=5, forward=True,
floating=False)
X_new = sfs.fit_transform(X, y)
```

### Backward Feature Elimination

This technique works as follows: start with the full set of features, and then remove one feature at a time until the desired number features (or performance criterion) is reached. A sample code block is shown as follows:

```
from mlxtend.feature_selection import SequentialFeatureSelector
from sklearn.neighbors import KNeighborsClassifier
```

```
X, y = make_classification(n_samples=1000, n_features=25,
n_informative=3)
knn = KNeighborsClassifier(n_neighbors=4)
sbs = SequentialFeatureSelector(knn, k_features=5, forward=False,
floating=False)
X_new = sbs.fit_transform(X, y)
```

### Boruta

Boruta is a feature selection algorithm that identifies all relevant features by comparing the importance of the original set of attributes with importance achievable at random. A sample code block is shown as follows:

```
from boruta import BorutaPy
from sklearn.ensemble import RandomForestClassifier

X, y = make_classification(n_samples=1000, n_features=25,
n_informative=3)
rf = RandomForestClassifier(n_jobs=-1, class_weight='balanced',
max_depth=5)
boruta_selector = BorutaPy(rf, n_estimators='auto', verbose=0,
random_state=1)
X_new = boruta_selector.fit_transform(X, y)
```

A third major category of feature selection techniques involves embedded methods, which is the topic of the next section.

## EMBEDDED METHODS

*Embedded methods* are algorithms that have built-in feature selection methods. In other words, embedded methods are feature selection techniques that are integrated into the training process of a machine learning algorithm. They exploit the model's learning process to evaluate the importance of each feature, and some of them inherently perform feature selection as a part of the training.

Embedded methods are efficient because they perform feature selection and model training simultaneously. The selected features are specifically tailored for the chosen model, which can result in better performance compared to filter and wrapper methods. Here's a list of some popular embedded methods for feature selection, followed by `Python` code samples:

- L1 regularization (Lasso)
- decision trees (and tree-based models)
- elastic net
- LightGBM
- recursive feature elimination (RFE)

### L1 Regularization (Lasso)

L1 regularization adds a penalty to the absolute values of coefficients. Features with zero coefficients after the training are considered unimportant.

```
from sklearn.linear_model import Lasso
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=1000, n_features=100, n_informative=10)
lasso = Lasso(alpha=0.1)
lasso.fit(X, y)
selected_features = [i for i, j in enumerate(lasso.coef_) if j != 0]
```

### Decision Trees (and Tree-Based Models)

Tree-based algorithms like decision trees, random forests, and gradient boosted trees rank features based on their importance derived from the number of splits for a given tree node as well as the depth of the tree. Here is a code block that uses a random forest classifier:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=1000, n_features=25, n_
informative=5)
clf = RandomForestClassifier()
clf.fit(X, y)
importances = clf.feature_importances_
selected_features = [i for i, j in enumerate(importances) if j > 0.05]
```

### Elastic Net

Elastic net combines L1 and L2 regularization, and it can be useful if there are multiple correlated features. Here is a code block that uses elastic net:

```
from sklearn.linear_model import ElasticNet
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=1000, n_features=100, n_informative=10)
en = ElasticNet(alpha=0.1, l1_ratio=0.5)
en.fit(X, y)
selected_features = [i for i, j in enumerate(en.coef_) if j != 0]
```

### LightGBM

LightGBM is a gradient boosting framework that uses tree-based learning algorithms. It offers built-in feature importance evaluations. Here is a code block that uses LightGBM:

```
from lightgbm import LGBMClassifier
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=1000, n_features=25,
n_informative=5)
lgbm = LGBMClassifier()
lgbm.fit(X, y)
importances = lgbm.feature_importances_
selected_features = [i for i, j in enumerate(importances) if j > 50]
```

**NOTE** *LightGBM requires an additional library to be installed.*

### Linear Models with Recursive Feature Elimination

RFE can be combined with any estimator that exposes the `coef_` or `feature_importances_` attribute after fitting a model to the data. Here is a code block that uses RFE:

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=1000, n_features=100,
n_informative=5)
estimator = LogisticRegression(max_iter=500)
selector = RFE(estimator, n_features_to_select=5, step=1)
X_new = selector.fit_transform(X, y)
```

## THE NEED FOR FEATURE SCALING AND TRANSFORMATIONS

There are several reasons for scaling the data values in a dataset, two of which are listed here:

- some machine learning algorithms are affected by unscaled data
- some machine learning algorithms assume data is normally distributed

In particular, machine learning algorithms that apply distance-based metrics to dataset features are affected by unscaled data, which includes the following algorithms:

- kNN (k nearest neighbor, which is a classification algorithm)
- kMeans (a clustering algorithm)

You can apply the following techniques to ensure that features are scaled:

- StandardScaler
- MinMaxScaler
- RobustScaler

In addition to scaling features, some machine learning algorithms assume that data is normally distributed. In particular, the following algorithms make this assumption:

- linear regression (a regression algorithm)
- logistic regression (a classification algorithm)

If you want to transform the data in a dataset so that it has a different distribution, the following transformations will transform features into normally distributed features:

- exponential transformation
- logarithmic transformation

- reciprocal transformation
- square root transformation

NOTE *The preceding list of scaling algorithms and transformation algorithms are discussed in the next chapter.*

## LABELED, UNLABELED, AND MULTICLASS CLASSIFICATION

Classification algorithms and regression algorithms work with labeled data, which means that there is a so-called target column in the dataset whose values identify the associated row. If the target column contains nonnumerical values, you can use the `Pandas` map() function to replace those values with numeric values. For example, if a gender feature contains the values male and female, replace with numeric values as shown here (`df` is a pre-populated `Pandas DataFrame`):

```
df['gender'] = df['gender'].map({'male':0, 'female':1})
```

The following subsection contains some additional details regarding labeled and unlabeled data.

### Labeled Versus Unlabeled Data

Labeled data is often preferred because it saves you from performing the task of labeling the data yourself. However, most data are unlabeled, and the cost of labeling that data can be prohibitive. While it's straightforward and inexpensive (e.g., Mechanical Turk) to manually label pictures of dogs and cats, it's obviously much more costly to label x-rays that contain signs of tumors.

One technique involves the use of a clustering algorithm (you can try several such algorithms) to create an optimal set of clusters, after which you can assign the same label to all the items in each cluster. However, there is an important consideration if you use this approach: is the accuracy of the labels good enough for your use case? For instance, if a clustering algorithm provides 80% accuracy and you need 90% accuracy, then you need explore how to improve the accuracy of labeled data if you proceed with the newly labeled dataset.

## WORKING WITH IMBALANCED DATASETS

Imbalanced datasets contain at least one class that has significantly more values than another class in the dataset. For example, if class A has 99% of the data and class B has 1%, which classification algorithm would you use? Unfortunately, classification algorithms don't work as well with this type of imbalanced dataset.

Imbalanced classification involves processing imbalanced datasets, and the following list contains several well-known techniques:

- random resampling rebalances the class distribution
- random undersampling deletes examples from the majority class
- random oversampling duplicates data in the minority class
- SMOTE (synthetic minority oversampling technique)

*Random resampling* rebalances the class distribution by resampling the data space in order to reduce the discrepancy between the number of rows in the majority class and the minority class.

On the other hand, the *random undersampling* technique removes samples that belong to the majority class from the dataset, and involves the following:

- randomly removes samples from majority class
- can be performed with or without replacement
- alleviates imbalance in the dataset
- may increase the variance of the classifier
- may discard useful or important samples

However, random undersampling does not work so well with extremely unbalanced datasets, such as a 99% and 1% split into two classes. Moreover, undersampling can result in losing information that is useful for a model.

*Random oversampling* generates new samples from a minority class: this technique duplicates examples from the minority class.

Another option to consider is the `Python` package `imbalanced-learn` in the `scikit-learn-contrib` project. This project provides various resampling techniques for datasets that exhibit class imbalance. More details are here:

*https://github.com/scikit-learn-contrib/imbalanced-learn*

Another well-known technique is called `SMOTE`, which involves data augmentation (i.e., synthesizing new data samples). `SMOTE` was initially developed by means of the `kNN` algorithm (other options are available), and it can be an effective technique for handling imbalanced classes. `SMOTE` is discussed in more detail in a later section.

## Detecting Imbalanced Data

This straightforward step involves counting the number of rows that are associated with each class. First read the dataset into a `Pandas` data frame (let's call it `df`) and then invoke the following code snippet:

```
df['your-target-column'].value_counts()
```

An example of the output from the preceding code snippet is here:

```
[OUT]
1    21000
0     6000
```

As you can see in the previous output, class 1 is more than three times larger than class 0, so this column in the dataset is imbalanced.

The `Python`-based open-source library `Scikit-Learn` has a vast set of algorithms for machine learning, some of which support a `class_weight` parameter, as listed here:

- LogisticRegression
- Perceptron
- RandomForest
- SVM

Simply set the value of the `class_weight` parameter equal to `balanced` before training the chosen model. Incidentally, it might also be worthwhile to train the chosen model with the default value for the `class_weight` parameter. Check the online Scikit-Learn documentation for more information.

### Rebalancing Datasets

Let's return to the example of a dataset for which class A has 99% of the data and class B has 1%. Which classification algorithm would you use? The following list contains several well-known techniques for handling imbalanced datasets (not in any particular order):

- random resampling (rebalances the class distribution)
- random oversampling (duplicates data in the minority class)
- random undersampling (deletes examples from the majority class)
- algorithm selection
- cross-validation for imbalanced data
- generating synthetic data (e.g., SMOTE)
- performance metric selection

Several of the techniques in the preceding list are discussed in the following subsections.

### Specify Stratify in Data Splits

This step also is straightforward because Scikit-Learn supports a `stratify` parameter that ensures the data is split so that the train data and test data contain the same proportion of class values. For example, if a dataset contains 60% and 40%, respectively, of class `A` and class `B` in a column, then the train data and test data will contain the same proportions for class `A` and `B`.

The following code block demonstrates how to split a dataset where `X` and `y` have already been initialized so that the data is stratified:

```
from sklearn.model_selection import train_test_split

X_train,X_test,y_train,y_test = train_test_split(X,
                                                 y,
                                                 test_size=0.25,
                                                 random_state=42,
                                                 stratify=y)
```

## FEATURE IMPORTANCE

You probably want to know which features in a dataset are significant as well as their relative importance. For example, the number of bedrooms and bathrooms in a house are significant features that affect the price of a house, whereas the ticket number for an airplane ticket is most likely not a significant feature.

Suppose that a dataset has a "target" column, which is a column whose values we want to predict during the testing phase (i.e., after the training phase). A target column exists in datasets for classification tasks (which includes linear regression and classification algorithms), whereas datasets without a target column are clustering tasks. In essence, the process of assigning labels to an unlabeled dataset results in the creation a target column in a dataset.

The key idea pertaining to feature importance involves assigning a numeric score between 0 and 100 that is assigned to each feature: this numeric score is a measure of the extent to which that feature can predict the target variable. The conditions for assigning feature importance are summarized here:

- select all the relevant features
- assign a value between 0 and 100 to each feature
- ensure the sum of the feature values is 100

As a side note: if we divide the feature values by 100 we have a probability distribution.

One straightforward way to determine feature importance involves the random forest algorithm in `Sklearn`. For a given feature, `Sklearn` calculates its importance (which is a numeric value) by examining the average reduction in impurity in the set of tree nodes that involve that feature. This calculation is performed for each feature after the training step has completed, after which the values are scaled (essentially a probability distribution) to simplify the comparison of the importance of features.

Listing 5.1 displays the contents of `feature_importance.py` that shows you how to determine the important features in the `Titanic` dataset as well as their relative importance.

***LISTING 5.1: feature_importance.py***

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

import pandas as pd
import numpy as np

df = pd.read_csv("titanic2.csv",
                 usecols = ['survived','sex','age','class','embarked'])
y = df.pop("survived")

# convert categories to numeric values:
df['sex'] = df['sex'].replace({'male':1, "female":0})
df['class'] = df['class'].replace({'First':1, "Second":2, "Third":3})
df['embarked'] = df['embarked'].replace({'S':0, "C":1, "Q":2})

# perform train test split:
X_train, X_test, y_train, y_test = train_test_split(df, y,
test_size=0.2, random_state=42)

rng = np.random.RandomState(0)
random_forest = RandomForestClassifier(n_estimators=10, random_state=rng)
random_forest.fit(X_train,y_train)
```

```
# determine the relative feature importance:
feature_imp = pd.DataFrame({'feature':X_train.columns,'importance':
np.round(random_forest.feature_importances_,3)})
feature_imp = feature_imp.sort_values('importance',ascending=False).
set_index('feature')
print("feature_imp.head():")
print(feature_imp.head())
```

Listing 5.1 starts with several `import` statements and then initializes the data frame `df` with several columns (features) from a `CSV` file. The next portion of Listing 5.1 converts the contents of the features `sex`, `class`, and `embarked` with corresponding integer values, followed by the standard `Sklearn` class `train_test_split()` that constructs four rectangular subregions of the data frame `df`: the data in `X_train` and `y_train` are used during the training step, whereas the data in `X_test` and `y_test` are used for the test (i.e., prediction) step.

Next, the variable `random_forest` is initialized as an instance of the class `RandomForestClassifier` that is defined in `sklearn.ensemble` (which makes sense because a forest is a collection, or ensemble, of trees). Then the `fit()` method is invoked in order to train the model `random_forest`.

At this point the variable `feature_imp` is initialized as a data frame that contains the columns of `X_train`, as well as initializing the parameter importance so that the feature importance values are displayed in decreasing order and with three decimal places of accuracy. The final code snippet displays the top five features with respect to their relative importance. Now launch the code in Listing 5.1 and you will see the following output:

```
feature_imp.head():
          importance
feature
age            0.583
sex            0.346
class          0.038
embarked       0.033
```

As you can see from the preceding output, the age feature is the most important feature for determining whether or not an individual survived the sinking of the Titanic, followed by gender, as well as two low-priority features, class and point of embarkation.

## WHAT IS SMOTE?

`SMOTE` (synthetic minority oversampling technique) is a technique for synthesizing new samples for a dataset, which is data augmentation (i.e., synthesizing new data samples) that is performed before you use a classification algorithm. `SMOTE` was initially developed by means of the `kNN` algorithm (other options are available), and it can be an effective technique for handling imbalanced classes. This technique is based on linear interpolation:

- Step 1: select samples that are close in the feature space
- Step 2: draw a line between the samples in the feature space
- Step 3: draw a new sample at a point along that line

A more detailed explanation of the SMOTE algorithm is here:

- select a random sample "a" from the minority class
- now find k nearest neighbors for that example
- select a random neighbor "b" from the nearest neighbors
- create a line L that connects "a" and "b"
- randomly select one or more points "c" on line L

If need be, you can repeat this process for the other (k-1) nearest neighbors in order to distribute the synthetic values more evenly among the nearest neighbors.

## SMOTE Extensions

The initial `SMOTE` algorithm is based on the `kNN` classification algorithm, which has been extended in various ways, such as replacing `kNN` with `SVM`. A list of `SMOTE` extensions is shown as follows:

- borderline-SMOTE (kNN)
- borderline-SMOTE (SVM)
- adaptive synthetic sampling (ADASYN)
- DBSMOTE

Perform an Internet search for more details about these algorithms, and also navigate to the following URL:

*https://en.wikipedia.org/wiki/Oversampling_and_undersampling_in_data_analysis*

## An Alternative to SMOTE

The following article presents an alternative to SMOTE and undersampling:

*https://medium.com/@ali.soleymani.co/stop-using-smote-to-treat-class-imbalance-take-this-intuitive-approach-instead-9cb822b8dc45*

According to the preceding article, SMOTE and undersampling do not achieve high levels of recall and also additional complexity. The author provides another technique in the preceding link, which is applied to a dataset that contains two classes, with the following values:

```
Nmin = number of elements in the minority class
Nmaj = number of elements in the majority class
```

Now find a pair of positive numbers that satisfy the following:

```
Nmin * Wmin = Nmaj * Wmaj
```

Now assign the values Wmin and Wmaj as the weights of the elements in the minority class and majority class, respectively. One simple solution involves calculating the LCM (lowest common multiple) of Nmin and Nmaj, and then computing values for Wmin and Wmax as follows:

```
lcm = lowest common multiple of Nmin and Nmaj = LCM(Nmin,Nmaj)
Wmin = lcm/Nmin
Wmax = lcm/Nmax
```

Example #1:

```
Nmin = 500
Nmaj = 1200
```

Since Nmaj:Nmin = 12:5, select values for Wmin and Wmaj such that Wmin:Wmaj = 12:5. In addition, `LCM(Nmin,Nmaj) = 6000`, which means that:

```
Wmin = lcm/Nmin = 6000/500 = 12
Wmax = lcm/Nmax = 6000/1200 = 5
```

Example #2:

```
Nmin = 50
Nmaj = 950
```

Since Nmaj:Nmin = 19:1, select values for Wmin and Wmaj such that Wmin:Wmaj = 19:1. In addition, `LCM(Nmin,Nmaj) = 950`, which means that:

```
Wmin = lcm/Nmin = 950/50 = 19
Wmax = lcm/Nmax = 950/950 = 1
```

The link at the beginning of this section describes the preceding technique in greater detail, along with the results of training models using this technique (including comparison charts).

## WHAT ARE TRANSFORMS?

Transforms involve applying functions to a dataset in order to achieve linearity, after which you might gain insights into the transformed data that were not apparent in the original datasets. For instance, if the transformed data resembles a linear-like distribution, use linear regression to find the best-fitting line for the transformed data.

You can apply a transformation to one or to both of the variables. For example, a logarithmic transformation calculates the logarithm of the independent variable and makes no modification to the dependent variable. On the other hand, a power transformation calculates the logarithm of the independent variable as well as the dependent variable.

Keep in mind that machine learning algorithms that use the Euclidean distance between pairs of values are biased toward the larger class in imbalanced datasets. Therefore, it's important that you scale the values in the dataset, which can be performed via standardization as well as normalization. Several common transformations for achieving linearity are available, some of which are listed as follows:

- cube root transformation
- hyperbolic transformation
- logarithmic transformation
- power transformation
- square root transformation

The following subsections briefly describe each transformation in the preceding bullet list.

## Cube Root Transformation

Listing 5.4 displays the contents of `cube_root_transformation.py` that illustrates how to detect time gaps and display a summary of results.

***LISTING 5.4: cube_root_transformation.py***

```
import numpy as np
import matplotlib.pyplot as plt

#make this example reproducible
np.random.seed(0)

#create beta distributed random variable with 300 values
data = np.random.beta(a=4, b=15, size=300)

#create cube root transformed data
data_cbrt = np.cbrt(data)

#define grid of plots
fig, axs = plt.subplots(nrows=1, ncols=2)

#create histograms
axs[0].hist(data, edgecolor='black')
axs[1].hist(data_cbrt, edgecolor='black')

#add title to each histogram
axs[0].set_title('Original Data')
axs[1].set_title('Cube-Root-Transformed Data')
plt.show()
```

Listing 5.4 starts with `import` statements and then initializes the variable `data` with a set of 300 randomly distributed values to create a sample beta distribution. Next, the variable `data_cbrt` is initialized with the cube root of the values in `data` by invoking the `cbrt()` method in `NumPy`. The final portion of Listing 5.4 generates a histogram based on the values in data, followed by a second histogram based on the values in `data_cbrt`. Now launch the code in Listing 5.4 and you will see an image that is similar to Figure 5.1.

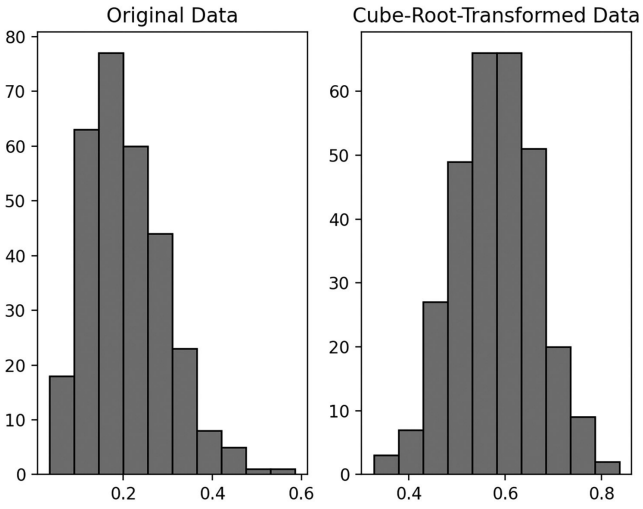



***FIGURE 5.1*** A cube root data transformation.

## Other Transformations

This section contains code snippets that show you how to transform data via the following transformation:

- a square root function
- an exponential function
- a hyperbolic function
- a logarithmic function
- a power transformation

Perform a square root transformation by replacing the code block shown in bold in Listing 5.4 with the following code block:

```
#create square root transformed data
data_sqrt = np.sqrt(data)
```

Perform an exponential transformation by replacing the code block shown in bold in Listing 5.4 with the following code block:

```
#create exponential transformed data
data_exp = np.exp(data)
```

Perform a hyperbolic transformation by replacing the code block shown in bold in Listing 5.4 with the following code block:

```
#create hyperbolic transformed data
data_hyperbolic = np.tanh(data)
```

Perform a logarithmic transformation by replacing the code block shown in bold in Listing 5.4 with the following code block:

```
#create logarithmic transformed data
data_log = np.log(data + 1)
```

Perform a power transformation by replacing the code block shown in bold in Listing 5.4 with the following code block:

```
#create power transformed data
data_transformed = np.power(data, 2)
```

## SUMMARY

This chapter started with a brief introduction to feature selection, which is one aspect of feature engineering that is discussed in the next chapter. You learned that feature selection performs a selection of a subset of existing features, whereas feature extraction finds a linear combination of existing features in a dataset.

In addition, you learned about the three categories of feature selection, which are filter methods, wrapper methods, and embedded methods. You also saw that each of these categories of feature selection provides multiple algorithms.

Next you learned about imbalanced datasets, and techniques such as undersampling and oversampling for adjusting the number of rows of the classes of the target feature, as well as the SMOTE algorithm for generating synthetic data. Finally, you learned various types of transformations that you can perform on the data in a dataset in order to change the distribution of the data.

CHAPTER 6

# FEATURE ENGINEERING

This chapter explores various topics pertaining to feature engineering that complement feature selection (discussed in the previous chapter) such as feature extraction.

The first part of this chapter introduces the notion of feature engineering, followed by the topic of feature importance, which includes feature selection (discussed in Chapter 5) as well as feature extraction. In addition, you will learn about feature scaling and some of the machine learning algorithms that require feature scaling.

The second part discusses data and machine learning, which includes labeled versus unlabeled data and multiclass classification. The third part discusses sampling techniques, such as undersampling and oversampling, as well as the limitations of resampling data. The fourth part of this chapter discusses `SMOTE`, which is a technique for generating synthetic data, followed by data augmentation.

The final section discusses `NLP`-based data augmentation. You will learn about Transformer-based text augmentation for the `BERT` and `T5` language models. Note that this portion of the chapter is marked "optional" because the `NLP` concepts and the details of the `BERT` and `T5` data models are not presented in this book.

## WHAT IS FEATURE ENGINEERING?

Feature engineering requires creativity and domain knowledge. The goal is to improve model performance while also ensuring that the features make sense in the context of the problem being solved.

Feature engineering is a crucial step in the process of building machine learning models. It involves creating new features from the existing data or transforming the existing features to improve the model's performance. Feature engineering is important for several reasons, some of which are listed as follows:

- improves model accuracy
- helps in understanding the underlying structure of the data
- can reduce the complexity of models

Feature engineering is a term that encompasses feature selection and feature extraction. As you saw in Chapter 5, feature selection involves selecting a subset of *existing* features in a dataset, whereas feature extraction involves creating *new* features that are based on a combination of existing features.

Moreover, feature extraction can be performed via a linear combination of features (such as `PCA` in Chapter 7) as well as a nonlinear combination of features. This chapter only discusses algorithms that involve linear combinations of features. The following subsections provide additional information regarding feature selection and feature extraction.

## Types of Feature Engineering

The following bullet list contains different types of feature engineering:

- feature creation: deriving new features from existing ones
- feature transformation: normalization, scaling, or applying mathematical transformations
- feature selection: selecting a subset of the most important features
- feature extraction: reducing the dimensionality of the data, e.g., PCA

The following bullet list contains different techniques for feature engineering:

- binning: grouping values into bins
- one-hot encoding: converting categorical variables into a binary matrix
- polynomial features: creating interaction terms
- aggregation: summarizing multiple data points
- embedding: representing complex structures in simpler formats

The remainder of this section contains short `Python` code blocks that illustrate some of the techniques that are listed in the two previous bullet lists.

As you can undoubtedly surmise, *feature creation* involves a new feature based on existing features. Here is a code block that illustrates a simple example of creating a new feature that is based on two existing features in a data frame:

```
import pandas as pd

# Sample dataframe
df = pd.DataFrame({
        'height': [5.5, 6.1, 5.8],
        'weight': [150, 180, 160]
     })

# Creating a BMI (Body Mass Index) feature
df['BMI'] = df['weight'] / (df['height']**2)
```

*Feature transformation* involves changing the distribution of the data in a dataset. One technique for doing so is called min-max scaling to scale a feature to a [0, 1] range. The following snippet illustrates min-max scaling:

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
df['height_scaled'] = scaler.fit_transform(df[['height']])
```

*One-hot encoding* involves converting a categorical feature into a binary matrix.

```
df = pd.DataFrame({'color': ['red', 'blue', 'green']})
df_encoded = pd.get_dummies(df, columns=['color'])
```

*Binning* is the process of grouping values into bins, an example of which is shown here:

```
df['height_bin'] = pd.cut(df['height'], bins=[5, 5.5, 6, 6.5],
labels=['short', 'average', 'tall'])
```

The next section summarizes a common sequence of feature engineering steps (among others) that you need to perform in order to adequately prepare a dataset for a machine learning model.

## WHAT STEPS ARE REQUIRED TO TRAIN A MODEL?

The following bullet list contains one fairly typical sequence of steps, starting from the data preparation and through to the training of a model:

- data cleaning (in Chapter 3)
- data wrangling (in Chapter 4)
- imbalanced datasets
- feature scaling
- feature transformations
- feature selection (in Chapter 5)
- feature extraction (e.g., PCA in Chapter 7)
- hyperparameter tuning
- a different algorithm (optional)
- cloud-based training

The amount of time that is required for each step in the preceding bullet list depends heavily on the nature of the data in the dataset, followed by the choice of machine learning algorithms.

If you have a dataset that is in pristine condition (i.e., no missing data, no incorrect data, and no outliers), then the first two steps are essentially completed. However, if you have an imbalanced dataset, consider downsampling or upsampling the dataset, or possibly a combination of both techniques.

Note that balanced data in a dataset is desirable for algorithms that use a distance metric, because imbalanced data can cause algorithms to "favor" the class with the larger number of data points. On the other hand, from a desirability perspective, we want *everyone* healthy in a cancer-related dataset: as a result, we would prefer a highly imbalanced dataset in which all patients are healthy. Thus, the balanced-versus-imbalanced dataset involves two opposite goals.

If you decide to use an algorithm that uses a distance-based metric (such as `kNN` or `kMeans`), then you need to make sure that the data is properly scaled (as discussed in Chapter 5). In addition, there is the scenario in which you apply a transformation (discussed in Chapter 5) to a dataset in order to transform data so that it becomes normally distributed.

Another aspect to keep in mind is the number of features in a dataset. If the dataset contains a very limited number of features, and it's obvious which features you need to select for training a model, then feature selection will probably work well. By contrast, if there are many features, or the choice of significant features is not obvious, then consider performing dimensionality reduction (such as PCA) in order to work with a smaller number of features.

Finally, if the dataset is more than 500 gigabytes, you might encounter space-related issues on a laptop that can be avoided by working in a cloud-based environment. Google Colaboratory enables you to train models on a GPU for 12 consecutive hours per day, every day, and it's free. If need be, you can consider the USD 9.99/month which provides additional RAM and execution time. Moreover, GCP offers a 2-terabyte storage option for USD 5 per month, and a 5-terabyte storage option for USD 20 per month, which will probably accommodate most (if not all) of your datasets. In addition to Google Colaboratory and GCP, there are other cloud-based solutions available, so shop around to find the solution that works best for your needs.

Keep in mind that even if you perform all the preceding steps that you believe are necessary, the trained model might not achieve the expected results on the test set. In this case, you can attempt either (or both) of the following:

- hyperparameter tuning
- train on different algorithms

Hyperparameter tuning can be a time-consuming process, so you might be tempted to skip the tuning step and select another algorithm. However, remember that there is no *a priori* assurance that a different algorithm will produce better results: in fact, they might be worse.

## MACHINE LEARNING AND ALGORITHM SELECTION

After you have cleaned a dataset, scaled the data, and performed transformations (if necessary), you can select a machine learning algorithm to train a model on the dataset. A labeled dataset is a dataset that contains a "target" column, which is a column that contains values that identify each row in the dataset. Moreover, a labeled dataset enables you to select from various classification algorithms, including:

- logistic regression
- naive Bayes
- decision trees
- random forests
- SVMs (support vector machines)
- CNNs (part of deep learning)

If you have unlabeled data, then you can explore one or more of the following clustering algorithms (and other algorithms are also available):

```
kMeans
meanShift
kMeans++
kMedioid
DBScan
```

Another consideration involves *imbalanced datasets* (discussed previously in this chapter), which can adversely affect the accuracy of some machine learning algorithms as well as techniques for handling imbalanced datasets, both of which are discussed in Chapter 5.

### Training Large Datasets

If you have a dataset that is over 100MB (or whatever you consider to be "large") and you want to obtain results more quickly, these suggestions might save you a significant amount of time.

First, perform your data cleaning steps on a 10% subset of the data. Second, use the smaller dataset in the previous step with each of the algorithms that you plan to use. This approach will give you faster results than the time for training the model just once on the entire dataset.

After completing the preceding sequence of steps, try randomly selecting yet another 10% and randomly select one of the algorithms to train the model and see if the results are comparable.

Finally, train the model on the entire dataset with all the algorithms, and ideally the results will be comparable. However, if there are differences then you could explore the data to determine if and where there might be something anomalous in the data.

Of course, avail yourself of the GPU in Google Colaboratory (or some other cloud-based service), because it's probably much faster and also free to use for at least 10 hours.

## FEATURE IMPORTANCE

Before delving into the details of feature engineering, you probably want to know which features in a dataset are significant and their relative importance. For example, the number of bedrooms and bathrooms in a house are significant features that affect the price of a house, whereas the ticket number for an airplane ticket is most likely not a significant feature.

Suppose that a dataset has a "target" column, which is a column whose values we want to predict during the testing phase (i.e., after the training phase). A target column exists in datasets for classification problems (which includes linear regression and classification algorithms), whereas datasets without a target column are clustering tasks.

The key idea pertaining to feature importance involves assigning a numeric score between 0 and 100 that is assigned to each feature, which measures the extent to which that feature can predict the target variable. The conditions for assigning feature importance are summarized here:

- select all the relevant features
- assign a value between 0 and 100 to each feature
- ensure the sum of the feature values is 100

As a side note: if we divide the feature values by 100, we have a probability distribution.

One straightforward way to determine feature importance involves the random forest algorithm in `Sklearn`. For a given feature, `Sklearn` calculates its importance (which is a numeric value) by examining the average reduction in impurity in the set of tree nodes that involve that feature. This calculation is performed for each feature after the training step has completed, after which the values are scaled (essentially a probability distribution) to simplify the comparison of the importance of features.

Listing 6.1 displays the contents of `feature_importance.py` that shows you how to determine the important features in the Titanic dataset as well as their relative importance.

***LISTING 6.1: feature_importance.py***

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

import pandas as pd
import numpy as np

df = pd.read_csv("titanic2.csv")
df = df[["survived","sex","age","class","embarked"]]
y = df.pop("survived")

# convert categories to numeric values:
df['sex'] = df['sex'].replace({'male':1, "female":0})
df['class'] = df['class'].replace({'First':1, "Second":2, "Third":3})
df['embarked'] = df['embarked'].replace({'S':0, "C":1, "Q":2})

# perform train test split:
X_train, X_test, y_train, y_test = train_test_split(df, y,
test_size=0.2, random_state=42)

rng = np.random.RandomState(0)
random_forest = RandomForestClassifier(n_estimators=10, random_state=rng)
random_forest.fit(X_train,y_train)

# determine the relative feature importance:
feature_imp = pd.DataFrame({'feature':X_train.columns,'importance':
np.round(random_forest.feature_importances_,3)})
feature_imp = feature_imp.sort_values('importance',ascending=False).
set_index('feature')
print("feature_imp.head():")
print(feature_imp.head())
```

Listing 6.1 starts with several `import` statements and then initializes the variable `df` as a data frame that contains the contents of `titanic2.csv` (same as `titanic.csv` except that rows with null values have been removed). The next snippet restricts `df` so that it contains only five features, and then initializes the variable `y` with the target column `survived`, as shown here:

```
df = df[["survived","sex","age","class","embarked"]]
y = df.pop("survived")
```

Now we need to map three features of `df` from string values to numeric counterparts, which is performed by invoking the `df.replace()` method with a `JSON` list of name/value pairs (you can use different integer values if you wish to do so).

The next step is a standard mechanism that populates train-related arrays and test-related arrays with a single line of code. We proceed to initialize `rng` as a random value that's specified in the next code snippet that initializes the `random_forest` variable as an instance of the `RandomForestClassifier` class. At this point we can fit the data to the model by invoking the method `random_forest.fit()` with the training-related data.

The final block of code sets up the `Pandas DataFrame feature_imp` so that it's populated with the relative importance of the selected features and displays the results. Now launch the code in Listing 6.1 and you will see the following output:

```
feature_imp.head():
          importance
feature
age            0.583
sex            0.346
class          0.038
embarked       0.033
```

## FEATURE ENGINEERING AND EXTRACTION

In addition to creating a dataset and "cleaning" its values, generally you will examine the features in a dataset to determine which ones (if any) have little or no value and can be removed from the dataset. The process for doing so involves three main techniques:

- feature engineering
- feature selection
- feature extraction (aka feature projection)

**NOTE** *Feature selection is described in Chapter 5 and that material will not be repeated in this chapter.*

### Feature Engineering

*Feature engineering* is the process of either (a) selecting an existing subset of features (as discussed in Chapter 5) or (b) determining a new set of features that are based on a combination of existing features in order to create a meaningful dataset for a given task.

Domain expertise is often required for this process, even in the cases of relatively simple datasets. Feature engineering can be tedious and expensive, and in some cases, you might consider using automated feature learning. After you have created a dataset, it's a good idea to perform feature selection or feature extraction (or both) to ensure that you have a high-quality dataset.

### Feature Extraction

*Feature extraction* creates new features that are based on combinations of the original features. By contrast, feature selection involves determining a subset of the existing features, which is to say, no new features are created. The net effect of feature selection and feature extraction results in *dimensionality reduction* for a given dataset (not discussed in this book).

Some of the advantages of feature extraction are listed here:

- Can capture interactions between features.
- New features might be more informative than the original ones.
- Often used in situations where the number of features is very high.

Some of the disadvantages of feature extraction are listed here:

- Transformed features often lose their original interpretable meaning.
- Not always straightforward to decide on the number of extracted features.

After you have ascertained the most important features in a dataset (and also cleaned that dataset), the next step involves selecting a model, which is briefly discussed in the next section.

### Feature Extraction Algorithms

The following list contains various well-known feature extraction algorithms in machine learning:

- principal component analysis (PCA)
- independent component analysis (ICA)
- linear discriminant analysis (LDA)

Note that the algorithms in the preceding list perform linear extraction, whereas the following list involves nonlinear extraction/reduction algorithms:

- locally linear embedding (LLE)
- t-distributed stochastic neighbor embedding (t-SNE)
- UMap

In particular, t-SNE is capable of capturing complex nonlinear relationships in the data. This algorithm uses Kullback-Leibler divergence (discussed in Chapter 7) to minimize the divergence between two distributions: a distribution that measures pairwise similarities between the input features, and a distribution that measures pairwise similarities between the points in the low-dimensional space. This makes it particularly good at preserving local structure and can reveal clusters in the data, making it popular for visualization purposes.

At the same time, keep in mind that there are some caveats regarding the t-SNE algorithm (created by Geoffrey Hinton, who is often called the godfather of deep learning):

- high computational complexity
- careful tuning of hyperparameters
- random initialization can yield different results for each "run"
- difficulty in the interpretation of the reduced dimensions

Note that the t-SNE algorithm and the UMap algorithm are both discussed in more detail in Chapter 7.

## FEATURE HASHING

Feature hashing, also known as the hashing trick, is a technique used in machine learning for dimensionality reduction and efficient handling of high-dimensional categorical features. It's particularly useful for large-scale and streaming data because it allows for a fixed-size representation of a potentially infinite feature space. Feature hashing involves the following:

- hash function
- fixed-size array
- collisions

A hash function maps each unique feature name (or identifier) to an integer in a fixed range [0,n-1], where n is the size of the hash table. Multiple feature identifiers may be mapped to the same index due to collisions.

Features are hashed into a fixed-size array or "hash table." The value at each position in the array is the sum of the values of the features that hash to that position.

A collision occurs when different features hash to the same index, at which point a linked list is populated with the feature value that resulted in a collision. If too many collisions occur, then a larger fixed-size array is required: check for online articles that provide additional details.

Advantages of feature hashing include the following:

- Memory Efficient: By using a fixed-size hash table, the memory footprint is kept low, especially useful for high-dimensional data.
- Computationally Efficient: The hashing operation is generally fast, making the technique suitable for real-time and streaming applications.
- No Need for Dictionary: Unlike traditional methods that maintain a dictionary to map categorical variables to numerical identifiers, feature hashing does not require this.

Disadvantages of feature hashing include the following:

- Collisions: Features can collide, causing a loss of information. The likelihood of this can be reduced by increasing the hash table size.
- No Inverse Mapping: Once features are hashed, you cannot revert them back to their original form, which can make the model difficult to interpret.
- No Feature Importance: Since multiple features may be hashed to the same index, it's hard to interpret the importance of individual features.

Listing 6.2 displays the contents of `feature_hashing.py` that shows you how to perform feature hashing on a very simple dataset.

***LISTING 6.2: feature_hashing.py***

```
from sklearn.feature_extraction import FeatureHasher

# Sample data
data = [{'dog': 1, 'cat':2, 'elephant':4},{'dog': 2, 'run': 5}]

# Initialize FeatureHasher
hasher = FeatureHasher(n_features=4, input_type='dict')

# Transform the data
hashed_features = hasher.transform(data)

# The hashed features are now in a fixed-size array
print(hashed_features.toarray())
```

Listing 6.2 starts with an `import` statement and then initializes the variable `data` with a very simple set of name/value pairs. The next code snippet initializes the variable `hasher` as

an instance of the class `FeatureHasher`, and then invokes the `transform()` method with the contents of `data`. The final code snippet displays the hashed features.

```
initial: [{'dog': 1, 'cat': 2, 'elephant': 4}, {'dog': 2, 'run': 5}]
hashed:  [[ 0. -1. -4.  2.]
 [-5. -2.  0.  0.]]
```

## FEATURE SCALING AND ML ALGORITHMS

This section discusses types of scaling that you can perform, along with algorithms that require feature scaling, followed by algorithms that do not benefit from feature scaling.

### Selecting the Type of Scaling

Feature scaling is important in data preprocessing for various algorithms, some of which are listed here:

- Lasso and Ridge penalties
- distance-based models
- k-NN (k nearest neighbor)
- clustering (kMeans)
- SVMs (support vector machines)
- ANNs (artificial neural networks)

The preceding algorithms perform better when the predictors have the same scale or are within the same boundaries. As a general rule, select a particular type of scaling as follows:

- standardization or robust scaling if outliers exist
- standardization for Gaussian distributions
- normalization for non-normal distributions

### Algorithms That Require Feature Scaling

Algorithms and models that depend on distance metrics are sensitive to feature scaling as well as normalization, some of which are listed as follows:

- kMeans clustering
- kNN (k nearest neighbor)
- linear regression
- logistic regression
- NNs (neural networks)
- PCA (principle component analysis)
- SVMs (support vector machines)

### Algorithms That Do Not Require Feature Scaling

Algorithms such as decision trees (for classification or for regression) and random forests involve Gini impurity and entropy instead of distance metrics, so they are not sensitive to feature

scaling. The following list contains some well-known machine learning algorithms that do not require feature scaling:

- decision trees
- tree-based ensemble methods (e.g., random forest)
- naive Bayes
- graph-based algorithms

## DATA SAMPLING TECHNIQUES

The following techniques are examples of well-known sampling data from a dataset:

- undersampling
- oversampling
- resampling

The techniques in the preceding bullet list are briefly described in the following subsections, along with some of the trade-offs. Note that the previous chapter discussed SMOTE, which is a powerful technique (originally based on the `kNN` algorithm) for generating synthetic data in order to achieve more balanced data in a dataset.

### Undersampling

*Undersampling* refers to methods whose purpose is to balance the class distribution for a classification dataset that has a skewed class distribution. These methods focus on a subset of the larger class and are applied to a training dataset, after which a model can be trained on the dataset.

*Random undersampling* also involves randomly selecting examples; however, the random samples are taken from the majority class and then deleted from the training dataset, thereby reducing the imbalance in the dataset.

The *random undersampling* technique removes samples from the dataset, and involves the following:

- randomly remove samples from majority class
- can be performed with or without replacement
- alleviates imbalance in the dataset
- may increase the variance of the classifier
- may discard useful or important samples

However, random undersampling does not work so well with a dataset that has a 99%/1% split into two classes. Moreover, undersampling can result in information loss that is useful for a model.

### Oversampling

*Random oversampling* focuses on randomly selecting (and replacing) examples from the minority class in order to reduce the class imbalance in the training dataset. This technique involves the following:

- oversample examples in the minority class
- duplicate examples from the minority class
- can balance distribution (no new information)

The second technique is better than the preceding technique, which involves the following:

- synthesize new examples from minority class
- a type of data augmentation for tabular data
- this technique can be very effective
- generate new samples from minority class

Another option is the `Python` package `imbalanced-learn` that provides various resampling techniques for datasets that exhibit class imbalance. This package is available in the `scikit-learn-contrib` project. More details are here:

*https://github.com/scikit-learn-contrib/imbalanced-learn*

### Resampling

Random resampling transforms the training dataset into a new dataset, which is quite effective for imbalanced classification problems. Many techniques for handling imbalanced datasets typically involve either generating more samples for the underrepresented class using `SMOTE` (discussed in Chapter 5) or reducing the number of samples selected from the dominant class.

However, some datasets are fundamentally imbalanced due to the nature of the data in the datasets. For example, datasets contain far fewer people with cancer, diabetes, or other life-threatening diseases than the number of healthy people in the dataset. Similarly, datasets containing credit card fraud also have a very low percentage of fraud, and therefore are fundamentally imbalanced.

Unfortunately, oversampling the smaller class can lead to overfitting, and undersampling the dominant class can lead to underfitting. Moreover, machine learning algorithms differ in terms of their sensitivity to imbalanced datasets. For example, logistic regression is less sensitive to class imbalance than a decision tree, whereas ensemble models are more impervious to overfitting.

## DATA AUGMENTATION

In simple terms, data augmentation refers to adding more data to a dataset. Earlier you learned about oversampling, which is a form of data augmentation. In general, the data augmentation techniques that are available depend on the type of data that you want to augment, such as:

- image data
- tabular data
- NLP-related data

Well-known techniques for augmentation image data are typically 2D transforms, some of which are listed here:

- scaling images
- rotating images
- skewing images

Keep in mind that machine learning frameworks such as TensorFlow 2/Keras have built-in support for 2D transforms.

Listing 6.3 displays the contents of `image_augment1.py` that shows you how to augment a `PNG` file.

***LISTING 6.3: image_augment1.py***

```
import numpy as np
import matplotlib.pyplot as plt
import skimage.io as io
from skimage.transform import rotate

filename = "sample3.png"
img = io.imread(filename)

def augment_img(img):
  fig,ax = plt.subplots(nrows=1,ncols=5,figsize=(10,8))

  # original image:
  ax[0].imshow(img)
  ax[0].axis('off')

  # rotate image 45 degrees (counter clockwise):
  ax[1].imshow(rotate(img, angle=45, mode = 'wrap'))
  ax[1].axis('off')

  # rotate image 45 degrees (counter clockwise):
  ax[2].imshow(np.fliplr(img))
  ax[2].axis('off')

  # reflect image via horizontal axis:
  ax[3].imshow(np.flipud(img))
  ax[3].axis('off')

  # rotate image 90 degrees (counter clockwise):
  ax[4].imshow(np.rot90(img))
  ax[4].axis('off')

# augment then display images:
augment_img(img)
plt.show()
```

Listing 6.3 starts with several `import` statements and then initializes the variable `filename` as a reference to a `PNG` and then initializes `img` with the contents of the `PNG`.

The next portion of Listing 6.3 is the `Python` function `augment_img()` that performs four transformations on an image that involves rotations and reflections. The final code snippet invokes this `Python` function with the variable `img`. Launch the code in Listing 6.3 and you will see the original images and three transformed images that are shown in Figure 6.1.

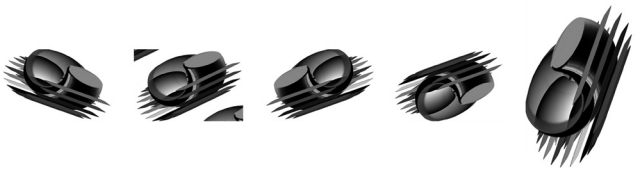


***FIGURE 6.1*** Image augmentation via 2D transformations.

As for tabular data, consider a dataset that consists of 1,000 rows, where each row represents information about a patient. Specifically, 950 patients are healthy, and 50 patients have a terminal disease, which is obviously a highly imbalanced dataset.

## SUMMARY

This chapter started with an explanation of feature importance, and an introduction to feature engineering. Then you learned about feature extraction and feature scaling, as well as some algorithms that require feature scaling.

In addition, you learned about labeled versus unlabeled data and multiclass classification. Next, you learned about imbalanced datasets, how to rebalance them, and how to specify the `stratify` parameter.

Furthermore, you learned about sampling techniques and the `SMOTE` algorithm for generating synthetic data. Finally, you got an introduction to `NLP`-based data augmentation that is relevant for language models such as `BERT` and `T5`.

CHAPTER 7

# DIMENSIONALITY REDUCTION

This chapter discusses dimensionality reduction techniques as well as covariance and correlation matrices.

The first portion of this chapter discusses covariance matrices and correlation matrices. In addition, you will learn about eigenvalues and eigenvectors, and how to calculate them using the Gauss Jordan Elimination algorithm.

The second portion of this chapter discusses the Principal Component Analysis (PCA) algorithm, which is a well-known technique for dimensionality reduction. The section regarding PCA flows nicely into the section that discusses several dimensionality reduction techniques, such as SVD (Singular Value Decomposition), LLE (Locally Linear Embedding), UMAP, t-SNE, and PHATE.

The third section discusses distance metrics, along with well-known distance metrics, such as Pearson Correlation Coefficient, Jaccard Index, and Local Sensitivity Hashing. The final section introduces you to Scikit-Learn, which is a very powerful Python-based machine learning library. This section also introduces you to Bayesian Inference, followed by an introduction to vector spaces.

## COVARIANCE AND CORRELATION MATRICES

This section explains two important matrices: the covariance matrix and the correlation matrix. Although these are relevant for PCA (principal component analysis), which is discussed later, these matrices are not specific to PCA, which is the rationale for discussing them in a separate section. If you are familiar with these matrices, feel free to skim through this section.

### Covariance Matrix

As a reminder, the statistical quantity called the variance of a random variable `X` is defined as follows:

```
variance(x) = [SUM (x - xbar)*(x-xbar)]/n
```

A *covariance matrix* C is a nxn matrix whose values on the main diagonal are the variance of the variables X1, X2, . . . , Xn. The other values of C are the covariance values of each pair of variables Xi and Xj.

The formula for the covariance of the variables X and Y is a generalization of the variance of a variable, and the formula is shown here:

```
covariance(X, Y) = [SUM (x - xbar)*(y-ybar)]/n
```

Notice that you can reverse the order of the product of terms (multiplication is commutative), and therefore the covariance matrix C is a symmetric matrix:

```
covariance(X, Y) = covariance(Y,X)
```

Suppose that a CSV file contains four numeric features, all of which have been scaled appropriately, and let's call them x1, x2, x3, and x4. Then the covariance matrix C is a 4x4 square matrix that is defined as follows:

```
cov(x1,x1) cov(x1,x2) cov(x1,x3) cov(x1,x4)
cov(x2,x1) cov(x2,x2) cov(x2,x3) cov(x2,x4)
cov(x3,x1) cov(x3,x2) cov(x3,x3) cov(x3,x4)
cov(x4,x1) cov(x4,x2) cov(x4,x3) cov(x4,x4)
```

Note that the following is true for the diagonal entries in the preceding covariance matrix C:

```
var(x1,x1) = cov(x1,x1)
var(x2,x2) = cov(x2,x2)
var(x3,x3) = cov(x3,x3)
var(x4,x4) = cov(x4,x4)
```

In addition, C is a symmetric matrix, which is to say that the transpose of matrix C (rows become columns and columns become rows) is identical to the matrix C. The latter is true because (as you saw in the previous section) `cov(x,y) = cov(y,x)` for any feature x and any feature y.

## Covariance Matrix: An Example

Suppose we have the two-column matrix A defined as follows:

```
      x  y
A = | 1  1 | <= 6x2 matrix
    | 2  1 |
    | 3  2 |
    | 4  2 |
    | 5  3 |
    | 6  3 |
```

The mean `x_bar` of column `x` is `(1+2+3+4+5+6)/6 = 3.5`, and the mean `y_bar` of column `y` is `(1+1+2+2+3+3)/6 = 2`. Now subtract `x_bar` from column `x` and subtract `y_bar` from column `y` and we get matrix `B`, as shown here:

```
B = | -2.5 -1 | <= 6x2 matrix
    | -1.5 -1 |
```

```
| -0.5  0 |
|  0.5  0 |
|  1.5  1 |
|  2.5  1 |
```

Let `Bt` indicate the transpose of the matrix `B`, which means that `Bt` is a 2 x 6 matrix, as shown here:

```
Bt = |-2.5 -1.5 -0.5 0.5, 1.5, 2.5|
     |-1   -1    0   0    1    1  |
```

The covariance matrix `C` is the product of `Bt` and `B`, as shown here:

```
C = Bt * B = | 15.25 4 |
             |  4    8 |
```

Note that if the units of measure of features `x` and `y` do not have a similar scale, then the covariance matrix is adversely affected. In this case, the solution is simple: use the correlation matrix, which is defined in the next section.

## The Correlation Matrix

As you learned in the preceding section, if the units of measure of features `x` and `y` do not have a similar scale, then the covariance matrix is adversely affected. The solution involves the *correlation matrix*, which equals the covariance values `cov(x,y)` divided by the standard deviation `stdx` and `stdy` of `x` and `y`, respectively, as shown here:

```
corr(x,y) = cov(x,y)/[stdx * stdy]
```

The correlation matrix no longer has units of measure, and we can use this matrix to find the eigenvalues and eigenvectors.

## Eigenvalues and Eigenvectors

According to a well-known theorem in mathematics (which you can find online), the eigenvalues of a symmetric matrix are real numbers. Consequently, the eigenvectors of C are vectors in a Euclidean vector space (not a complex vector space).

In case you're wondering, a nonzero vector x' is an eigenvector of the matrix C if there is a nonzero scalar lambda such that `C*x' = lambda * x'`.

Now suppose that the eigenvalues of `C` are `b1`, `b2`, `b3`, and `b4`, in decreasing numeric order from left to right, and that the corresponding eigenvectors of `C` are the vectors `w1`, `w2`, `w3`, and `w4`. These four vectors form the principal components of the new matrix.

Now that you understand how to calculate the covariance matrix and the correlation matrix, you are ready for an example of calculating eigenvalues and eigenvectors, which are the topic of the next section.

## Calculating Eigenvectors: A Simple Example

As a simple illustration of calculating eigenvalues and eigenvectors, suppose that the square matrix C is defined as follows:

```
C = | 1  3 |
    | 3  1 |
```

Let I denote the 2x2 identity matrix, and let b' be an eigenvalue of `C`, which means that there is an eigenvector x' such that:

```
C*x' = b' * x', or
(C-b*I)x' = 0 (the right side is a 2x1 vector)
```

Since x' is nonzero, that means the following is true (where `det` refers to the *determinant* of a matrix):

```
det(C-b*I) = det |1-b 3  | = (1-b)*(1-b)-9 = 0
                 |3   1-b|
```

We can expand the quadratic equation in the preceding line to get:

```
det(C-b*I) = (1-b)*(1-b) - 9
           = 1 - 2*b + b*b - 9
           = -8 - 2*b + b*b
           = b*b - 2*b - 8
```

Use the quadratic formula (or perform factorization by visual inspection) to determine that the solution for `det(C-b*I) = 0` is `b = -2` or `b = 4`. Next, substitute `b = -2` into `(C-b*I) x' = 0` and we get the following result:

```
|1-(-2) 3      | |x1| = |0|
|3      1-(-2)| |x2|   |0|
```

The preceding reduces to the following identical equations:

```
3*x1 + 3*x2 = 0
3*x1 + 3*x2 = 0
```

The general solution is `x1 = -x2`, and we can choose any nonzero value for x2, so let's set `x2 = 1` (any other nonzero value will do just fine), which yields `x1 = -1`. Therefore, the eigenvector [-1, 1] is associated with the eigenvalue -2. In a similar fashion, if x' is an eigenvector whose eigenvalue is 4, then [1,1] is an eigenvector.

Notice that the eigenvectors [-1, 1] and [1,1] are orthogonal because their inner product is zero, as shown here:

```
[-1,1] * [1,1] = (-1)*1 + (1)*1 = 0
```

*In fact, the set of eigenvectors of a square matrix (whose eigenvalues are real) are always pairwise orthogonal, regardless of the dimensionality of the matrix.*

## Gauss Jordan Elimination (Optional)

This simple technique enables you to find the solution to systems of linear equations "in place," which involves a sequence of arithmetic operations to transform a given matrix to an identity matrix.

The following example combines the Gauss-Jordan elimination technique (which finds the solution to a set of linear equations) with the "bookkeeper's method," which determines the inverse of an invertible matrix (its determinant is nonzero).

This technique involves two adjacent matrices: the left-side matrix is the initial matrix, and the right-side matrix is an identity matrix. Next, perform various linear operations on the left-side matrix to reduce it to an identity matrix: the matrix on the right side equals its inverse. For example, consider the following pair of linear equations whose solution is x = 1 and y = 2:

```
2*x + 2*y = 6
4*x - 1*y = 2
```

Step 1: create a 2x2 matrix with the coefficients of x in column one and the coefficients of y in column two, followed by the 2x2 identity matrix, and finally a column from the numbers on the right of the equals sign:

```
| 2  2 | 1 0 | 6|
| 4 -1 | 0 1 | 2|
```

Step 2: add (-2) times the first row to the second row:

```
| 2  2  | 1  0 |6  |
| 0  -5 | -2 1 |-10|
```

Step 3: divide the second row by 5:

```
| 2  2  | 1      0 |6    |
| 0  -1 | -2/5 1/5 |-10/5|
```

Step 4: add 2 times the second row to the first row:

```
| 2  0  |  1/5 2/5 |2 |
| 0  -1 | -2/5 1/5 |-2|
```

Step 5: divide the first row by 2:

```
| 1  0  | -2/10 2/10 |1 |
| 0  -1 | -2/5   1/5 |-2|
```

Step 6: multiply the second row by (-1):

```
| 1  0  | -2/10 2/10 |1|
| 0  1  |  2/5  -1/5 |2|
```

As you can see, the left-side matrix is the 2x2 identity matrix, the right-side matrix is the inverse of the original matrix, and the rightmost column is the solution to the original pair of linear equations (x=1 and y=2).

## PCA (PRINCIPAL COMPONENT ANALYSIS)

This section discusses PCA because it's a very popular technique that you will encounter frequently. Other techniques (such as SVD, also discussed later) are more efficient than PCA, so it's worthwhile to learn other dimensionality reduction techniques as well.

PCA is an unsupervised linear dimensionality reduction technique for transforming the feature space of a dataset into another feature space that has smaller dimensionality. Since PCA involves a correlation matrix, which in turn involves variance, PCA is well-suited for features that contain continuous data. Although you can use PCA for discrete data (such as binary data), the results are less meaningful. A good discussion of PCA and discrete versus continuous data is here:

*https://stackoverflow.com/questions/40795141/pca-for-categorical-features*

In particular, PCA involves linear algebra in order to calculate eigenvalues and eigenvectors to linearly transform a feature space into another feature space that has lower dimensionality. PCA constructs so-called "components," each of which is a linear combination of features from the initial dataset. Moreover, the components are pair-wise orthogonal, and they are the columns of a matrix whereby the leftmost column is the most important component, and the rightmost column is the least important component. Keep in mind the following points regarding the `PCA` algorithm:

- PCA is a variance-based algorithm
- PCA creates variables that are linear combinations of the original variables
- The new variables are all pair-wise orthogonal
- PCA can be a useful preprocessing step before clustering
- PCA is generally preferred for data reduction

`PCA` can be useful for variables that are strongly correlated. If most of the coefficients in the correlation matrix are smaller than 0.3, `PCA` is not helpful. `PCA` provides some advantages: less computation time for training a model (for example, using only five features instead of 100 features), a simpler model, and the ability to render the data visually when two or three features are selected. Here is a key point regarding `PCA`:

*PCA calculates the eigenvalues and the eigenvectors of the covariance (or correlation) matrix C.*

If you have four or five components, you won't be able to display them visually, but you could select subsets of three components for visualization, and perhaps gain some additional insight into the dataset.

The `PCA` algorithm involves the following sequence of steps:

1. Calculate the correlation matrix (from the covariance matrix) C of a dataset.
2. Find the eigenvalues of C.
3. Find the eigenvectors of C.
4. Construct a new matrix that comprises the eigenvectors.

The covariance matrix and correlation matrix were explained in a previous section. You also saw the definition of eigenvalues and eigenvectors, along with an example of calculating eigenvalues and eigenvectors.

The eigenvectors are treated as column vectors that are placed adjacent to each other in decreasing order (from left to right) with respect to their associated eigenvectors.

PCA uses variance as a measure of information: the higher the variance, the more important the component. In fact, just to jump ahead slightly: PCA determines the eigenvalues and eigenvectors of a covariance matrix (discussed in a previous section), and constructs a new matrix whose columns are eigenvectors, ordered from left to right based on the maximum eigenvalue in the leftmost column, decreasing until the rightmost eigenvector also has the smallest eigenvalue.

Alternatively, there is an interesting theorem in linear algebra: if C is a symmetric matrix, then there is a diagonal matrix D and an orthogonal matrix P (the columns are pair-wise orthogonal, which means their pair-wise inner product is zero), such that the following holds:

```
C = P * D * Pt (where Pt is the transpose of matrix P)
```

In fact, the diagonal values of D are eigenvalues, and the columns of P are the corresponding eigenvectors of the matrix C.

Fortunately, we can use NumPy and Pandas to calculate the mean, standard deviation, covariance matrix, correlation matrix, as well as the matrices D and P in order to determine the eigenvalues and eigenvectors.

As an interesting point: any positive definite square matrix has real-valued eigenvectors, which also applies to the covariance matrix C because it is a real-valued symmetric matrix.

## The New Matrix of Eigenvectors

The previous section described how the matrices D and P are determined. The leftmost eigenvector of D has the largest eigenvalue, the next eigenvector has the second-largest eigenvalue, and so forth. This fact is very convenient: the eigenvector with the highest eigenvalue is the principal component of the dataset. The eigenvector with the second-highest eigenvalue is the second principal component, and so forth. You specify the number of principal components that you want via the n_components hyperparameter in the PCA class of Sklearn.

As a simple and minimalistic example, consider the following code block that uses PCA for a (somewhat contrived) dataset:

```
import numpy as np
from Sklearn.decomposition import PCA
data = np.array([[-1,-1], [-2,-1], [-3,-2], [1,1], [2,1], [3,2]])
pca = PCA(n_components=2)
pca.fit(X)
```

Note a trade-off here: we greatly reduce the number of components, which reduces the computation time and the complexity of the model, but we also lose some accuracy. However, if the unselected eigenvalues are small, we lose only a small amount of accuracy.

Now let's define the variables NM, NMt, PC, SD, and SDt as follows:

- NM denotes the matrix with the new principal components
- NMt is the transpose of NM
- PC is the matrix of the subset of selected principal components
- SD is the matrix of scaled data from the original dataset
- SDt is the transpose of SD

Then the matrix NM can be expressed via the following formula:

```
NM = PCt * SDt
```

Although PCA is a very nice technique for dimensionality reduction, keep in mind the following limitations of PCA:

- less suitable for data with nonlinear relationships
- less suitable for special classification problems

A related algorithm is called Kernel PCA, which is an extension of PCA that introduces a nonlinear transformation so you can still use the PCA approach.

## DIMENSIONALITY REDUCTION

The previous section described various types of feature selection techniques, whereas dimensionality reduction involves a transformation of the data (such as PCA) or a projection of the data. Some of the well-known algorithms that perform dimensionality reduction are listed here:

- principal component analysis (PCA)
- kernel PCA
- graph-based kernel PCA
- SVD (singular value decomposition)
- non-negative matrix factorization (NMF)
- linear discriminant analysis (LDA)
- generalized discriminant analysis (GDA)
- isomap embedding
- locally linear embedding
- modified locally linear embedding
- VAEs (variational autoencoders)

VAEs (the last item in the preceding list) are deep learning models that have a number of use cases. Although VAEs are beyond the scope of this book, a comparison of PCA and VAEs is here:

*https://towardsdatascience.com/dimensionality-reduction-pca-versus-autoencoders-338fcaf3297d*

The following algorithms combine feature extraction and dimensionality reduction (some of which belong to both lists):

- PCA (principal component analysis)
- LDA (linear discriminant analysis)
- CCA (canonical correlation analysis)
- NMF (non-negative matrix factorization)

These algorithms can be used during a preprocessing step before using clustering or some other algorithm (such as kNN) on a dataset.

One other group of algorithms involves methods based on projections, which includes t-distributed stochastic neighbor embedding (t-SNE) as well as UMAP.

## DIMENSIONALITY REDUCTION TECHNIQUES

In simplified terms, dimensionality reduction refers to reducing a high-dimensional vector space to a lower dimensional vector space. There are several techniques that perform dimensionality reduction (such as PCA in the previous section), as listed here:

- PCA (principal component analysis)
- SVD (singular value decomposition)
- LLE (locally linear embedding)
- UMAP
- t-SNE
- PHATE

The previous section discussed PCA because PCA is a common algorithm for dimensionality reduction in machine learning. The other algorithms in the preceding list are discussed in a high-level fashion in the following subsections.

Dimensionality reduction techniques belong to different categories, such as matrix factorization (PCA and SVD), model-based reduction (LDA), and manifold learning (SOM and t-SNE).

### The Curse of Dimensionality

This term refers to the following phenomenon: as you increase the number of dimensions in a dataset, the data points in the dataset tend to be farther apart, which in turn increases the possibility of overfitting. In a perfect world, the data points in a dataset are evenly distributed (how wonderful this would be!), but in reality, different features have different distributions. One feature might be almost constant, a second feature might have a Gaussian distribution, a third feature might be bimodal, and so forth.

Nevertheless, many dimensionality reduction algorithms involve projecting data points in a dataset to a lower-dimensional space (the other technique is discussed later).

### What Are Manifolds (Optional)?

Although this is a concept in topology, which might seem wildly out of place in this book, it's a concept that is useful for understanding the second popular dimensionality reduction technique called *manifold learning*.

As a simple example, walking on a sidewalk or driving a car feels like movement in two dimensions (provided that you are not traversing a hill). In reality, you are moving along a sphere-like object (the Earth), which means that you are moving in three dimensions as you walk on a non-planar surface. However, the curvative of the non-planar surface so small that you don't even notice its effect, which is why this movement feels as though it's happening along a plane.

In essence, small regions on a three-dimensional sphere are locally "similar" to a two-dimensional plane (as described in the preceding paragraph). In general, a manifold is a topological space that locally resembles Euclidean space for points that are "close" together. In mathematical terms, given any point of an n-dimensional manifold, there is a neighborhood of that point that is homeomorphic to Euclidean space of the same dimensionality.

Some simple examples: lines and circles are one-manifolds but a "figure eight" is not a one-manifold. Two-dimensional manifolds (also called surfaces) include planes, spheres, and the torus ("donut").

## SVD (Singular Value Decomposition)

SVD is a technique in linear algebra that factors any mxn matrix A into the product of three matrices U, S, and V, as shown here:

```
A = U*S*Vt
```

The preceding formula is true for any matrix A (square, rectangular, invertible, or non-invertible), and the matrices U, S, and V are as follows:

- U: (mxm) is orthogonal (U*Ut = I) with left singular vectors
- V: (nxn) is orthogonal (V*Vt = I) with right singular vectors
- S: (mxn) is diagonal with "singular values"

Recall that if B, C, and D are compatible matrices, then the following transpose rules are true:

```
(B*C)t = Ct * Bt
(B*C*D)t = Dt * Ct * Bt
```

Now we can determine the nature of the matrices U and V by computing At*A and A*At, as follows:

```
At*A = (V*S*Ut)*(U*S*Vt) = V*S^2*Vt (because Ut*U = I)
```

In fact, V is the matrix of eigenvectors for At*A, and S^2 is the matrix of eigenvalues for At*A. Now let's compute A*At, as follows:

```
A*At = (U*S*Vt)*(V*S*Ut) = U*S^2*Ut (because Vt*V = I)
```

Now U is the matrix of eigenvectors for A*At, and S^2 is the matrix of eigenvalues for At*A. Since S^2 is a diagonal matrix of eigenvalues, the diagonal matrix S consists of the square root of the eigenvalues.

Keep in mind that there are some scalability and performance-related issues regarding SVD, as follows:

- complexity is O(m*n^2) when n < m (n x m matrix)
- not scalable for millions of words/documents
- difficult to incorporate new words/documents

In case you're wondering, one application of SVD is the decomposition of the ratings matrix in recommender systems into a product of three matrices. Another application of SVD pertains to gene expression, which is discussed in the following link:

*http://homepages.math.uic.edu/~friedlan/svdwvumerge.pdf*

There are several variants of SVD that might be of interest, and perhaps better suited to your particular needs. For example, Truncated SVD reduces dimensionality by selecting

only the largest singular values, keeping the first t columns of U and V. Details for Truncated SVD as well as Thin SVD and Compact SVD are provided here: *https://en.wikipedia.org/wiki/Singular_value_decomposition*

Another variant of SVD is SVDpp (sometimes called SVD++), which is described here:

*https://surprise.readthedocs.io/en/stable/matrix_factorization.html*

Finally, HybridSVD, PureSVD, SVDpp, and timeSVDpp are additional variations of SVD, and you can perform an online search for additional information.

## LLE (Locally Linear Embedding)

LLE is a nonlinear dimensionality reduction technique that is based on the following two tasks:

1. compute the linear distance of a datapoint to its nearest neighbor, and then
2. attempt to determine a lower-dimensional representation of the data points such that the local relationships are preserved.

Thus, the intuition of LLE is similar to the intuition of t-SNE. An example of using LocallyLinearEmbedding with Sklearn is here:

*https://scikit-learn.org/stable/modules/generated/Sklearn.manifold.LocallyLinearEmbedding.html*

## UMAP

UMAP is an acronym for uniform manifold approximation and projection, and it's a nonlinear dimensionality reduction technique. Although it is similar to the t-SNE algorithm (discussed in the next section), UMAP makes the assumption that the data is uniformly distributed on a locally connected Riemannian manifold and that the Riemannian metric is locally constant or approximately locally constant. A detailed discussion regarding Riemannian metrics is here:

*https://arxiv.org/pdf/1212.2474.pdf*

## t-SNE ("tee-snee")

The t-SNE (t-distributed stochastic neighbor embedding) algorithm is a nonlinear dimensionality reduction technique in machine learning for visualization.

The goal of t-SNE is to perform the following:

1. Map similar objects to nearby points.
2. Map dissimilar objects to more distant points.

The implementation of t-SNE involves two probability distributions: one for pairs of high-dimensional objects that perform #1, and another for pairs of low-dimensional objects that perform #2. The next step involves minimizing the KL (Kullback-Leibler) divergence for these two probability distributions.

Note that PCA is a much older technique (1930s) whereas t-SNE was developed more recently (2008). PCA performs linear dimension reduction that (a) maximizes variance and (b) preserves

large pairwise distances. As a result, different data points will be moved farther apart in PCA, which might not work as well with nonlinear data.

By contrast, t-SNE preserves small pairwise distances by means of an algorithm that calculates a similarity measure between data points in the high-dimensional space as well as in the low-dimensional space. In addition, t-SNE uses a cost function in order to optimize similarity measures.

## PHATE

PHATE is an acronym for "potential of heat-diffusion affinity-based transition embedding." PHATE is a visualization and dimensionality reduction technique that does resemble t-SNE: both are sensitive to local and global relationships and attempt to preserve relationships among close objects and distant objects.

Note that this technique shows up in biological data, which is not discussed in this book. However, more information about PHATE is available here:

*https://www.biorxiv.org/content/10.1101/120378v1.full.pdf*

## LINEAR VERSUS NONLINEAR REDUCTION TECHNIQUES

One detail that has been glossed over is the difference between linear dimensionality reduction versus nonlinear dimensionality reduction algorithms. Before discussing those two types of algorithms, a mathematical definition is required.

Suppose that T is a mapping from vector space U to the vector space V (which can have different dimensions), denoted as T:U->V. Then T is a *linear transformation* if the following are both true for any vectors u1 and u2 in U and any real number a:

1. `T(u1+u2) = T(u1) + T(u2)`
2. `T(a*u)   = a*T(u)`

For example, rotating a vector in the plane or scaling a vector in the plane is a linear transformation: shifting a vector in the plane is not a linear transformation (it's an *affine transformation*).

If the dimensions of U and V are the same and equal n, then a linear transformation T can be represented as a nxn invertible matrix (i.e, it has an inverse). If the dimension of V is lower than the dimension of U, then the linear transformation T involves a *projection*. If the dimension of V is higher than the dimension of U, then the transformation T involves an *embedding*.

For example, if T doubles the size of the components of vectors in two-dimensional Euclidean space, then T can be represented by the following matrix:

```
| 2 0 |
| 0 2 |
```

If T halves the first component and triples the second component of vectors in two-dimensional space, then T can be represented by the following matrix:

```
| 0.5 0 |
| 0   3 |
```

If T maps the vector of the form (x,y,z) to a vector of the form (x,y,0), then T is a projection, and T can be represented by the following matrix:

```
| 1 0 0 |
| 0 1 0 |
| 0 0 0 |
```

Based on the definition of a linear transformation that you saw in the beginning of this section, PCA is a *linear* dimensionality reduction algorithm, whereas kernel PCA is a *nonlinear* dimensionality reduction algorithm.

Linear reduction involves computing a linear combination of features or performing a linear transformation. An example of the latter transforms a spanning set of vectors by a non-singular matrix. Specifically, suppose that the set B of vectors {v1, v2, . . . , vn} are orthogonal and they form a basis for an n-dimensional vector space V. In addition, suppose that M is a nxn non-singular matrix (its determinant is non-zero). Then {Mv1, Mv2, . . . , Mvn} is a linear transformation of the basis B that also spans the vector space V.

By contrast, nonlinear (NL) dimensionality reduction techniques perform nonlinear transformations. However, in many cases, NL dimensionality reduction techniques are related to linear methods, and they tend to belong to one of two groups:

1. NL techniques that provide high-to-low mapping (or vice versa)
2. NL techniques that provide visualization

Mapping methods are used to perform an initial feature extraction step, after which machine learning algorithms can perform pattern recognition. On the other hand, NL techniques that provide visualization perform calculations involving the distance between objects.

## TYPES OF DISTANCE METRICS

Nonlinear dimensionality reduction techniques can also have different distance metrics. For example, linear reduction techniques can use the Euclidean distance metric (based on the Pythagorean theorem). However, you need to use a different distance metric to measure the distance between two points on a sphere (or some other curved surface). In the case of NLP, the *cosine similarity* metric is used to measure the distance between word embeddings (which are vectors of floating-point numbers that represent words or tokens).

Distance metrics are used for measuring physical distances, and some well-known distance metrics are listed here:

- Euclidean distance
- Manhattan distance
- Chebyshev distance

The Euclidean algorithm also obeys the *triangle inequality*, which states that for any triangle in the Euclidean plane, the sum of the lengths of any pair of sides must be greater than the length of the third side.

In spherical geometry, you can define the distance between two points as the arc of a great circle that passes through the two points (always selecting the smaller of the two arcs when they are different).

In addition to physical metrics, there are algorithms that implement the concept of "edit distance" (the distance between strings), as listed here:

- Hamming distance
- Jaro–Winkler distance
- Lee distance
- Levenshtein distance
- Mahalanobis distance metric
- Wasserstein metric

The *Mahalanobis* metric is based on an interesting idea: given a point P and a probability distribution D, this metric measures the number of standard deviations that separate point P from distribution D. More information about Mahalanobis is here:

*https://en.wikipedia.org/wiki/Mahalanobis_distance*

In the branch of mathematics called topology, a metric space is a set for which distances between all members of the set are defined. Various metrics are available (such as the *Hausdorff* metric), depending on the type of topology.

The *Wasserstein* metric measures the distance between two probability distributions over a metric space X. This metric is also called the "earth mover's metric" for the following reason: given two-unit piles of dirt, it's the measure of the minimum cost of moving one pile on top of the other pile.

KL Divergence bears some superficial resemblance to the Wasserstein metric. However, there are some important differences between them. Specifically, the Wasserstein metric has the following properties:

1. It is a metric.
2. It is symmetric.
3. It satisfies the triangle inequality.

On the other hand, KL divergence has the following properties:

1. It is not a metric (it's a divergence).
2. It is not symmetric: KL(P,Q) != KL(Q,P).
3. It does not satisfy the triangle inequality.

Note that JS (Jenson-Shannon) divergence (which is based on KL divergence) is a true metric, which would enable a more meaningful comparison with other metrics (such as the Wasserstein metric).

## WELL-KNOWN DISTANCE METRICS

There are several similarity metrics available, such as item similarity metrics, Jaccard (user-based) similarity, and cosine similarity (which is used to compare vectors of numbers). The following subsections introduce you to these similarity metrics.

Another well-known distance metric is the so-called "taxicab" metric, which is also called the Manhattan distance metric. Given two points A and B in a rectangular grid, the taxicab metric calculates the distance between two points by counting the number of "blocks" that must be traversed in order to reach B from A (the other direction has the same taxicab metric value). For example, if you need to travel two blocks north and then three blocks east in a rectangular grid, then the Manhattan distance is 5.

In fact, there are various other metrics available, which you can learn about by searching Wikipedia. In the case of `NLP`, the most frequently used distance metric is calculated via the cosine similarity of two vectors, and it's derived from the formula for the inner ("dot") product of two vectors.

### Pearson Correlation Coefficient

*Pearson similarity* is the Pearson coefficient between two vectors. Given random variables X and Y, and the following terms:

```
std(X)   = standard deviation of X
std(Y)   = standard deviation of Y
cov(X,Y) = covariance of X and Y
```

Then the Pearson correlation coefficient `rho(X,Y)` is defined as follows:

```
                cov(X,Y)
rho(X,Y) =  -------------
            std(X)*std(Y)
```

Keep in mind that the Pearson coefficient is limited to items of the same type. More information about the Pearson correlation coefficient is here:

*https://en.wikipedia.org/wiki/Pearson_correlation_coefficient*

### Jaccard Index (or Similarity)

The *Jaccard similarity* is based on the number of users which have rated item A and B divided by the number of users who have rated either A or B. Jaccard similarity is based on unique words in a sentence and is unaffected by duplicates, whereas cosine similarity is based on the length of all word vectors (which changes when duplicates are added). The choice between cosine similarity and Jaccard similarity depends on whether or not word duplicates are important.

The following `Python` method illustrates how to compute the Jaccard similarity of two sentences:

```
def get_jaccard_sim(str1, str2):
  set1 = set(str1.split())
```

```
set2 = set(str2.split())
set3 = set1.intersection(set2)
# (size of intersection) / (size of union):
return float(len(set3)) / (len(set1) + len(set2) - len(set3))
```

Jaccard similarity can be used in situations involving Boolean values, such as product purchases (true/false), instead of numeric values. More information is available here: *https://en.wikipedia.org/wiki/Jaccard_index*

### Local Sensitivity Hashing (Optional)

If you are familiar with hash algorithms, you know that they are algorithms that create a hash table that associates items with a value. The advantage of hash tables is that the lookup time to determine whether or not an item exists in the hash table is constant. Of course, it's possible for two items to "collide," which means that they both occupy the same bucket in the hash table. In this case, a bucket can consist of a list of items that can be searched in more or less constant time. If there are too many items in the same bucket, then a different hashing function can be selected to reduce the number of collisions. The goal of a hash table is to minimize the number of collisions.

The *local sensitivity hashing* (LSH) algorithm hashes similar input items into the same "buckets." In fact, the goal of LSH is to maximize the number of collisions, whereas traditional hashing algorithms attempt to minimize the number of collisions.

Since similar items end up in the same buckets, LSH is useful for data clustering and nearest neighbor searches. Moreover, LSH is a dimensionality reduction technique that places data points of high dimensionality closer together in a lower-dimensional space, while simultaneously preserving the relative distances between those data points.

More details about LSH are here:

*https://en.wikipedia.org/wiki/Locality-sensitive_hashing*

## WHAT IS SKLEARN?

Before we discuss any code samples, please keep in mind that `Sklearn` is an immensely useful `Python` library that supports a huge number of machine learning algorithms. In particular, `Sklearn` supports many classification algorithms, such as logistic regression, naive Bayes, decision trees, random forests, and SVMs (support vector machines). Although entire books are available that are dedicated to `Sklearn`, this chapter contains only a few pages of `Sklearn` material.

One-hot encoding and label encoding and `Sklearn` `SimpleImputer` class are found here:

*https://www.kdnuggets.com/2020/07/easy-guide-data-preprocessing-python.html*

The following link contains a list of some of the best new features in version 0.25 of `Sklearn`:

*https://towardsdatascience.com/the-10-best-new-features-in-scikit-learn-0-24-f45e49b6741b*

If you decide that you want to acquire a deep level of knowledge about `Sklearn`, navigate to the Web pages that contain very detailed documentation for `Sklearn`. Moreover, if you have "how to" questions involving `Sklearn`, you can almost always find suitable answers on Stack Overflow.

In addition to support for machine learning algorithms, Sklearn provides various built-in datasets that you can access with literally one line of code. In fact, Listing 7.1 displays the contents of sklearn_iris1.py that illustrates how you can easily load the Iris dataset into a Pandas data frame.

***LISTING 7.1: sklearn_iris1.py***

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris()

print("=> iris keys:")
for key in iris.keys():
  print(key)
print()

#print("iris dimensions:")
#print(iris.shape)
#print()

print("=> iris feature names:")
for feature in iris.feature_names:
  print(feature)
print()

X = iris.data[:, [2, 3]]
y = iris.target
print('=> Class labels:', np.unique(y))
print()

print("=> target:")
print(iris.target)
print()

print("=> all data:")
print(iris.data)
```

Listing 7.1 starts with several import statements and then initializes the variable iris with the Iris dataset. Next, a loop displays the keys in the dataset, followed by another loop that displays the feature names.

The next portion of Listing 7.1 initializes the variable X with the feature values in columns 2 and 3, and then initializes the variable y with the values of the target column. Now launch the code in Listing 7.1 and you will see the following output (truncated to save space):

```
=> iris keys:
data
target
target_names
DESCR
```

```
feature_names
filename

=> iris feature names:
sepal length (cm)
sepal width (cm)
petal length (cm)
petal width (cm)

=> Class labels: [0 1 2]

=> target:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2]

=> all data:
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 // details omitted for brevity
 [6.5 3.  5.2 2. ]
 [6.2 3.4 5.4 2.3]
 [5.9 3.  5.1 1.8]]
```

## Sklearn, Pandas, and the IRIS Dataset

Listing 7.2 displays the contents of `pandas_iris1.py` that illustrates how to load the contents of the `Iris` dataset (from `Sklearn`) into a `Pandas` data frame.

***LISTING 7.2: pandas_iris1.py***

```
import numpy as np
import pandas as pd
from Sklearn.datasets import load_iris

iris = load_iris()

print("=> IRIS feature names:")
for feature in iris.feature_names:
  print(feature)
print()

# Create a dataframe with the feature variables
df = pd.DataFrame(iris.data, columns=iris.feature_names)

print("=> number of rows:")
print(len(df))
print()
```

```
print("=> number of columns:")
print(len(df.columns))
print()

print("=> number of rows and columns:")
print(df.shape)
print()

print("=> number of elements:")
print(df.size)
print()

print("=> IRIS details:")
print(df.info())
print()

print("=> top five rows:")
print(df.head())
print()

X = iris.data[:, [2, 3]]
y = iris.target
print('=> Class labels:', np.unique(y))
```

Listing 7.2 starts with several `import` statements and then initializes the variable `iris` with the `Iris` dataset. Next, a loop displays the feature names. The next code snippet initializes the variable `df` as a `Pandas` data frame that contains the data from the `Iris` dataset.

The next block of code invokes some attributes and methods of a `Pandas` data frame to display the number of rows, columns, and elements in the data frame, as well as the details of the `Iris` dataset, the first five rows, and the unique labels in the `Iris` dataset. Launch the code in Listing 7.2 and you will see the following output:

```
=> IRIS feature names:
sepal length (cm)
sepal width (cm)
petal length (cm)
petal width (cm)

=> number of rows:
150

=> number of columns:
4

=> number of rows and columns:
(150, 4)

=> number of elements:
600

=> IRIS details:
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 4 columns):
sepal length (cm)    150 non-null float64
sepal width (cm)     150 non-null float64
petal length (cm)    150 non-null float64
petal width (cm)     150 non-null float64
dtypes: float64(4)
memory usage: 4.8 KB
None

=> top five rows:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

=> Class labels: [0 1 2]
```

## Sklearn and Outlier Detection

Listing 7.3 displays the contents of `elliptic_envelope.py` that illustrates how to use the `EllipticEnvelope` class in `Sklearn` in order to detect outliers in a dataset.

***LISTING 7.3: elliptic_envelope.py***

```
import numpy as np

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.covariance import EllipticEnvelope

datapoints = [[-1,1], [1,1], [1,-1], [-1,-1], [2,2]]
columns = ["x", "y"]
index = [0,1,2,3,4]
df = pd.DataFrame(np.array(datapoints),columns=columns,index=index)

# display data points in a scatterplot:
plt.scatter(df["x"], df["y"], color = "r")
plt.grid()

data = df[['x', 'y']].values

# the contamination parameter defines the proportion of values
# that will be identified as outliers: values are in [0, 0.5]
```

```
model1 = EllipticEnvelope(contamination = 0.1)
model1.fit(data)

# new data for prediction:
pred_data = [[3,3], [1.5,1.5], [-2,-2], [-3,-3]]
new_data = np.array(pred_data)

#print("new_data[:,0]:", new_data[:,0])
#print("new_data[:,1]:", new_data[:,1])

# display new data points in a scatterplot:
plt.scatter(new_data[:,0], new_data[:,1], s=100, color = "b")

# predict on new data:
pred1 = model1.predict(new_data)

# negative values are outliers:
print("negative predicted values are outliers:")
print(pred1)

for i in range(new_data.shape[0]):
  print("new_data[",i,"] = ",new_data[i],"is an outlier:",pred1[i]==-1)

#plt.show()
```

Listing 7.3 starts with several `import` statements and then initializes the variable datapoint with an array of 2D points in order to define the `Pandas` data frame `df`.

The next section displays a scatterplot of the data points (drawn in red), and then initializes the variable `model1` as an instance of the `EllipticEnvelope` class. The `fit()` method of `model1` is invoked in order to train the model on the data points

Next, the variable `pred_data` is initialized with a different set of data points, and their values will be examined in order to determine if any of them are outliers. After the points in `pred_data` are added to the scatterplot (and drawn in blue), the `predict()` method of the variable `model1` is invoked with `pred_data` and the result is assigned to the variable `pred1`. Finally, a loop iterates through the contents of `pred1`, and a predicted value of -1 indicates an outlier. Now launch the code in Listing 7.3 and you will see the following output:

```
negative predicted values are outliers:
[-1  1 -1 -1]
new_data[ 0 ] =  [3. 3.] is an outlier: True
new_data[ 1 ] =  [1.5 1.5] is an outlier: False
new_data[ 2 ] =  [-2. -2.] is an outlier: True
```

Uncomment the last code snippet in Listing 7.3 and you will see the initial datapoint displayed in red and the test-related points in blue.

## WHAT IS BAYESIAN INFERENCE?

Bayesian inference is an important technique in statistics that involves statistical inference and Bayes' theorem to update the probability for a hypothesis as more information becomes

available. Bayesian inference is often called "Bayesian probability," and it's important in dynamic analysis of sequential data.

## Bayes' Theorem

Given two sets A and B, let's define the following numeric values (all of them are between 0 and 1):

```
P(A) = probability of being in set A
P(B) = probability of being in set B
P(Both) = probability of being in A intersect B
P(A|B) = probability of being in A (given you're in B)
P(B|A) = probability of being in B (given you're in A)
```

Then the following formulas are also true:

```
P(A|B) = P(Both)/P(B) (#1)
P(B|A) = P(Both)/P(A) (#2)
```

Multiply the preceding pair of equations by the term that appears in the denominator, and we get these equations:

```
P(B)*P(A|B) = P(Both) (#3)
P(A)*P(B|A) = P(Both) (#4)
```

Now set the left side of equations #3 and #4 equal to each another and that gives us this equation:

```
P(B)*P(A|B) = P(A)*P(B|A) (#5)
```

Divide both sides of #5 by P(B) and we get this well-known equation:

```
P(A|B) = P(A)*P(A|B)/P(B) (#6)
```

## Some Bayesian Terminology

In the previous section, we derived the following relationship:

```
P(h|d) = (P(d|h) * P(h)) / P(d)
```

Each of the four terms in the preceding equation has a name.

First, the *posterior probability* is `P(h|d)`, which is the probability of hypothesis `h` given the data `d`.

Second, `P(d|h)` is the probability of data `d` given that the hypothesis `h` was true.

Third, the *prior probability* of `h` is `P(h)`, which is the probability of hypothesis `h` being true (regardless of the data).

Finally, `P(d)` is the probability of the data (regardless of the hypothesis).

*We are interested in calculating the posterior probability of P(h|d) from the prior probability p(h) with P(D) and P(d|h).*

### What Is MAP?

The maximum a posteriori (MAP) hypothesis is the hypothesis with the highest probability, which is the maximum probable hypothesis. This can be written as follows:

```
MAP(h) = max(P(h|d))
```

or:

```
MAP(h) = max((P(d|h) * P(h)) / P(d))
```

or:

```
MAP(h) = max(P(d|h) * P(h))
```

### Why Use Bayes' Theorem?

Bayes' theorem describes the probability of an event based on the prior knowledge of the conditions that might be related to the event. If we know the conditional probability, we can use the Bayes rule to find out the reverse probabilities. The previous statement is the general representation of the Bayes rule.

## WHAT ARE VECTOR SPACES?

If you are already familiar with vector spaces, feel free to quickly skim through this section.

A *vector space* is a mathematical construct whose elements are *vectors* that typically consist of a tuple of numbers. A real-valued vector space contains vectors of real numbers, whereas a complex vector space contains vectors of complex numbers. The dimensionality of numeric vector spaces is the minimum number of linearly independent vectors. For example, the vector **u** = [2,3] in the Euclidean plane is a real-valued two-dimensional vector, whereas the vector **v** = [2+5i,4-7j] in the complex plane is a complex two-dimensional vector.

Every vector in a numeric vector space can be expressed as a linear combination of the vectors on a basis. A *spanning set* is a superset of a basis, which means that some vectors in the spanning set are a linear combination of the other vectors in the spanning set: those vectors are dependent vectors. If the dependent vectors are removed from a spanning set, the remaining vectors form a basis. As a side note: the vectors in a basis are not required to be unit vectors nor must they be orthogonal to each other (examples shown later).

The real number line can be viewed as a one-dimensional vector space. The vector **i** = [1,0] is a unit vector that forms a so-called basis for this vector space because any other vector is a multiple of the vector i.

The Euclidean plane can be viewed as a two-dimensional vector space. The unit vectors **i** = [1,0] and **j** = [0,1] are unit vectors that form a basis for this vector space because any other vector is a linear combination of the vectors **i** and **j**. Notice that the dot product of i and j is 0, which means that they are orthogonal. However, the vectors in the set {2∗**i**, 3∗**j**} are not unit vectors, and yet they form a basis for the Euclidean plane.

As another example, the vectors in the set {**i**+**j**, **i**} also form a basis, even though they are not orthogonal and one of them is not a unit vector. However, any basis can be transformed into any other basis for a given vector space, which means that any basis can be transformed into a basis that consists of orthonormal unit vectors (search online for proof of this statement).

In the three-dimensional Euclidean space, the set of vectors {**i**,**j**,**k**} form an orthonormal basis, where the unit vectors **i**, **j**, and **k** are [1,0,0], [0,1,0], and [0,0,1], respectively.

In general, an n-dimensional vector space V has a basic S that consists of n orthonormal vectors. If you create a matrix M whose rows are the vectors in a basis, then the inverse of matrix M exists because the rows are linearly independent.

In fact, a set of n linear equations in n unknowns has a unique solution if (and only if) the associated matrix M is invertible. Statistical packages contain one or more algorithms for solving systems of linear equations.

## SUMMARY

This chapter started with an introduction to covariance matrices and correlation matrices. Next, you learned about eigenvalues and eigenvectors, and how to calculate them using the Gauss Jordan Elimination algorithm.

Then you learned about the Principal Component Analysis (PCA) algorithm, which followed by other well-known techniques for dimensionality reduction techniques, such as SVD (Singular Value Decomposition), LLE (Locally Linear Embedding), UMAP, t-SNE, and PHATE.

In addition, you learned about distance metrics and some well-known distance metrics, such as Pearson Correlation Coefficient, Jaccard Index, and Local Sensitivity Hashing. Finally, you learned about the machine learning library Scikit-Learn, Bayesian Inference, and then an introduction to vector spaces.

APPENDIX

# WORKING WITH AWK

This appendix introduces you to the `awk` command, which is a highly versatile utility for manipulating data and restructuring datasets. In fact, this utility is so versatile that entire books have been written about the `awk` utility. Awk is essentially an entire programming language in a single command, which accepts standard input, gives standard output, and uses regular expressions and metacharacters in the same way other Unix commands do. This lets you plug it into other expressions and do almost anything, at the cost of adding complexity to a command string that may already be doing quite a lot already. It is almost always worthwhile to add a comment when using awk; it is so versatile that it won't be clear which of the many features you are using at a glance.

The first part of this appendix provides a very brief introduction of the `awk` command. You will learn about some built-in variables for `awk`, and also how to manipulate string variables using `awk`. Note that some of these string-related examples can also be handled using other `bash` commands.

The second part of this appendix shows you conditional logic, `while` loops, and `for` loops in `awk` in order to manipulate the rows and columns in datasets. This section also shows you how to delete lines and merge lines in datasets, and also how to print the contents of a file as a single line of text. You will see how to "join" lines and groups of lines in datasets.

The third section contains code samples that involve metacharacters and character sets in `awk` commands. You will also see how to use conditional logic in `awk` commands in order to determine whether or not to print a line of text.

The fourth section illustrates how to "split" a text string that contains multiple "." characters as a delimiter, followed by examples of `awk` to perform numeric calculations (such as addition, subtraction, multiplication, and division) in files containing numeric data. This section also shows you various numeric functions that are available in awk, and also how to print text in a fixed set of columns.

The fifth section explains how to align columns in a dataset and also how to align and merge columns in a dataset. You will see how to delete columns, how to select a subset of columns from a dataset, and how to work with multiline records in datasets. This section contains some one-line `awk` commands that can be useful for manipulating the contents of datasets.

The final section of this appendix has a pair of use cases involving nested quotes and date formats in structured data sets.

## THE AWK COMMAND

The awk (Aho, Weinberger, and Kernighan) command has a C-like syntax and you can use this utility to perform very complex operations on numbers and text strings.

As a side comment, there is also the gawk command that is GNU awk, as well as the nawk command that is "new" awk (neither command is discussed in this book). One advantage of nawk is that it allows you to set externally the value of an internal variable.

### Built-In Variables That Control awk

The awk command provides variables that you can change from their default values in order to control how awk performs operations. Examples of such variables (and their default values) include: FS (" "), RS ("\n"), OFS (" "), ORS ("\n") , SUBSEP, and IGNORECASE. The variables FS and RS specify the field separator and record separator, whereas the variables OFS and ORS specify the output field separator and the output record separator, respectively.

You can think of the field separators as delimiters/IFS we used earlier in other commands. The record separators behave in a way similar to how sed treats individual lines—for example sed can match or delete a range of lines instead of matching or deleting something that matches a regular expression (and the default awk record separator is the newline character, so by default awk and sed have a similar ability to manipulate and reference lines in a text file).

As a simple example, you can print a blank line after each line of a file by changing the ORS, from a default of one newline to two newlines, as shown here:

```
cat columns.txt | awk 'BEGIN { ORS ="\n\n" } ; { print $0 }'
```

Other built-in variables include FILENAME (the name of the file that awk is currently reading), FNR (the current record number in the current file), NF (the number of fields in the current input record), and NR (the number of input records awk has processed since the beginning of the program's execution).

Consult the online documentation for additional information regarding these (and other) arguments for the awk command.

### How Does the awk Command Work?

The awk command reads the input files one record at a time (by default, one record is one line). If a record matches a pattern, then an action is performed (otherwise no action is performed). If the search pattern is not given, then awk performs the given actions for each record of the input. The default behavior if no action is given is to print all the records that match the given pattern. Finally, empty braces without any action does nothing; that is, it will not perform the default printing operation. Note that each statement in actions should be delimited by semicolons.

In other to make the preceding paragraph more concrete, here are some simple examples involving text strings and the awk command (the results are displayed after each code snippet). The -F switch sets the field separator to whatever follows it, in this case a space. Switches will often provide a shortcut to an action that normally needs a command inside a BEGIN{} block:

```
x="a b c d e"
echo $x |awk -F" " '{print $1}'
```

```
a
echo $x |awk -F" " '{print NF}'
5
echo $x |awk -F" " '{print $0}'
a b c d e
echo $x |awk -F" " '{print $3, $1}'
c a
```

Now let's change the `FS` (record separator) to an empty string to calculate the length of a string, this time using the `BEGIN{}` syntax:

```
echo "abc" | awk 'BEGIN { FS = "" } ; { print NF }'
3
```

The following example illustrates several equivalent ways to specify `test.txt` as the input file for an `awk` command:

```
awk < test.txt '{ print $1 }'
awk '{ print $1 }' < test.txt
awk '{ print $1 }' test.txt
```

Yet another way is shown here (but as we've discussed earlier, it can be inefficient, so only do it if the cat is adding value in some way):

```
cat test.txt | awk '{ print $1 }'
```

This simple example of four ways to do the same task should illustrate why commenting awk calls of any complexity is almost always a good idea. The next person to look at your code may not know/remember the syntax you are using.

## ALIGNING TEXT WITH THE PRINTF STATEMENT

Since `awk` is a programming language inside a single command, it also has its own way of producing formatted output via the `printf` command.

Listing A.1 displays the contents of `columns2.txt` and Listing A.2 displays the contents of the shell script `AlignColumns1.sh` that shows you how to align the columns in a text file.

***LISTING A.1: columns2.txt***

```
one two
three four
one two three four
five six
one two three
four five
```

***LISTING A.2: AlignColumns1.sh***

```
awk '
{
   # left-align  $1 on a 10-char column
```

```
   # right-align $2 on a 10-char column
   # right-align $3 on a 10-char column
   # right-align $4 on a 10-char column
   printf("%-10s*%10s*%10s*%10s*\n", $1, $2, $3, $4)
}
' columns2.txt
```

Listing A.2 contains a `printf()` statement that displays the first four fields of each row in the file `columns2.txt`, where each field is ten characters wide.

The output from launching the code in Listing A.2 is here:

```
one        *       two*          *          *
three      *      four*          *          *
one        *       two*     three*      four*
five       *       six*          *          *
one        *       two*     three*          *
four       *       five*         *          *
```

Keep in mind that `printf` is reasonably powerful and as such has its own syntax, which is beyond the scope of this appendix. A search online can find the manual pages and also discussions of "how to do X with printf()."

## CONDITIONAL LOGIC AND CONTROL STATEMENTS

Like other programming languages, `awk` provides support for conditional logic (if/else) and control statements (for/while loops). `awk` is the only way to put conditional logic inside a piped command stream without creating, installing, and adding to the path a custom executable shell script. The following code block shows you how to use if/else logic:

```
echo "" | awk '
BEGIN { x = 10 }
{
  if (x % 2 == 0) }
      print "x is even"
  }
  else }
      print "x is odd"
  }
}
'
```

The preceding code block initializes the variable `x` with the value 10 and prints "`x is even`" if x is divisible by 2, otherwise it prints "`x is odd`".

### The while Statement

The following code block illustrates how to use a `while` loop in `awk`:

```
echo "" | awk '
{
  x = 0
  while(x < 4) {
    print "x:",x
```

```
    x = x + 1
  }
}
'
```

The preceding code block generates the following output:

```
x:0
x:1
x:2
x:3
```

The following code block illustrates how to use a `do while` loop in `awk`:

```
echo "" | awk '
{
  x = 0

  do {
    print "x:",x
    x = x + 1
  } while(x < 4)
}
'
```

The preceding code block generates the following output:

```
x:0
x:1
x:2
x:3
```

## A for loop in awk

Listing A.3 displays the contents of `Loop.sh` that illustrates how to print a list of numbers in a loop. Note that "i++" is another way of writing "i=i+1" `in awk` (and most c-derived languages).

***LISTING A.3: Loop.sh***

```
echo "" | awk '
BEGIN {}
{
  for(i=0; i<5; i++) {
    printf("%3d", i)
  }
}
END { print "\n" }
'
```

Listing A.3 contains a `for` loop that prints numbers on the same line via the `printf()` statement. Notice that a newline is printed only in the `END` block of the code. The output from Listing A.3 is here:

```
0 1 2 3 4
```

### A for loop with a break Statement

The following code block illustrates how to use a `break` statement in a `for` loop in `awk`:

```
echo "" | awk '
{
  for(x=1; x<4; x++) {
     print "x:",x
     if(x == 2) {
        break;
     }
  }
}
'
```

The preceding code block prints output only until the variable x has the value 2, after which the loop exits (because of the break inside the conditional logic). The following output is displayed:

```
x:1
```

### The next and continue Statements

The following code snippet illustrates how to use `next` and `continue` in a `for` loop in `awk`:

```
awk '
{
   /expression1/ { var1 = 5; next }
   /expression2/ { var2 = 7; next }
   /expression3/ { continue }
   // some other code block here
' somefile
```

When the current line matches `expression1`, then `var1` is assigned the value 5 and `awk` reads the next input line: hence, `expression2` and `expression3` will not be tested. If `expression1` does not match and `expression2` *does* match, then `var2` is assigned the value 7 and then `awk` will read the next input line. If only `expression3` results in a positive match, then `awk` skips the remaining block of code and processes the next input line.

## DELETING ALTERNATE LINES IN DATASETS

Listing A.4 displays the contents of `linepairs.csv` and Listing A.5 displays the contents of `deletelines.sh` that illustrates how to print alternating lines from the dataset `linepairs.csv` that have exactly two columns.

***LISTING A.4: linepairs.csv***

```
a,b,c,d
e,f,g,h
1,2,3,4
5,6,7,8
```

***LISTING A.5: deletelines.sh***

```
inputfile="linepairs.csv"
outputfile="linepairsdeleted.csv"
awk ' NR%2 {printf "%s", $0; print ""; next}' < $inputfile > $outputfile
```

Listing A.5 checks if the current record number `NR` is divisible by 2, in which case it prints the current line and skips the next line in the dataset. The output is redirected to the specified output file, the contents of which are here:

```
a,b,c,d
1,2,3,4
```

A slightly more common task involves merging consecutive lines, which is the topic of the next section.

## MERGING LINES IN DATASETS

Listing A.6 displays the contents of `columns.txt` and Listing A.7 displays the contents of `ColumnCount1.sh` that illustrates how to print the lines from the text file `columns.txt` that have exactly two columns.

***LISTING A.6: columns.txt***

```
one two three
one two
one two three four
one
one three
one four
```

***LISTING A.7: ColumnCount1.sh***

```
awk '
{
   if( NF == 2 ) { print $0 }
}
' columns.txt
```

Listing A.7 is straightforward: if the current record number is even, then the current line is printed (i.e., odd-numbered rows are skipped). The output from launching the code in Listing A.7 is here:

```
one two
one three
one four
```

If you want to display the lines that do *not* contain two columns, use the following code snippet:

```
if( NF != 2 ) { print $0 }
```

## Printing File Contents as a Single Line

The contents of `test4.txt` are here (note the blank lines):

```
abc

def

abc

abc
```

The following code snippet illustrates how to print the contents of `test4.txt` as a single line:

```
awk '{printf("%s", $0)}' test4.txt
```

The output of the preceding code snippet is here. See if you can tell what is happening before reading the explanation in the next paragraph:

```
Abcdefabcabc
```

Explanation: `%s` here is the record separator syntax for `printf`, and the end quote after it means the record separator is the empty field "". Our default record separator for `awk` is /n (newline); what the `printf` is doing is stripping out all the newlines. The blank rows will vanish entirely, as all they have is the newline, so the result is that any actual text will be merged together with nothing between them. Had we added a space between the `%s` and the ending quote, there would be a space between each character block, plus an extra space for each newline.

Notice how the following comment helps the comprehension of the code snippet:

```
# Merging all text into a single line by removing the newlines
awk '{printf("%s", $0)}' test4.txt
```

## Joining Groups of Lines in a Text File

Listing A.8 displays the contents of `digits.txt` and Listing A.9 displays the contents of `digits.sh` that "joins" three consecutive lines of text in the file `digits.txt`.

***LISTING A.8: digits.txt***

```
1
2
3
4
5
6
```

```
7
8
9
```

***LISTING A.9: digits.sh***

```
awk -F" " '{
  printf("%d",$0)
  if(NR % 3 == 0) { printf("\n") }
}' digits.txt
```

Listing A.9 prints three consecutive lines of text on the same line, after which a linefeed is printed. This has the effect of "joining" every three consecutive lines of text. The output from launching `digits.sh` is here:

```
123
456
789
```

## Joining Alternate Lines in a Text File

Listing A.10 displays the contents of `columns2.txt` and Listing A.11 displays the contents of `JoinLines.sh` that "joins" two consecutive lines of text in the file `columns2.txt`.

***LISTING A.10: columns2.txt***

```
one two
three four
one two three four
five six
one two three
four five
```

***LISTING A.11: JoinLines.sh***

```
awk '
{
   printf("%s",$0)
   if( $1 !~ /one/) { print " " }
}
' columns2.txt
```

The output from launching Listing A.11 is here:

```
one two three four
one two three four five six
one two three four five
```

Notice that the code in Listing A.11 depends on the presence of the string "one" as the first field in alternating lines of text—we are merging based on matching a simple pattern, instead of tying it to record combinations.

To merge each pair of lines instead of merging based on matching a pattern, use the modified code in Listing A.12.

***LISTING A.12: JoinLines2.sh***

```
awk '
BEGIN { count = 0 }
{
   printf("%s",$0)
   if( ++count % 2 == 0) { print " " }
} columns2.txt
```

Yet another way to "join" consecutive lines is shown in Listing A.13, where the input file and output file refer to files that you can populate with data. This is another example of an awk command that might be a puzzle if encountered in a program without a comment. It is doing exactly the same thing as Listing A.12, but its purpose is less obvious because of the more compact syntax.

***LISTING A.13: JoinLines2.sh***

```
inputfile="linepairs.csv"
outputfile="linepairsjoined.csv"
awk ' NR%2 {printf "%s,", $0; next;}1' < $inputfile > $outputfile
```

## MATCHING WITH META CHARACTERS AND CHARACTER SETS

If we can match a simple pattern, by now you probably expect that you can also match a regular expression, just as we did in `grep` and `sed`. Listing A.14 displays the contents of `Patterns1.sh` that uses metacharacters to match the beginning and the end of a line of text in the file `columns2.txt`.

***LISTING A.14: Patterns1.sh***

```
awk '
   /^f/    { print $1 }
   /two $/ { print $1 }
' columns2.txt
```

The output from launching Listing A.14 is here:

```
one
five
four
```

Listing A.15 displays the contents of `RemoveColumns.txt` with lines that contain a different number of columns.

***LISTING A.15: columns3.txt***

```
123 one two
456 three four
```

```
one two three four
five 123 six
one two three
four five
```

Listing A.16 displays the contents of `MatchAlpha1.sh` that matches text lines that start with alphabetic characters as well as lines that contain numeric strings in the second column.

***LISTING A.16: MatchAlpha1.sh***

```
awk '
{
   if( $0 ~ /^[0-9]/) { print $0 }
   if( $0 ~ /^[a-z]+ [0-9]/) { print $0 }
}
' columns3.txt
```

The output from Listing A.16 is here:

```
123 one two
456 three four
five 123 six
```

## PRINTING LINES USING CONDITIONAL LOGIC

Listing A.17 displays the contents of `products.txt` that contains three columns of information.

***LISTING A.17: products.txt***

```
MobilePhone 400  new
Tablet      300  new
Tablet      300  used
MobilePhone 200  used
MobilePhone 100  used
```

The following code snippet prints the lines of text in `products.txt` whose second column is greater than `300`:

```
awk '$2 > 300' products.txt
```

The output of the preceding code snippet is here:

```
MobilePhone 400  new
```

The following code snippet prints the lines of text in `products.txt` whose product is "new":

```
awk '($3 == "new")' products.txt
```

The output of the preceding code snippet is here:

```
MobilePhone 400  new
Tablet      300  new
```

The following code snippet prints the first and third columns of the lines of text in `products.txt` whose cost equals `300`:

```
awk ' $2 == 300 { print $1, $3 }' products.txt
```

The output of the preceding code snippet is here:

```
Tablet new
Tablet used
```

The following code snippet prints the first and third columns of the lines of text in `products.txt` that start with the string `Tablet`:

```
awk '/^Tablet/ { print $1, $3 }' products.txt
```

The output of the preceding code snippet is here:

```
Tablet new
Tablet used
```

## SPLITTING FILENAMES WITH AWK

Listing A.18 displays the contents of `SplitFilename2.sh` that illustrates how to split a filename containing the "." character in order to increment the numeric value of one of the components of the filename. Note that this code only works for a filename with exactly the expected syntax. It is possible to write more complex code to count the number of segments, or alternately to just say "change the field right before the .zip", which would only require that the filename had a format matching the final two sections (<anystructure>.number.zip).

***LISTING A.18: SplitFilename2.sh***

```
echo "05.20.144q.az.1.zip" | awk -F"." '
{
  f5=$5 + 1
  printf("%s.%s.%s.%s.%s.%s",$1,$2,$3,$4,f5,$6)
}'
```

The output from Listing A.18 is here:

```
05.20.144q.az.2.zip
```

## WORKING WITH POSTFIX ARITHMETIC OPERATORS

Listing A.19 displays the contents of `mixednumbers.txt` that contains postfix operators, which means numbers where the negative (and/or positive) sign appears at the end of a column value instead of the beginning of the number.

***LISTING A.19: mixednumbers.txt***

```
324.000-|10|983.000-
453.000-|30|298.000-
783.000-|20|347.000-
```

Listing A.20 displays the contents of `AddSubtract1.sh` that illustrates how to add the rows of numbers in Listing A.19.

***LISTING A.20: AddSubtract1.sh***

```
myFile="mixednumbers.txt"

awk -F"|" '
BEGIN { line = 0; total = 0 }
{
   split($1, arr, "-")
   f1 = arr[1]
   if($1 ~ /-/) { f1 = -f1 }
   line += f1

   split($2, arr, "-")
   f2 = arr[1]
   if($2 ~ /-/) { f2 = -f2 }
   line += f2

   split($3, arr, "-")
   f3 = arr[1]
   if($3 ~ /-/) { f3 = -f3 }
   line += f3

   printf("f1: %d f2: %d f3: %d line: %d\n",f1,f2,f3, line)
   total += line
   line = 0
}
END { print "Total: ",total }
' $myfile
```

The output from Listing A.20 is here. See if you can work out what the code is doing before reading the explanation that follows:

```
f1: -324 f2: 10 f3: -983 line: -1297
f1: -453 f2: 30 f3: -298 line: -721
f1: -783 f2: 20 f3: -347 line: -1110
Total:  -3128
```

The code assumes we know the format of the file. The split function turns each field record into a length two vector, first position = number, second position either an empty value or a dash, and then captures the first position number into a variable. The if statement just sees if the original field has a dash in it. If the field has a dash, then the numeric variable is made negative, otherwise it is left alone. Then it adds the line up.

## NUMERIC FUNCTIONS IN AWK

The `int(x)` function returns the integer portion of a number. If the number is not already an integer, it falls between two integers. Of the two possible integers, the function will return the one closest to zero. This is different from a rounding function, which chooses the closer integer.

For example, `int(3)` is 3, `int(3.9)` is 3, int(-3.9) is -3, and `int(-3)` is -3 as well. An example of the `int(x)` function in an `awk` command is here:

```
awk 'BEGIN {
   print int(3.534);
   print int(4);
   print int(-5.223);
   print int(-5);
}'
```

The output is here:

```
3
4
-5
-5
```

The `exp(x)` function gives you the exponential of `x`, or reports an error if `x` is out of range. The range of values `x` can have depends on your machine's floating point representation.

```
awk 'BEGIN{
   print exp(123434346);
   print exp(0);
   print exp(-12);
}'
```

The output is here:

```
inf
1
6.14421e-06
```

The `log(x)` function gives you the natural logarithm of `x`, if `x` is positive; otherwise, it reports an error (`inf` means infinity and `nan` in output means "not a number").

```
awk 'BEGIN{
  print log(12);
  print log(0);
  print log(1);
  print log(-1);
}'
```

The output is here:

```
2.48491
-inf
0
nan
```

The `sin(x)` function gives you the sine of `x` and `cos(x)` gives you the cosine of `x`, with `x` in radians:

```
awk 'BEGIN {
   print cos(90);
   print cos(45);
}'
```

The output is here:

```
-0.448074
0.525322
```

The `rand()` function gives you a random number. The values of `rand()` are uniformly distributed between 0 and 1: the value is never 0 and never 1.

Often you want random integers instead. Here is a user-defined function you can use to obtain a random nonnegative integer less than n:

```
function randint(n) {
     return int(n * rand())
}
```

The product produces a random real number greater than 0 and less than n. We then make it an integer (using int) between 0 and n - 1.

Here is an example where a similar function is used to produce random integers between 1 and n:

```
awk '
# Function to roll a simulated die.
function roll(n) { return 1 + int(rand() * n) }
# Roll 3 six-sided dice and print total number of points.
{
      printf("%d points\n", roll(6)+roll(6)+roll(6))
}'
```

Note that `rand` starts generating numbers from the same point (or "seed") each time `awk` is invoked. Hence, a program will produce the same results each time it is launched. If you want a program to do different things each time it is used, you must change the seed to a value that will be different in each run.

Use the `srand(x)` function to set the starting point, or seed, for generating random numbers to the value `x`. Each seed value leads to a particular sequence of "random" numbers. Thus, if you set the seed to the same value a second time, you will get the same sequence of "random" numbers again. If you omit the argument x, as in `srand()`, then the current date and time of day are used for a seed. This is how to obtain random numbers that are truly unpredictable. The return value of `srand()` is the previous seed. This makes it easy to keep track of the seeds for use in consistently reproducing sequences of random numbers.

The `time()` function (not in all versions of `awk`) returns the current time in seconds since January 1, 1970. The function `ctime` (not in all versions of `awk`) takes a numeric argument in seconds and returns a string representing the corresponding date, suitable for printing or further processing.

The `sqrt(x)` function gives you the positive square root of `x`. It reports an error if x is negative. Thus, `sqrt(4)` is 2.

```
awk 'BEGIN{
   print sqrt(16);
   print sqrt(0);
   print sqrt(-12);
}'
```

The output is here:

```
4
0
nan
```

## ONE-LINE AWK COMMANDS

The code snippets in this section reference the text file `short1.txt`, which you can populate with any data of your choice.

The following code snippet prints each line preceded by the number of fields in each line:

```
awk '{print NF ":" $0}' short1.txt
```

Print the rightmost field in each line:

```
awk '{print $NF}' short1.txt
```

Print the lines that contain more than two fields:

```
awk '{if(NF > 2) print }' short1.txt
```

Print the value of the rightmost field if the current line contains more than two fields:

```
awk '{if(NF > 2) print $NF }' short1.txt
```

Remove leading and trailing whitespaces:

```
echo " a b c " | awk '{gsub(/^[ \t]+|[ \t]+$/,"");print}'
```

Print the first and third fields in reverse order for the lines that contain at least three fields:

```
awk '{if(NF > 2) print $3, $1}' short1.txt
```

Print the lines that contain the string one:

```
awk '{if(/one/) print }' *txt
```

As you can see from the preceding code snippets, it's easy to extract information or subsets of rows and columns from text files using simple conditional logic and built-in variables in the `awk` command.

## USEFUL SHORT AWK SCRIPTS

This section contains a set of short `awk` -based scripts for performing various operations. Some of these scripts can also be used in other shell scripts to perform more complex operations. Listing A.21 displays the contents of the file `data.txt` that is used in various code samples in this section.

***LISTING A.21: data.txt***

```
this is line one that contains more than 40 characters
this is line two
this is line three that also contains more than 40 characters
four

this is line six and the preceding line is empty

line eight and the preceding line is also empty
```

The following code snippet prints every line that is longer than forty characters:

```
awk 'length($0) > 40' data.txt
```

Now print the length of the longest line in data.txt:

```
awk '{ if (x < length()) x = length() }
END { print "maximum line length is " x }' < data.txt
```

The input is processed by the expand utility to change tabs into spaces, so the widths compared are actually the right-margin columns.

Print every line that has at least one field:

```
awk 'NF > 0' data.txt
```

The preceding code snippet illustrates an easy way to delete blank lines from a file (or rather, to create a new file similar to the old file but from which the blank lines have been removed).

Print seven random numbers from 0 to 100, inclusive:

```
awk 'BEGIN { for (i = 1; i <= 7; i++)
print int(101 * rand()) }'
```

Count the lines in a file:

```
awk 'END { print NR }' < data.txt
```

Print the even-numbered lines in the data file:

```
awk 'NR % 2 == 0' data.txt
```

If you use the expression `'NR % 2 == 1'` in the previous code snippet, the program will print the odd-numbered lines.

Insert a duplicate of every line in a text file:

```
awk '{print $0, '\n', $0}' < data.txt
```

Insert a duplicate of every line in a text file and also remove blank lines:

```
awk '{print $0, "\n", $0}' < data.txt | awk 'NF > 0'
```

Insert a blank line after every line in a text file:

```
awk '{print $0, "\n"}' < data.txt
```

## PRINTING THE WORDS IN A TEXT STRING IN AWK

Listing A.22 displays the contents of `Fields2.sh` that illustrates how to print the words in a text string using the `awk` command.

***LISTING A.22: Fields2.sh***

```
echo "a b c d e"| awk '
{
  for(i=1; i<=NF; i++) {
     print "Field ",i,":",$i
  }
}
'
```

The output from Listing A.22 is here:

```
Field  1 : a
Field  2 : b
Field  3 : c
Field  4 : d
Field  5 : e
```

## COUNTING OCCURRENCES OF A STRING IN SPECIFIC ROWS

Listing A.23 and Listing A.24 display the contents `data1.csv` and `data2.csv`, respectively, and Listing A.25 displays the contents of `checkrows.sh` that illustrates how to count the number of occurrences of the string "past" in column 3 in rows 2, 5, and 7.

***LISTING A.23: data1.csv***

```
in,the,past,or,the,present
for,the,past,or,the,present
in,the,past,or,the,present
for,the,paste,or,the,future
in,the,past,or,the,present
completely,unrelated,line1
in,the,past,or,the,present
completely,unrelated,line2
```

***LISTING A.24: data2.csv***

```
in,the,past,or,the,present
completely,unrelated,line1
for,the,past,or,the,present
completely,unrelated,line2
for,the,paste,or,the,future
in,the,past,or,the,present
in,the,past,or,the,present
completely,unrelated,line3
```

***LISTING A.25: checkrows.sh***

```
files="`ls data*.csv| tr '\n' ' '`"
echo "List of files: $files"

awk -F"," '
( FNR==2 || FNR==5 || FNR==7 ) {
    if ( $3 ~ "past" ) { count++ }
}
END {
    printf "past: matched %d times (INEXACT) ", count
    printf "in field 3 in lines 2/5/7\n"
}' data*.csv
```

Listing A.25 looks for occurrences in the string `past` in columns 2, 5, and 7 because of the following code snippet:

```
( FNR==2 || FNR==5 || FNR==7 ) {
    if ( $3 ~ "past" ) { count++ }
}
```

If a match occurs, then the value of count is incremented. The `END` block reports the number of times that the string past was found in columns 2, 5, and 7. Note that strings such as `paste` and `pasted` will match the string past. The output from Listing A.25 is here:

```
List of files: data1.csv data2.csv
past: matched 5 times (INEXACT) in field 3 in lines 2/5/7
```

The shell script `checkrows2.sh` replaces the term `$3 ~ "past"` with the term `$3 == "past"` in `checkrows.sh` in order to check for exact matches, which produces the following output:

```
List of files: data1.csv data2.csv
past: matched 4 times (EXACT) in field 3 in lines 2/5/7
```

## PRINTING A STRING IN A FIXED NUMBER OF COLUMNS

Listing A.26 displays the contents of `FixedFieldCount1.sh` that illustrates how to print the words in a text string using the `awk` command.

***LISTING A.26: FixedFieldCount1.sh***

```
echo "aa bb cc dd ee ff gg hh"| awk '
BEGIN { colCount = 3 }
{

  for(i=1; i<=NF; i++) {
     printf("%s ", $i)
     if(i % colCount == 0) {
        print " "
     }
  }
}
'
```

The output from Listing A.26 is here:

```
aa bb cc
dd ee ff
gg hh
```

## PRINTING A DATASET IN A FIXED NUMBER OF COLUMNS

Listing A.27 displays the contents of `VariableColumns.txt` with lines of text that contain a different number of columns.

***LISTING A.27: VariableColumns.txt***

```
this is line one
this is line number one
this is the third and final line
```

Listing A.28 displays the contents of `Fields3.sh` that illustrates how to print the words in a text string using the `awk` command.

***LISTING A.28: Fields3.sh***

```
awk '{printf("%s ", $0)}' | awk '
BEGIN { columnCount = 3 }
{
  for(i=1; i<=NF; i++) {
     printf("%s ", $i)
     if( i % columnCount == 0 )
       print " "
  }
}
' VariableColumns.txt
```

The output from Listing A.28 is here:

```
this is line
one this is
```

```
line number one
this is the
third and final
line
```

## ALIGNING COLUMNS IN DATASETS

If you have read the preceding two examples, the code sample in this section is easy to understand: you will see how to realign columns of data that are correct in terms of their content but have been placed in different rows (and therefore are misaligned). Listing A.29 displays the contents of `mixed-data.txt` with misaligned data values. In addition, the first line and final line in Listing A.29 are empty lines, which will be removed by the shell script in this section.

***LISTING A.29: mixed-data.txt***

```
Sara, Jones, 1000, CA, Sally, Smith, 2000, IL,
Dave, Jones, 3000, FL, John, Jones,
4000, CA,
Dave, Jones, 5000, NY, Mike,
Jones, 6000, NY, Tony, Jones, 7000, WA
```

Listing A.30 displays the contents of `mixed-data.sh` that illustrates how to realign the dataset in Listing A.29.

***LISTING A.30: mixed-data.sh***

```
#----------------------------------------
# 1) remove blank lines
# 2) remove line feeds
# 3) print a LF after every fourth field
# 4) remove trailing ',' from each row
#----------------------------------------

inputfile="mixed-data.txt"
outputfile="aligned-data.txt"

grep -v "^$" $inputfile |awk -F"," '{printf("%s",$0)}' | awk '
BEGIN { columnCount = 4 }
{
   for(i=1; i<=NF; i++) {
     printf("%s ", $i)
     if( i % columnCount  == 0) { print "" }
   }
}' > temp-columns

# 4) remove trailing ',' from output:
cat temp-columns | sed 's/, $//' | sed 's/ $//' > $outputfile
```

Listing A.30 starts with a `grep` command that removes blank lines, followed by an `awk` command that prints the rows of the dataset as a single line of text. The second `awk` command initializes the `columnCount` variable with the value 4 in the `BEGIN` block, followed by a `for` loop that

iterates through the input fields. After four fields are printed on the same output line, a linefeed is printed, which has the effect of realigning the input dataset as an output dataset consisting of rows that have four fields. The output from Listing A.30 is here:

```
Sara, Jones, 1000, CA
Sally, Smith, 2000, IL
Dave, Jones, 3000, FL
John, Jones, 4000, CA
Dave, Jones, 5000, NY
Mike, Jones, 6000, NY
Tony, Jones, 7000, WA
```

## ALIGNING COLUMNS AND MULTIPLE ROWS IN DATASETS

The preceding section showed you how to realign a dataset so that each row contains the same number of columns and also represents a single data record. The code sample in this section illustrates how to realign columns of data that are correct in terms of their content and also place two records in each line of the new dataset. Listing A.31 displays the contents of `mixed-data2.txt` with misaligned data values, followed by Listing A.32 that displays the contents of `aligned-data2.csv` with the correctly formatted dataset.

***LISTING A.31: mixed-data2.txt***

```
Sara, Jones, 1000, CA, Sally, Smith, 2000, IL,
Dave, Jones, 3000, FL, John, Jones,
4000, CA,
Dave, Jones, 5000, NY, Mike,
Jones, 6000, NY, Tony, Jones, 7000, WA
```

***LISTING A.32: aligned-data2.csv***

```
Sara, Jones, 1000, CA, Sally, Smith, 2000, IL
Dave, Jones, 3000, FL, John, Jones, 4000, CA
Dave, Jones, 5000, NY, Mike, Jones, 6000, NY
Tony, Jones, 7000, WA
```

Listing A.33 displays the contents of `mixed-data2.sh` that illustrates how to realign the dataset in Listing A.31.

***LISTING A.33: mixed-data2.sh***

```
#----------------------------------------
# 1) remove blank lines
# 2) remove line feeds
# 3) print a LF after every 8 fields
# 4) remove trailing ',' from each row
#----------------------------------------
inputfile="mixed-data2.txt"
outputfile="aligned-data2.txt"

grep -v "^$" $inputfile |awk -F"," '{printf("%s",$0)}' | awk '
BEGIN { columnCount = 4; rowCount = 2; currRow = 0 }
```

```
{
   for(i=1; i<=NF; i++) {
     printf("%s ", $i)
     if( i % columnCount == 0) { ++currRow }
     if(currRow > 0 && currRow % rowCount == 0) {currRow = 0; print ""}
   }
}' > temp-columns

# 4) remove trailing ',' from output:
cat temp-columns | sed 's/, $//' | sed 's/ $//' > $outputfile
```

Listing A.33 is very similar to Listing A.30. The key idea is to print a linefeed character after a pair of "normal" records have been processed, which is implemented via the code that is shown in bold in Listing A.33.

Now you can generalize Listing A.33 very easily by changing the initial value of the `rowCount` variable to any other positive integer, and the code will work correctly without any further modification. For example, if you initialize `rowCount` to the value `5`, then every row in the new dataset (with the possible exception of the final output row) will contain 5 "normal" data records.

## REMOVING A COLUMN FROM A TEXT FILE

Listing A.34 displays the contents of `VariableColumns.txt` with lines of text that contain a different number of columns.

***LISTING A.34: VariableColumns.txt***

```
this is line one
this is line number one
this is the third and final line
```

Listing A.35 displays the contents of `RemoveColumn.sh` that removes the first column from a text file.

***LISTING A.35: RemoveColumn.sh***

```
awk '{ for (i=2; i<=NF; i++) printf "%s ", $i; printf "\n"; }' products.
txt
```

The loop is between 2 and `NF`, which iterates over all the fields except for the first field. In addition, `printf` explicitly adds newlines. The output of the preceding code snippet is here:

```
400 new
300 new
300 used
200 used
100 used
```

## SUBSETS OF COLUMN-ALIGNED ROWS IN DATASETS

Listing A.35 showed you how to align the rows of a dataset, and the code sample in this section illustrates how to extract a subset of the existing columns and a subset of the rows.

Listing A.36 displays the contents of `sub-rows-cols.txt` of the desired dataset that contains two columns from every even row of the file `aligned-data.txt`.

***LISTING A.36: sub-rows-cols.txt***

```
Sara, 1000
Dave, 3000
Dave, 5000
Tony, 7000
```

Listing A.37 displays the contents of `sub-rows-cols.sh` that illustrates how to generate the dataset in Listing A.36. Most of the code is the same as Listing A.33, with the new code shown in bold.

***LISTING A.37: sub-rows-cols.sh***

```
#----------------------------------------
# 1) remove blank lines
# 2) remove line feeds
# 3) print a LF after every fourth field
# 4) remove trailing ',' from each row
#----------------------------------------

inputfile="mixed-data.txt"

grep -v "^$" $inputfile |awk -F"," '{printf("%s",$0)}' | awk '
BEGIN { columnCount = 4 }
{
   for(i=1; i<=NF; i++) {
     printf("%s ", $i)
     if( i % columnCount  == 0) { print "" }
   }
}' > temp-columns

# 4) remove trailing ',' from output:
cat temp-columns | sed 's/, $//' | sed 's/$//' > temp-columns2

cat temp-columns2 | awk '
BEGIN { rowCount = 2; currRow = 0 }
{
   if(currRow % rowCount == 0) { print $1, $3 }
   ++currRow
}' > temp-columns3

cat temp-columns3 | sed 's/,$//' | sed 's/ $//' > $outputfile
```

Listing A.37 contains a new block of code that redirects the output of step 4 to a temporary file `temp-columns2` whose contents are processed by another `awk` command in the last section of Listing A.37. Notice that that `awk` command contains a `BEGIN` block that initializes the variables `rowCount` and `currRow` with the values 2 and 0, respectively.

The main block prints columns 1 and 3 of the current line if the current row number is even, and then the value of `currRow` is incremented. The output of this `awk` command is redirected

to yet another temporary file that is the input to the final code snippet, which uses the `cat` command and two occurrences of the `sed` command in order to remove a trailing "," and a trailing space, as shown here:

```
cat temp-columns3 | sed 's/,$//' | sed 's/ $//' > $outputfile
```

Keep in mind that there are other ways to perform the functionality in Listing A.37, and the main purpose is to show you different techniques for combining various bash commands.

## COUNTING WORD FREQUENCY IN DATASETS

Listing A.38 displays the contents of `WordCounts1.sh` that illustrates how to count the frequency of words in a file.

***LISTING A.38: WordCounts1.sh***

```
awk '
# Print list of word frequencies
{
    for (i = 1; i <= NF; i++)
        freq[$i]++
}
END {
    for (word in freq)
        printf "%s\t%d\n", word, freq[word]
}
' columns2.txt
```

Listing A.38 contains a block of code that processes the lines in `columns2.txt`. Each time that a word (of a line) is encountered, the code increments the number of occurrences of that word in the hash table `freq`. The `END` block contains a `for` loop that displays the number of occurrences of each word in `columns2.txt`.

The output from Listing A.38 is here:

```
two     3
one     3
three   3
six     1
four    3
five    2
```

Listing A.39 displays the contents of `WordCounts2.sh` that perform a case insensitive word count.

***LISTING A.39: WordCounts2.sh***

```
awk '
{
    # convert everything to lower case
    $0 = tolower($0)

    # remove punctuation
```

```
   #gsub(/[^[:alnum:]_[:blank:]]/, "", $0)

    for(i=1; i<=NF; i++) {
       freq[$i]++
    }
}
END {
    for(word in freq) {
       printf "%s\t%d\n", word, freq[word]
    }
}
' columns4.txt
```

Listing A.39 is almost identical to Listing A.38, with the addition of the following code snippet that converts the text in each input line to lowercase letters, as shown here:

```
$0 = tolower($0)
```

Listing A.40 displays the contents of `columns4.txt`.

***LISTING A.40: columns4.txt***

```
123 ONE TWO
456 three four
ONE TWO THREE FOUR
five 123 six
one two three
four five
```

The output from launching Listing A.39 with `columns4.txt` is here:

```
456       1
two       3
one       3
three     3
six       1
123       2
four      3
five      2
```

## DISPLAYING ONLY "PURE" WORDS IN A DATASET

For simplicity, let's work with a text string and that way we can see the intermediate results as we work toward the solution. Listing A.41 displays the contents of `onlywords.sh` that contains three `awk` commands for displaying the words, integers, and alphanumeric strings, respectively, in a text string.

***LISTING A.41: onlywords.sh***

```
x="ghi abc Ghi 123 #def5 123z"

echo "Only words:"
echo $x |tr -s ' ' '\n' | awk -F" " '
```

```
{
  if($0 ~ /^[a-zA-Z]+$/) { print $0 }
}
' | sort | uniq
echo

echo "Only integers:"
echo $x |tr -s ' ' '\n' | awk -F" " '
{
  if($0 ~ /^[0-9]+$/) { print $0 }
}
' | sort | uniq
echo

echo "Only alphanumeric words:"
echo $x |tr -s ' ' '\n' | awk -F" " '
{
  if($0 ~ /^[0-9a-zA-Z]+$/) { print $0 }
}
' | sort | uniq
echo
```

Listing A.41 starts by initializing the variable x:

```
x="ghi abc Ghi 123 #def5 123z"
```

The next step is to split x into words:

```
echo $x |tr -s ' ' '\n'
```

The output is here:

```
ghi
abc
Ghi
123
#def5
123z
```

The third step is to invoke `awk` and check for words that match the regular expression `^[a-zA-Z]+`, which matches any string consisting of one or more uppercase and/or lowercase letters (and nothing else):

```
if($0 ~ /^[a-zA-Z]+$/) { print $0 }
```

The output is here:

```
ghi
abc
Ghi
```

Finally, if you also want to sort the output and print only the unique words, redirect the output from the `awk` command to the `sort` command and the `uniq` command.

The second `awk` command uses the regular expression `^[0-9]+` to check for integers and the third `awk` command uses the regular expression `^[0-9a-zA-Z]+` to check for alphanumeric words. The output from launching Listing A.41 is here:

```
Only words:
Ghi
abc
ghi

Only integers:
123

Only alphanumeric words:
123
123z
Ghi
abc
ghi
```

Now you can replace the variable `x` with a dataset in order to retrieve only alphabetic strings from that dataset.

## WORKING WITH MULTILINE RECORDS IN AWK

Listing A.42 displays the contents of `employee.txt` and Listing A.43 displays the contents of `Employees.sh` that illustrates how to concatenate text lines in a file.

***LISTING A.42: employees.txt***

```
Name:  Jane Edwards
EmpId: 12345
Address: 123 Main Street Chicago Illinois

Name:  John Smith
EmpId: 23456
Address: 432 Lombard Avenue SF California
```

***LISTING A.43: employees.sh***

```
inputfile="employees.txt"
outputfile="employees2.txt"

awk '
{
  if($0 ~ /^Name:/) {
    x = substr($0,8) ","
    next
  }

  if( $0 ~ /^Empid:/) {
```

```
   #skip the Empid data row
   #x = x substr($0,7) ","
    next
  }

  if($0 ~ /^Address:/) {
    x = x substr($0,9)
    print x
  }
}
' < $inputfile > $outputfile
```

The output from launching the code in Listing A.43 is here:

```
Jane Edwards, 123 Main Street Chicago Illinois
John Smith, 432 Lombard Avenue SF California
```

Now that you have seen a plethora of `awk` code snippets and shell scripts containing the `awk` command that illustrate various type of tasks that you can perform on files and datasets, you are ready for some use cases. The next section (which is the first use case) shows you how to replace multiple field delimiters with a single delimiter, and the second use case shows you how to manipulate date strings.

## A SIMPLE USE CASE

The code sample in this section shows you how to use the `awk` command in order to split the comma-separated fields in the rows of a dataset, where fields can contain nested quotes of arbitrary depth.

Listing A.44 displays the contents of the file `quotes3.csv` that contains a "," delimiter and multiple quoted fields.

***LISTING A.44: quotes3.csv***

```
field5,field4,field3,"field2,foo,bar",field1,field6,field7,"fieldZ"
fname1,"fname2,other,stuff",fname3,"fname4,foo,bar",fname5
"lname1,a,b","lname2,c,d","lname3,e,f","lname4,foo,bar",lname5
```

Listing A.45 displays the contents of the file `delim1.sh` that illustrates how to replace the delimiters in `delim1.csv` with a "," character.

***LISTING A.45 delim1.sh***

```
#inputfile="quotes1.csv"
#inputfile="quotes2.csv"
inputfile="quotes3.csv"

grep -v "^$" $inputfile |  awk '
{
   print "LINE #" NR ": " $0
   printf ("-------------------------\n")
```

```
   for (i = 0; ++i <= NF;)
     printf "field #%d : %s\n", i, $i
   printf ("\n")
}' FPAT='([^,]+)|("[^"]+")' < $inputfile
```

The output from launching the shell script in Listing A.44 is here:

```
LINE #1:
field5,field4,field3,"field2,foo,bar",field1,field6,field7,"fieldZ"
-------------------------
field #1 : field5
field #2 : field4
field #3 : field3
field #4 : "field2,foo,bar"
field #5 : field1
field #6 : field6
field #7 : field7
field #8 : "fieldZ"

LINE #2: fname1,"fname2,other,stuff",fname3,"fname4,foo,bar",fname5
-------------------------
field #1 : fname1
field #2 : "fname2,other,stuff"
field #3 : fname3
field #4 : "fname4,foo,bar"
field #5 : fname5

LINE #3: "lname1,a,b","lname2,c,d","lname3,e,f","lname4,foo,bar",lname5
-------------------------
field #1 : "lname1,a,b"
field #2 : "lname2,c,d"
field #3 : "lname3,e,f"
field #4 : "lname4,foo,bar"
field #5 : lname5

LINE #4: "Outer1 "Inner "Inner "Inner C" B" A" Outer1","XYZ1,c,d","XYZ2l
name3,e,f"
-------------------------
field #1 : "Outer1 "Inner "Inner "Inner C" B" A" Outer1"
field #2 : "XYZ1,c,d"
field #3 : "XYZ2lname3,e,f"

LINE #5:
-------------------------
```

As you can see, the task in this section is very easily solved via the `awk` command.

## ANOTHER USE CASE

The code sample in this section shows you how to use the `awk` command in order to reformat the date field in a dataset and change the order of the fields in the new dataset. For example, given the following input line in the original dataset:

```
Jane,Smith,20140805234658
```

The reformatted line in the output dataset has this format:

```
2014-08-05 23:46:58,Jane,Smith
```

Listing A.46 displays the contents of the file `dates2.csv` that contains a "," delimiter and three fields.

***LISTING A.46 dates2.csv***

```
Jane,Smith,20140805234658
Jack,Jones,20170805234652
Dave,Stone,20160805234655
John,Smith,20130805234646
Jean,Davis,20140805234649
Thad,Smith,20150805234637
Jack,Pruit,20160805234638
```

Listing A.47 displays the contents of `string2date2.sh` that converts the date field to a new format and shifts the new date to the first field.

***LISTING A.47: string2date2.sh***

```
inputfile="dates2.csv"
outputfile="formatteddates2.csv"

rm -f $outputfile; touch $outputfile

for line in 'cat $inputfile'
do
  fname='echo $line |cut -d"," -f1'
  lname='echo $line |cut -d"," -f2'
  date1='echo $line |cut -d"," -f3'

  # convert to new date format
  newdate='echo $date1 | awk '{ print substr($0,1,4)"-"substr($0,5,2)"-
"substr($0,7,2)" "substr($0,9,2)":"substr($0,11,2)":"substr($0,13,2)}''

  # append newly formatted row to output file
  echo "${newdate},${fname},${lname}" >> $outputfile
done
```

The content of the new dataset is here:

```
2014-08-05 23:46:58,Jane,Smith
2017-08-05 23:46:52,Jack,Jones
2016-08-05 23:46:55,Dave,Stone
2013-08-05 23:46:46,John,Smith
2014-08-05 23:46:49,Jean,Davis
2015-08-05 23:46:37,Thad,Smith
2016-08-05 23:46:38,Jack,Pruit
```

## SUMMARY

This appendix introduced the `awk` command, which is essentially an entire programming language packaged into a single Unix command.

We explored some of its built-in variables as well as conditional logic, while loops, and `for` loops in `awk` in order to manipulate the rows and columns in datasets. You then saw how to delete lines and merge lines in datasets, and also how to print the contents of a file as a single line of text. Next you learned how to use meta characters and character sets in `awk` commands. You learned how to perform numeric calculations (such as addition, subtraction, multiplication, and division) in files containing numeric data, and also some numeric functions that are available in `awk`.

In addition, you saw how to align columns in a dataset, how to delete columns, how to select a subset of columns from a dataset, and how to work with multiline records in datasets. Finally, you saw a couple of simple use cases involving nested quotes and date formats in a structured dataset.

At this point you have all the tools necessary to do quite sophisticated data cleansing and processing, and it is strongly encouraged that you try to apply them on some task or problem of interest. The final step of the learning process is doing something real.

"I saw something similar once, I wonder if there is a way to . . .?" or the even more common "how do I do XXX in language YYY?" You can't ask those questions if you don't have a sense of what is possible.

At this point there is one more thing to say: congratulations! You have completed a fast-paced yet dense book, and if you are a bash neophyte, the material will probably keep you busy for many hours. The combined effect demonstrates that the universe of possibilities is larger than the examples in this book, and ultimately they will spark ideas in you. Good luck!

# INDEX


**A**

Anomaly detection
  Density-based spatial clustering of applications with noise (DBSCAN), 50
  with histogram-based outlier score (HBOS), 50
  isolation forest, 50
  local outlier factor (LOF) technique, 49
  numeric outlier, 50
  Python outlier detection (PyOD), 50
  Z-score, 50
Autoregressive conditional heteroscedasticity (ARCH) modeling technique, 21
`awk` (Aho, Weinberger, and Kernighan)
  built-in variables, 180
  conditional logic and control statements
    `for` loop, 184
    `next` and `continue` statements, 184
    printing lines, 189–190
    `while` loop, 182–183
  count occurrences of string, 196–197
  data cleaning, 64–66
  datasets
    aligning columns in, 199–201
    count the frequency of words, 203–204
    delete lines in, 184–185
    displaying the words, 204–206
    merge lines in, 185–188
    subsets of column-aligned rows in, 201–203
  data wrangling, 97
  meta characters and character sets, 188–189
  multiline records in, 206–207
  numeric functions, 191–194
  one-line commands, 194
  postfix arithmetic operators, 190–191
  `printf` command, 181–182
  print words in text string, 196
  short `awk` -based scripts, 195–196
  splitting filenames, 190
  use case, 207–209
  working procedure, 180–181

**B**

Bayesian inference, 175–176
Bayes' theorem, 176, 177
Binning data
  Doane's formula, 17
  Freedman-Diaconis's choice, 17
  potential issues, 18
  Sturges's rule, 18
  techniques for, 17
Black-box shift detector, 14

**C**

Categorical data
  mapping to numeric values, 19–20
  processing inconsistent data values, 18–19
Collinearity, 21–22
Correlation, 22
Correlation matrix, 157
Covariance matrix, 155–157

**D**
Data augmentation techniques, 152–154
Data cleaning
  alphabetic date format conversion, 72–74
  awk utility, 64–66
  with countries, cities, and telephone codes, 80–85
  date and time date formats, 74–80
  definition, 51–52
  handle mismatched attribute values, 57–58
  on a kaggle dataset, 86–89
  numeric date format conversion, 68–72
  for person's title, 52–53
  phone number conversion, 66–68
  replacing multiple strings, 56–57
  sed utility, 60–62
  SMOTE technique, 52
  in SQL, 54–56
  string to date conversion, 58–60
  truncate the rows in a CSV file, 63–64
  variable column counts, 62–63
Data drift, 14
Data leakage, 14–15
Data preprocessing, 12–13
Data sampling techniques
  oversampling, 151–152
  resampling, 152
  undersampling, 151
Datasets, 8–9
  awk
    aligning columns in, 199–201
    count the frequency of words, 203–204
    delete lines in, 184–185
    displaying the words, 204–206
    merge lines in, 185–188
    subsets of column-aligned rows in, 201–203
  feature scaling, 34
  imbalanced (*see* Imbalanced datasets)
  normalization
    min-max normalization, 31
    scaling numeric data, 31–32
  robust standardization technique, 34
  standardization technique, 32–33
  StandardScaler class, 33–34
  Titanic dataset, 2–6, 146
  working with currency, 22–26
  working with dates, 26–30
Data types
  binary data, 9
  categorical data, 10
  continuous *vs.* discrete data, 10–11
  interval data, 10
  in machine learning, 13
  nominal data, 9–10
  ordinal data, 10
  qualitative and quantitative data, 11
  random variables, 11
  ratio data, 10
  statistical data, types of, 11–12
Data wrangling
  awk utility, 97
  band-related information in CSV format
    Java, 109–113
    Python, 113–115
  calendar of events (COE), 115–118
  city_codes table, SQL statements, 108–109
  CSV files with multi-row records, 93–96
  data transformation, 92–93
  description, 91–92
  events project, 100–102
  generating city codes from country codes
    awk, 103–106
    Python, 106–108
  project automation script, 118–121
Density-based spatial clustering of applications with noise (DBSCAN), 50
Dimensionality reduction technique
  curse of dimensionality, 163
  feature extraction and, 162
  linear *vs.* nonlinear, 166–167
  LLE, 165
  manifold learning, 163–164
  PHATE, 166
  SVD, 164–165
  t-SNE, 165–166
  UMAP, 165
  VAEs, 162
Discrete *vs.* continuous data types, 16
Distance metrics, 167–168
Domain classifier, 14

**E**
Eigenvalues and eigenvectors, 157–158
Exploratory data analysis (EDA)
  data wrangling, 2
  and histograms, 6–7
  methods for data analysis, 2
  purpose of, 2
  Titanic dataset, 2–6
  types of, 2
  visualization techniques, 2

**F**
Feature engineering, 147
  binning, 143
  goals, 141
  one-hot encoding, 143
  types of, 142
Feature extraction
  advantages of, 147
  algorithms in machine learning, 148
  disadvantages of, 147
Feature hashing, 148–150
Feature importance
  conditions for, 145
  random forest algorithm in `Sklearn`, 145
  `random_forest.fit()` method, 146
  Titanic dataset, 146
Feature scaling, 150
Feature selection
  conditions for feature importance, 133–134
  embedded methods
    decision trees (and tree-based models), 128
    elastic net, 128
    LightGBM, 128–129
    L1 regularization (Lasso), 127–128
    recursive feature elimination, 129
  filter methods
    ANOVA F-test, 124–125
    chi-squared test, 124
    correlation coefficient, 125
    mutual information, 125
    variance threshold, 124
  goal, 123
  imbalanced datasets
    data splits, 132
    detection, 131–132
    random oversampling, 131
    random resampling and undersampling technique, 131
    rebalancing datasets, 132
    `SMOTE` technique, 131
  labeled *vs.* unlabeled data, 130
  scaling and transformations, 129–130
  wrapper methods
    backward feature elimination, 126–127
    Boruta, 127
    recursive feature elimination (RFE), 126
    recursive feature elimination with cross-validation (RFECV), 126
    sequential feature selection (SFS), 126

**G**
Gauss-Jordan elimination technique, 158–159
Google Colaboratory, 144

**H**
Heteroskedasticity, 21
Histogram-based outlier score (HBOS), 50
Homoskedasticity, 20
Huber and the Ridge classes, 39
Hyperparameter tuning, 144

**I**
Imbalanced datasets, 145
  data splits, 132
  detection, 131–132
  random oversampling, 131
  random resampling and undersampling technique, 131
  rebalancing datasets, 132
  `SMOTE` technique, 131
IQR (interquartile range) algorithm, 38

**J**
Jaccard similarity, 169–170

**L**
Locally linear embedding (LLE), 165
Local outlier factor (LOF) technique, 49
Local sensitivity hashing (LSH) algorithm, 170

**M**
Machine learning algorithms, 151
Maximum a posteriori (MAP) hypothesis, 177
Minimum Covariance Determinant, 38
Model selection, 15–16
Multicollinearity, 22

**N**
New matrix of eigenvectors, 161–162
Numeric outlier, 50

**O**
Outliers
  Carling median rule, 40
  *vs.* data drift, 37–38
  detection/removal, 38–39
  fraud detection, 48–49
  incorrectly scaled values *vs.*, 39
  MAD technique, 40

modified z-score, 40
NumPy methods, 40–42
Pandas, 42–45
with sklearn, 47–48
Tukey's boxplot, 40
types of, 37
z-score calculation, 45–47

**P**
PCA (principal component analysis), 159–161
Pearson correlation coefficient, 169
Potential of heat-diffusion affinity-based transition embedding (PHATE), 166
Python outlier detection (PyOD), 50

**S**
Singular value decomposition (SVD), 164–165
Sklearn, 170–175
SMOTE (synthetic minority oversampling technique)
alternative to, 135–136
extensions, 135

**T**
t-distributed stochastic neighbor embedding (t-SNE), 165–166
Training of a model, steps for, 143–144
Transforms
cube root transformation, 137–138
exponential transformation, 138
hyperbolic transformation, 138
logarithmic transformation, 139
power transformation, 139
square root transformation, 138
Trimming, 38

**U**
Uniform manifold approximation (UMAP), 165

**V**
Variance inflation factor (VIF), 22
Vector space, 177–178

**W**
Well-known distance metrics, 167, 169
Winsorizing, 38

**Z**
Z-score, 38, 50